

Informe Final del Proyecto de Software Orientado a la Web Viverapp

Aprendiz

Juan Rodríguez

Samuel Linares

Daniel Díaz

Jimena Hernández

Instructor

Angélica Triana

Servicio Nacional de Aprendizaje

Análisis y Desarrollo de

Software Ficha 20251

19 de agosto de 2025

Resumen(ultimas)

Se desarrollará un software web para gestionar integralmente ventas, domicilios y servicios de hotel de plantas en un vivero. El proceso de desarrollo comenzó con el análisis de necesidades del vivero y la definición de los requerimientos del sistema. Se diseñó una arquitectura web escalable y una interfaz amigable. Luego, se desarrollaron módulos para ventas, domicilios, hotel de plantas e inventario. Se integraron funciones para el seguimiento en tiempo real y notificaciones automáticas. Posteriormente, se realizaron pruebas funcionales y de usabilidad. Se corrigieron errores detectados y se optimizó el rendimiento. Finalmente, se implementó el sistema, se capacitó al personal y se puso en marcha con monitoreo continuo para asegurar su correcto funcionamiento.

***Palabras clave:* Vivero, software, hotel plantas.**

Abstract(ultimas)

Web-based software will be developed to comprehensively manage sales, delivery, and plant hotel services at a nursery. The development process began with an analysis of the nursery's needs and the definition of system requirements. A scalable web architecture and a user-friendly interface were designed. Modules for sales, delivery, plant hotel, and inventory were then developed. Features for real-time tracking and automatic notifications were integrated. Functional and usability testing was subsequently conducted. Detected errors were corrected and performance optimized. Finally, the system was implemented, staff trained, and commissioned with continuous monitoring to ensure proper operation.

Keywords: Nursery, software, plant hotel.

Tabla de Contenido

Introducción	11
Planteamiento del Problema	13
Descripción del Problema	13
Objetivo General	14
Objetivos Específicos	14
Justificación del Proyecto	15
Alcance del Proyecto	16
Matriz de Riesgo	17
Elicitación de Requisitos	19
Identificación de Procesos	19
Recolección de la Información del Software a Construir de acuerdo con las Necesidades del Cliente .	
22 Elección de la Técnica de Recolección de la Información	22
Diseño de los Formatos, según la Técnica o Técnicas de Recolección de la Información Seleccionada(s)	22
Aplicación de la Técnica de Recolección de la Información	29
Resultados de Entrevista Semiestructurada (síntesis)	29
Resultados de Encuesta a Clientes	30
Resultados de Observación Directa	31
Organización de la Información Recolectada	32
Especificación de Requerimientos, de acuerdo con la Información Recolectada, Aplicando Estándares de Requerimientos	35
Requerimientos Funcionales	35
Requerimientos No Funcionales	37
Requerimientos Normativos	39
Propuesta Técnica	40
Reglas del Negocio	43
Análisis de la Especificación de Requisitos del Software	45
Alternativas de Solución (prototipo o prototipos del sistema, mockups)	45
Diagrama de Casos de Uso y Extensibilidad	47
Diagramas de Actividades, y Secuencias	52
Construir el Modelo de Dominio del Sistema	56
El modelo entidad relación	56

Diseño de la Solución del Software de acuerdo con los Procedimientos y Requisitos Técnicos	58
Arquitectura del software (diagrama de componentes), y patrones de diseño de software	58
Diagrama de despliegue	59
Diseño Front-End - Interfaces Gráficas de Usuario	60
Interfaces Gráficas de Usuario Móviles	65
Mapa de Navegación	68
Determinar Tipos de Bases de Datos	70
Modelo de Datos Diagrama ER Normalización base de datos (tercera forma normal)	70
Diccionario de Datos	72
Políticas de Seguridad de los Datos	74
Protección de acceso	75
Confidencialidad	75
Integridad	75
Disponibilidad	75
Privacidad	75
Seguridad en las comunicaciones	76
Gestión de incidentes	76
Control de copias de seguridad	76
Construcción del Software	77
Tabla Usuarios	77
Tabla Clientes	77
Tabla Productos / Inventario	78
Tabla Ventas	78
Tabla Detalle de Ventas	78
Tabla Domicilios	79
Tabla Hotel de Plantas	79
Tabla Detalle de Hotel de Plantas	80
Objetos de la Base de Datos (procedimientos almacenados, vistas, disparadores)	80
Esquemas de Seguridad de los Datos	83
Codificación del Software de acuerdo con el Diseño Establecido	85
Front End	85
Estándar de Codificación	94
Convenciones de Nombres	94

Código Fuente de los Módulos del Software Web y Móvil	96
Servicios Web	109
Control de Versiones	111
Pruebas del Software	114
Planeación y diseño de la Pruebas a Nivel Unitario(módulos), con base en los Requerimientos Funcionales y no Funcionales	114
Requerimientos Funcionales para Validar	114
Requerimientos No Funcionales para Validar	115
Diseño de las Pruebas Unitarias	115
Camino básico	116
Ejecución de las Pruebas, Informe de Hallazgos	117
Corrección de Errores, y Documentación de Estos	119
Correcciones Realizadas	119
Evidencia de Corrección	120
Manejo de Alertas / Excepciones	121
Planeación y Diseño de las Pruebas a Nivel Sistema, con Base en los Requerimientos Funcionales y No Funcionales	122
Requerimientos Funcionales para Validar	122
Ejecución de las Pruebas, Informe de Hallazgos	125
Corrección de Errores, y Documentación de Estos	126
Correcciones Realizadas	126
Plan de Despliegue	128
Hosting	128
Dominio y Subdominio	128
Costos estimados	129
Implantación del Software	130
Plan de Capacitación de Usuarios del Sistema	131
Manual del Usuario	131
Acceso al Sistema	131
Módulos del Sistema	132
Copias de Seguridad de Datos y Respaldos	133
Tipos de Copias de Seguridad	133

Frecuencia de Respaldos	134
Medios de Almacenamiento	134
Política de Retención	134
Restauración de Datos	134
Acuerdos de Nivel de Usuario	135
Adopción de Buenas Prácticas en el Proceso de Desarrollo de Software	137
Conclusión	139
Glosario	140
Bibliografía	142
Apéndices	143

Lista de Figuras

Figura 1 Mapa de procesos	21
Figura 2 Mockup dashboard de ventas	45
Figura 3 Mockup del hotel de plantas	46
Figura 4 Mockup de las ordenes	46
Figura 5 Mockup de gestión de usuarios del administrador	47
Figura 6 Casos de uso sistema general	47
Figura 7 Casos de uso modulo hotel de plantas	48
Figura 8 Casos de uso de domicilios	49
Figura 9 Casos de uso módulo de inventario	50
Figura 10 Casos de uso módulo de ventas	51
Figura 11 Diagrama de actividades gestión de vivero	53
Figura 12 Diagrama de secuencia modulo hotel plantas	54
Figura 13 Diagrama de secuencia domicilios	54
Figura 14 Diagrama de secuencia modulo inventario	55
Figura 15 Diagrama de secuencia módulo de ventas	55
Figura 16 Diagrama de clases sistema de gestión del vivero	56
Figura 17 Modelo entidad-relación (notación moderna)	57
Figura 18 Diagrama de componentes	58
Figura 19 Diagrama de despliegue	59
Figura 20 Front del dashboard administrativo	60
Figura 21 Front de orders (pedidos)	61
Figura 22 Front de sales (ventas)	61
Figura 23 Front gestión de inventario	62
Figura 24 Front registro de producto	62
Figura 25 Front editar producto	63
Figura 26 Front registrar reserva al hotel de plantas	63
Figura 27 Front detalles de la consulta de la reserva	64
Figura 28 Front de la gestión de usuarios	64
Figura 29 Front de los resúmenes de reportes	65
Figura 30 Front del dashboard desde el punto de vista móvil	66
Figura 31 Front de ventas desde el punto de vista móvil reponsive	67
Figura 32 Front desde el punto de vista móvil de los reportes	67
Figura 33 Front dashboard landscape	68
Figura 34 Modelo relacional de Viverapp	71
Figura 35 Ramas de GitHub	112
Figura 36 Captura de pantalla repositorio	113
Figura 37 Prueba registro de planta	125
Figura 38 Prueba reserva hotel de plantas	126
Figura 39 Prueba de stock sin productos	126
Figura 40 Corrección prueba alerta stock	127
Figura 41 Guía para iniciar sesión	131

Lista de Tablas

Tabla 1 Matriz de riesgos del proyecto	17
Tabla 2 Organización de la información recolectada	32
Tabla 3 Resultados cuantitativos	33
Tabla 4 Resultados observación directa	34
Tabla 5 Síntesis comparativa	34
Tabla 6 Cuadro de requerimientos funcionales	35
Tabla 7 Cuadro de requerimientos no funcionales	37
Tabla 8 Requerimientos normativos	39
Tabla 9 Caso de uso extendido registrar estancia	48
Tabla 10 Caso de uso extendido validación estancia	49
Tabla 11 Caso de uso extendido de domicilios	50
Tabla 12 Diccionario de datos	72
Tabla 13 Casos de prueba	115
Tabla 14 Resultados pruebas funcionales	118
Tabla 15 Resultados pruebas no funcionales	118
Tabla 16 Tipos de alertas / excepciones	121
Tabla 17 Casos de prueba a nivel de sistema	123
Tabla 18 Costos estimados	129
Tabla 19 Definición de términos para Viverapp	140

Lista de Apéndices

Apéndice A Script del Front

143

Introducción

La industria de la belleza ha experimentado una transformación significativa los últimos años, impulsada por la creciente demanda de experiencias personalizadas, servicios ágiles y una gestión empresarial mas eficiente. En este contexto, las barberías y peluquerías han dejado de ser simples espacios de corte y arreglo personal para convertirse en centros especializados que ofrecen una experiencia integral de bienestar, estilo y atención al detalle. Esta evolución ha traído consigo nuevos retos: la necesidad de adaptarse a un cliente mas informado y exigente, optimizar los recursos internos, y mantenerse competitivo en un mercado cada vez mas dinamico.

A pesar de estos avances, una gran parte de los establecimientos aun opera bajo esquemas tradicionales que limitan su capacidad de crecimiento. La gestión manual de citas, el registro físico de clientes y la ausencia de indicadores claros sobre el rendimiento del negocio son solo algunas de las barreras que impiden una operación eficiente. Esta desconexion entre la modernización del servicio y la administración interna representa una oportunidad clave para la implementación de soluciones tecnológicas accesibles, escalables, y adaptadas a las necesidades reales del sector.

Este documento propone el desarrollo de un sistema de información integral para barberías, diseñado para automatizar y centralizar los procesos operativos críticos: programación de citas, gestión de clientes y registro e historial de citas. La digitalización de estos procesos no solo permite reducir errores y tiempos improductivos, sino que también fortalece la relación con los clientes mediante funcionalidades como recordatorios automáticos, seguimientos de preferencias y promociones personalizadas. El ultima instancia, se busca transformar la manera en que estos negocios operan, dotándolos de herramientas que les permitan crecer de forma ordenada, profesional y sostenible.

Planteamiento del Problema

Descripción del Problema

Las barberías, especialmente aquellas que operan como negocios independientes o familiares, suelen enfrentar desafíos relacionados con la administración de sus procesos. Muchos de estos establecimientos gestionan sus operaciones utilizando agendas físicas, hojas de cálculo o incluso confiando en la memoria, lo cual conlleva a una serie de dificultades que afectan directamente el desempeño del negocio y la experiencia del cliente.

Uno de los principales problemas es la desorganización en la programación de citas, lo cual genera retrasos, confusión en los horarios y pérdidas de clientes por tiempos de espera excesivos. Además, la falta de un sistema centralizado impide registrar adecuadamente la información de clientes, dificultando el seguimiento de sus preferencias o servicios previos, lo que limita la personalización del servicio.

Asimismo, las barberías que no utilizan herramientas digitales tienen menos capacidad para competir con negocios más modernos que ofrecen reservas en línea, promociones automáticas o pagos digitales. Esta desventaja tecnológica limita el crecimiento del negocio, la fidelización de clientes y su posicionamiento en el mercado.

Ante este panorama, surge la necesidad de implementar un sistema de información que permita automatizar y centralizar todos estos procesos, facilitando la gestión diaria, mejorando la experiencia del cliente y contribuyendo al crecimiento del negocio. Un software adaptado a las necesidades específicas de una barbería puede marcar la diferencia entre un negocio estancado y uno competitivo y eficiente.

Objetivo General

Diseñar e implementar un sistema de información integral para barberías que permita automatizar y optimizar los procesos operativos, administrativos, y comerciales del negocio. Esta solución tecnológica buscara mejorar la eficiencia en la gestión interna, fortalecer la atención al cliente mediante herramientas digitales personalizadas, y contribuir al crecimiento sostenible y competitivo

del establecimiento en el mercado actual.

Objetivos Específicos

Diseñar un modulo de gestión de clientes que permita almacenar y consultar datos personales, historial de servicios, preferencias y frecuencia de visitas, facilitando una atención personalizada.

Implementar un sistema digital de reservas y gestión de citas, con control de disponibilidad, asignación de horarios, confirmaciones automáticas y recordatorios via correo electrónico o mensajes.

Desarrollar una funcionalidad para el seguimiento del desempeño de los barberos, registrando los servicios realizados, valoraciones de los clientes y estadísticas de productividad.

Crear una interfaz intuitiva, responsiva, amigable y accesible desde dispositivos móviles y de escritorio, que facilite la navegación tanto para el personal como para los clientes.

Incorporar herramientas de análisis financiero, como reportes de ingresos, egresos, flujo de caja y rentabilidad, para apoyar la toma de decisiones estratégicas.

Establecer un sistema de fidelización de clientes, mediante promociones personalizadas, descuentos por frecuencia de visitas y seguimiento de fechas especiales como cumpleaños.

Garantizar la seguridad y privacidad de los datos, mediante protocolos de autenticación, respaldo automatico y cumplimiento de normativas vigentes sobre protección de información.

Justificación del Proyecto

La implementación de un software de gestión en una barbería tiene un impacto directo en la eficiencia operativa, la satisfacción del cliente y la rentabilidad del negocio. Automatizar procesos clave como la programación de citas, la administración de clientes, permite al equipo enfocarse en brindar un mejor servicio en lugar de ocuparse de tareas repetitivas y propensa a errores.

El sistema propuesto facilitará la organización interna, permitiendo que cada barbero consulte rápidamente las citas del día, los servicios realizados a cada cliente y las observaciones anteriores, lo que se traduce en un trato más personalizado. También permite reducir las ausencias mediante recordatorios automáticos por SMS o correo electrónico, mejorando el flujo de atención diaria.

Además, permite tener reportes claros de ingresos, servicios más demandados, desempeño del personal, lo cual es esencial para la toma de decisiones estratégicas.

La digitalización también mejora la imagen del negocio, proyectando una barbería moderna, profesional y alineada con las expectativas de los clientes actuales, quienes valoran la comodidad de agendar citas en línea, recibir notificaciones o tener una atención personalizada. Este proyecto, por tanto, tiene justificación tanto técnica como económica y comercial, siendo una inversión de crecimiento, competitividad y sostenibilidad del negocio.

Alcance del Proyecto

El presente proyecto contempla el desarrollo de un sistema de información web orientado a la gestión operativa y administrativa de barberías de pequeña y mediana escala. La solución permitirá optimizar la administración interna mediante los siguientes módulos funcionales:

- Gestión de usuarios y roles
- Registro y gestión de clientes
- Programación y control de citas
- Historial de servicios

- Gestion de pagos y ventas
- Modulo de reportes

La plataforma será desarrollada bajo arquitectura web responsive, permitiendo su acceso desde dispositivos móviles y de escritorio mediante un navegador estándar, sin requerir la instalación de software local en los equipos del cliente.

Matriz de Riesgo

Esta matriz identifica riesgos relevantes, su probabilidad de ocurrencia, el impacto potencial en el proyecto, y propone acciones de mitigación o respuesta, como se muestra en la Tabla 1.

Tabla 1*Matriz de riesgos del proyecto*

ID	Riesgo	Probabilidad	Impacto	Nivel de Riesgo	Acción de Mitigación / Contingencia
R1	Requerimientos mal definidos por parte del cliente	Alta	Alta	Critica	Realizar entrevistas detalladas, validación de requerimientos con el cliente y actas de aprobacion
R2	Cambios frecuentes en los requerimientos	Medio	Alta	Alta	Gestionar cambios con un comité y documentar cada solicitud
R3	Mala estimación de precios y costos	Alta	Alta	Critica	Cronograma detallado con márgenes de contingencia y control de avances.
R4	Dificultad para levantar información completa del negocio	Media	Media	Media	Aplicar varias técnicas de elicitation (entrevistas, observación, encuestas)
R5	Falta de participación del cliente en la etapa de analisis	Alta	Alta	Critica	Definir reuniones periodicas obligatorias y actas de compromiso
R6	Falta de claridad en la arquitectura del sistema	Baja	Alta	Media	Revisiones técnicas con el equipo y validación con un arquitecto de software

ID	Riesgo	Probabilidad	Impacto	Nivel de Riesgo	Acción de Mitigación / Contingencia
R7	Errores en la definición de interfaces de usuarios	Media	Media	Media	Uso de prototipos y pruebas de usabilidad con usuarios reales
R8	Mala asignación de tareas entre desarrolladores	Media	Alta	Alta	Uso de metodologías ágiles (Scrum), reuniones diarias de control
R9	Retrasos por desconocimiento de tecnologías	Alta	Alta	Critica	Capacitaciones previas, apoyo entre pares, documentación de buenas practicas
R10	Fallos de integración entre modulos	Media	Alta	Alta	Integracion continua y pruebas automatizadas
R11	Cobertura insuficiente de pruebas	Media	Alta	Alta	Plan de pruebas detallado con unitarias, integración y aceptacion
R12	Errores críticos no detectados antes de entrega	Baja	Critica	Critica	QA estricto, pruebas automatizadas y pruebas con usuarios finales
R13	Resistencia al cambio por parte de barberos y personal	Alta	Alta	Critica	Capacitaciones, manual de usuario y soporte inicial
R14	Fallos en la migración de datos de clientes antiguos	Media	Alta	Alta	Plan de migración con respaldo pruebas piloto antes de despliegue
R15	Rotacion del personal del equipo de desarrollo	Media	Media	Media	Documentacion exhaustiva y trabajo colaborativo para evitar dependencia

ID	Riesgo	Probabilidad	Impacto	Nivel de Riesgo	Acción de Mitigación / Contingencia
R16	Poca comunicación entre cliente y equipo	Alta	Alta	Critica	Reuniones periodicas, actas de compromiso y herramientas colaborativas
R17	Sobrecostos por falta de control financiero	Media	Alta	Alta	Monitoreo de gastos, control de presupuesto semanal y plan de contingencia
R18	Perdida de documentación del proyecto	Baja	Media	Media	Almacenamiento en la nube con respaldos y control de versiones
R19	Desercion de un integrante del grupo	Alta	Alta	Critica	Prevenir con roles claros, detectar con reuniones de seguimiento y mitigar redistribuyendo tareas con un plan de respaldo

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025). Se considera para la probabilidad Baja / Media / Alta. En el impacto qué tan grave sería el efecto si ocurre y en el nivel de riesgo la evaluación combinada de probabilidad e impacto.

Elicitación de Requisitos

Identificación de Procesos

El mapa de procesos del sistema de gestión integral para la barbería permite identificar, organizar y visualizar de manera clara las actividades principales que garantizan el funcionamiento del negocio. A través de este esquema se representan los procesos estratégicos, que orientan la toma de decisiones, la fidelización de clientes y la mejora continua; los procesos misionales, que corresponden a la programación de citas, atención del servicio, registro de ventas e historial del cliente; y los procesos de apoyo, que aseguran el soporte operativo mediante la gestión de usuarios, seguridad de la información y mantenimiento tecnológico.

Este mapa ofrece una visión global que facilita la optimización de los tiempos de atención, el control de la agenda del personal y la eficiencia en las operaciones diarias.

Los procesos misionales o procesos de negocio (Core Business) son los que directamente generan valor al cliente:

- Gestión de Citas y Agendamiento
 - Reserva y programación de citas en línea y presenciales.
 - Asignación de turnos según disponibilidad del barbero.
 - Modificación, reprogramación y cancelación de citas.
 - Control del estado de la cita (pendiente, finalizada).
- Gestión de Servicios y Atenciones
 - Registro de los servicios prestados (corte, barba, perfilado...).
 - Consulta e historial de preferencia de servicios por cliente.
 - Asignación del barbero responsable de la atención.
- Gestión de Ventas y Cobros
 - Registro de pagos de servicios.

- Control de métodos de pago.
- Registro de ingresos por barbero y turno.
- Gestion de Clientes
 - Registro e identificación de clientes (datos de contacto).
 - Historial de visitas y servicios contratados.
 - Registro de notas o preferencias especiales del cliente.

Los procesos de apoyo (Soporte interno) son:

- Soporte Tecnológico e Infraestructura Web
 - Alojamiento y disponibilidad de la aplicación web (Hosting y servidor).
 - Mantenimiento de la base de datos relacional y gestión de respaldos (backups).
 - Optimización de la interfaz adaptativa (Responsive Design) para el correcto despliegue en dispositivos móviles y de escritorio.
- Seguridad y Privacidad
 - Protección de datos personales de clientes y personal operativo.
 - Control de integridad de datos y restricción de accesos no autorizados a las funciones administrativas.
- Gestión Administrativa y Reportes de Control
 - Consolidación de información operativa para el monitoreo del negocio
 - Generación de reportes de desempeño e ingresos por periodo o por barbero

En los procesos administrativos (Gestión organizacional) se tienen:

- Planificación y Gestión Operativa
 - Definición de horarios de atención, turnos de trabajo y disponibilidad del personal.
 - Establecimiento de tarifas para los servicios ofrecidos.

- Definición de políticas de cancelación, reservas, y tiempo de tolerancia para las citas.
- Gestión de Desempeño y Talento Humano
 - Asignación y control de responsabilidades según el rol.
 - Monitoreo de productividad, número de servicios prestados e ingresos generados por cada barbero.
- Análisis de Negocio y Toma de Decisiones

Revisión de métricas clave a través del módulo de reportes.

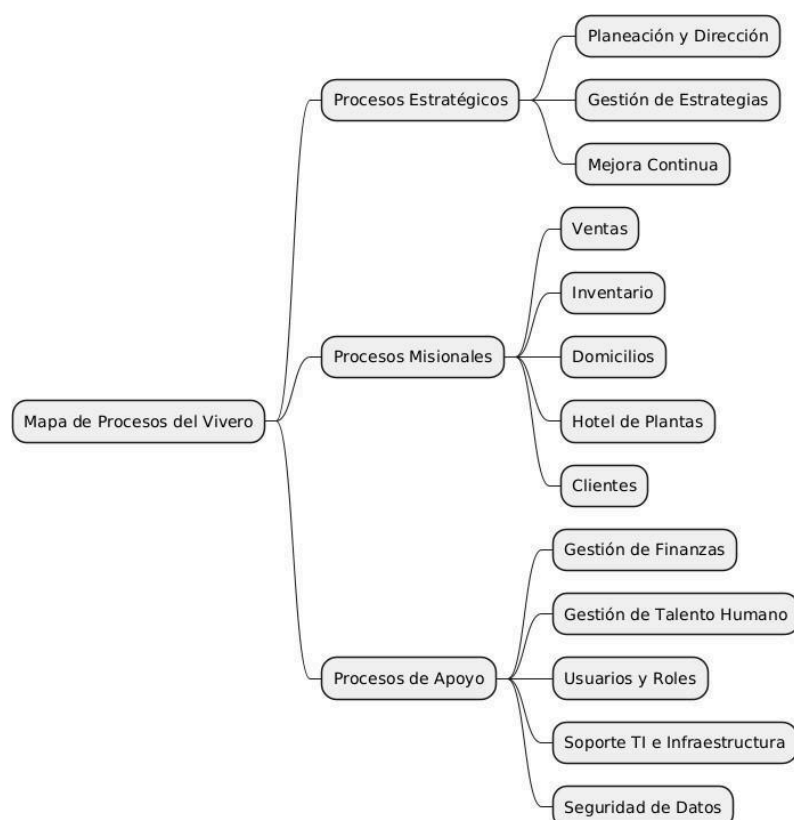
Evaluación financiera para la optimización de costos y ajuste de precios.

Planificación de estrategias de fidelización de clientes y promociones comerciales.

El siguiente mapa de procesos que muestra Figura 1, es la estructura general de la gestión, facilitando la comprensión de los flujos y su contribución a los objetivos institucionales.

Figura 1

Mapa de procesos



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Recolección de la Información del Software a Construir de acuerdo con las Necesidades del Cliente

Este proceso permite obtener los requerimientos clave del software, garantizando que la solución responda a las necesidades del usuario.

Elección de la Técnica de Recolección de la Información

La entrevista semiestructurada es una técnica cualitativa que permite obtener información detallada y profunda a través de una conversación guiada por preguntas previamente definidas, pero con flexibilidad para adaptarse a las respuestas del entrevistado. Se utilizó para conocer la experiencia de los administradores y empleados del vivero respecto a los procesos de ventas, domicilios, inventario y el hotel de plantas.

La encuesta es un instrumento cuantitativo que recopila datos estandarizados de un grupo de personas mediante un cuestionario estructurado. Se aplicó a los clientes del vivero con el fin de medir su nivel de satisfacción, identificar necesidades y conocer las expectativas sobre la implementación de un sistema web que optimice los servicios ofrecidos.

Finalmente, la observación directa consiste en el registro sistemático y objetivo de las actividades y comportamientos en el lugar donde ocurren los procesos. En este caso, se utilizó para analizar de primera mano cómo se realizan actualmente las ventas, la preparación de domicilios y la admisión de plantas en el hotel, identificando tiempos, dificultades y pasos que se realizan de forma manual.

Diseño de los Formatos, según la Técnica o Técnicas de Recolección de la Información Seleccionada(s)

En la Entrevista semiestructurada el objetivo es obtener información cualitativa profunda sobre procesos actuales, necesidades, problemas y expectativas (propietarios, gerentes, encargados de tienda/hotel/domicilios).

Duración estimada: 30–50 minutos.

Consentimiento (inicio): Buen día/tarde. Soy [nombre]. Estoy realizando una investigación/proyecto para diseñar un sistema de gestión para el vivero. La entrevista durará aprox. 30–50 minutos. Toda la información será tratada confidencialmente y solo se usará con fines del proyecto. ¿Está de acuerdo en participar y que la entrevista sea grabada (audio)? (Sí / No)

Datos del entrevistado (registro):

- Nombre
- Cargo
- Antigüedad en el vivero
- Fecha
- Lugar

Respecto a la guía de preguntas (orden flexible):

- Contexto y roles
 - ¿Podría describir brevemente sus tareas diarias en el vivero?
 - ¿Qué procesos supervisa o ejecuta directamente?
- Ventas y punto de venta
 - ¿Cómo se registran hoy las ventas (manual, en Excel, en sistema)?
 - ¿Qué problemas encuentra al registrar ventas o facturar?
 - ¿Qué información considera imprescindible en la ficha del producto al vender?
- Domicilios / entregas
 - ¿Cómo se recibe y procesa un pedido para domicilio?
 - ¿Cómo se calculan costos/tiempos de envío y cómo se asignan los repartidores?
 - ¿Qué dificultades hay con las entregas (retrasos, rutas, comunicación)?
- Hotel de plantas (servicio de hospedaje)
 - ¿Cómo ingresan las plantas al servicio (registro, ficha, fotos)?

- ¿Cómo se hacen los controles de riego, cambio de sustrato y otros cuidados?
- ¿Cómo se calcula la tarifa y se comunica al cliente el estado de la planta?
- Inventario y control
 - ¿Cómo llevan el inventario actualmente? (ubicaciones físicas, frecuencia conteo)
 - ¿Se presentan desajustes? ¿Cada cuánto hacen inventario físico?
 - ¿Cómo se manejan las existencias por vivero/almacén?
- Atención al cliente
 - ¿Cómo se registra y da seguimiento a las solicitudes o reclamaciones?
 - ¿Qué canales usan los clientes (teléfono, WhatsApp, redes, presencial)?
- Reportes y necesidades
 - ¿Qué reportes usa hoy y cuáles desearía tener automáticamente?
 - ¿Qué decisiones le gustaría tomar con información en tiempo real?
- Tecnología y adopción
 - ¿Qué herramientas tecnológicas ya usan? ¿Con qué nivel de satisfacción?
 - ¿Cuál sería la mayor barrera para implementar un nuevo sistema?
- Cierre
 - ¿Hay alguna función imprescindible que no hayamos mencionado?
 - ¿Desea agregar algo más?

Las instrucciones para el entrevistador son:

- Usar lenguaje claro; si la persona no entiende un término técnico, explicar
- Registrar citas textuales relevantes.
- Señalar observaciones no verbales relevantes (p. ej. muestra frustración, gestos).

Luego sigue la Encuesta (cuestionario) — dirigida a clientes y/o visitantes, su objetivo es medir uso, satisfacción y necesidades desde la perspectiva del cliente (para priorizar funcionalidades UX y servicios: pedidos online, seguimiento, hotel de plantas).

Formato: cuestionario corto, combinación de opciones múltiples y escala Likert (1–5). Duración ≈ 5–8 minutos. Se puede aplicar presencial o en línea.

Instrucciones iniciales (para encuestado): Esta encuesta busca conocer su experiencia con [Nombre del Vivero]. Sus respuestas son confidenciales y se usarán para mejorar nuestros servicios.

Tiempo estimado: 5 minutos.

Sección A — Datos demográficos (opcionales):

Edad: _____

Género: ☐ F ☐ M ☐ Otro ☐ Prefiero no decir

¿Vive en la misma ciudad del vivero? ☐ Sí ☐ No

Sección B — Uso y comportamientos

¿Con qué frecuencia compra plantas en este vivero?

☐ Varias veces al mes

☐ 1 vez al mes

☐ Cada pocos meses

☐ Menos de 1 vez al año

¿Ha usado el servicio de domicilio del vivero?

☐ Sí ☐ No

¿Conoce o ha utilizado el servicio de hospedaje/hotel de plantas?

☐ Sí, lo he usado

☐ Sí, lo conozco pero no lo he usado

☐ No lo conocía

Sección C — Satisfacción (escala 1–5: 1 = muy insatisfecho / 5 = muy satisfecho)

- Satisfacción general con el vivero
- Calidad de las plantas
- Tiempo de entrega a domicilio (si aplica)
- Claridad de la información sobre cuidados de cada planta
- Atención al cliente / asesoría
- Precio / relación calidad-precio
- Presentación / estado al recibir plantas a domicilio

Sección D — Preferencias funcionales

¿Qué funciones le gustaría encontrar en un sitio web/app del vivero? (marque todas)

- Comprar online
- Reserva del hotel de plantas
- Seguimiento en tiempo real de domicilio
- Fichas de cuidado y recordatorios
- Chat/asesoría en vivo
- Programar servicios de cuidado (riego, trasplante)
- Otros: _____

¿Con qué probabilidad usaría una app/web para reservar hospedaje de su planta? (0–10)

0 1 2 3 4 5 6 7 8 9 10

Sección E — Comentarios abiertos

¿Qué mejorarías del servicio actual del vivero? (respuesta libre)

Instrucciones de muestreo y tamaño

Si es cliente presencial: aplicar a una muestra de conveniencia en puntos de venta hasta

150–200 respuestas para obtener representatividad local.

Si es online: compartir en redes y WhatsApp; objetivo mínimo 100 respuestas.

Para análisis cuantitativo: usar frecuencias, medias, desviación estándar; para ítems Likert agrupar en satisfecho/neutral/insatisfecho.

Lista de observación (checklist) — para procesos operativos

El objetivo es registrar en terreno cómo se realizan procesos clave (venta presencial, recepción de pedido para domicilio, admisión al hotel de plantas, control de inventario).

Formato: checklist con columna de observaciones. Tiempo por observación: 30–60 minutos por proceso.

Observación 1 — Proceso de venta presencial / POS

- Fecha / Hora / Observador:
- Lugar / Caja / Empleado:

Pasos a observar (marcar Sí/No y observaciones):

- Cliente seleccionado producto y solicita información. (S/N) Obs:
- El vendedor usa ficha técnica del producto para asesorar. (S/N) Obs:
- Se verifica stock antes de cobrar. (S/N) Obs:
- Se imprime factura o ticket. (S/N) Obs:
- Se actualiza inventario inmediatamente. (S/N) Obs:
- Se empaqueta la planta correctamente (sustrato, protección). (S/N) Obs:
- Tiempo desde decisión de compra hasta entrega al cliente: ____ minutos.

Observación 2 — Recepción y preparación de pedido para domicilio

- Fecha / Hora / Observador:

Pasos:

- Pedido recibido por teléfono/WhatsApp/online: (S/N) Obs:
- Datos del cliente verificados (dirección / teléfono). (S/N) Obs:

- Se verifica disponibilidad de producto. (S/N) Obs:
- Se calcula y comunica tarifa y tiempo estimado. (S/N) Obs:
- Se asigna repartidor y se imprime/manda guía de envío. (S/N) Obs:
- Producto preparado y embalado para transporte. (S/N) Obs:
- Pedido sale de tienda en: hh:mm. Repartidor registra inicio y fin. (S/N) Obs:

Observación 3 — Admisión al hotel de plantas

- Fecha / Hora / Observador:

Pasos:

- Se realiza ficha de ingreso con: fotos, fecha entrada, estado general. (S/N) Obs:
- Se registran cuidados recomendados del cliente. (S/N) Obs:
- Se asigna ubicación física (estantería / estante). (S/N) Obs:
- Se calcula costo y se cobra o deja constancia de pago. (S/N) Obs:
- Existe calendario/registro de riego y actividades. (S/N) Obs:
- Se realizan controles periódicos según calendario. (S/N) Obs:
- Se comunica estado al cliente (frecuencia, medio). (S/N) Obs:

Observación 4 — Conteo/inventario

- Fecha / Hora / Observador:

Pasos:

- Se realiza conteo por especie y ubicación. (S/N) Obs:
- Las cantidades en sistema coinciden con las físicas (Sí / No). Si no, detalle:
- Se registran entradas recientes y salidas del día. (S/N) Obs:
- Tiempo invertido en conteo (minutos):
- Herramientas usadas (Excel, papel, sistema): _____

Instrucciones para el observador

- No intervenir; anotar comportamientos y tiempos exactos.
- Tomar fotos solo si el encuestado/encargado lo autoriza.
- Si hay variaciones, anotar causa aparente (ej. falta de personal, sistema caído).

Aplicación de la Técnica de Recolección de la Información

La aplicación de la técnica de recolección de la información consiste en poner en práctica métodos específicos para obtener datos relevantes en una investigación. Esta etapa permite recopilar evidencia útil y confiable, ajustada a los objetivos del estudio.

Resultados de Entrevista Semiestructurada (síntesis)

Perfil entrevistado: Administrador del vivero, 8 años de experiencia.

Respuestas clave:

Ventas: actualmente registran en cuaderno y luego en Excel.

Problema: se pierden datos de clientes frecuentes, y a veces se olvidan de descontar inventario.

Domicilios: se reciben pedidos por WhatsApp y llamadas. No tienen sistema de seguimiento en tiempo real, lo que genera reclamos de clientes (“no sé dónde viene mi pedido”).

Hotel de plantas: llevan un cuaderno con fecha de ingreso, nombre del cliente y cuidados especiales. No hay control automatizado de riego ni alertas. Se han presentado olvidos de riego.

Inventario: se hace conteo manual cada 15 días, pero siempre hay diferencias. “El sistema debería alertar cuando queda poco stock de ciertas especies”.

Reportes: al gerente le gustaría tener reportes de ventas por especie y ocupación del hotel de plantas.

Atención al cliente: la comunicación es principalmente por WhatsApp. “Los clientes piden fotos del estado de su planta en el hotel, pero se hace de manera muy informal”.

Conclusión entrevista: Existe una necesidad clara de sistematizar ventas, domicilios e inventario y de profesionalizar el hotel de plantas con alertas y fichas digitales.

Resultados de Encuesta a Clientes

Perfil de muestra:

65% mujeres, 30% hombres, 5% otros.

70% entre 25 y 45 años.

80% residentes en la misma ciudad.

Hallazgos:

Frecuencia de compra: 40% compra cada pocos meses, 25% cada mes, 20% varias veces al mes, 15% menos de una vez al año.

Uso de domicilio: 60% sí lo ha usado, 40% no.

Conocimiento del hotel de plantas: 15% lo ha usado, 50% lo conoce, pero no lo ha usado, 35% no lo conocía.

Nivel de satisfacción (escala 1–5):

Calidad de las plantas: 4.6

Atención al cliente: 4.3

Tiempo de entrega a domicilio: 3.2 (punto débil)

Claridad en cuidados de plantas: 3.8

Relación calidad-precio: 4.1

Funciones deseadas en un sistema web/app (múltiple elección):

Comprar online: 80%

Seguimiento en tiempo real del domicilio: 70%

Reserva en hotel de plantas: 65%

Fichas de cuidado y recordatorios: 55%

Chat de asesoría en vivo: 40%

Comentario abierto recurrente: “Sería muy útil poder reservar o pagar online sin tener que escribir por WhatsApp”.

Conclusión encuesta: Los clientes están satisfechos con la calidad, pero el gran problema es la logística del domicilio. También hay interés en digitalizar el hotel de plantas y las compras online.

Resultados de Observación Directa

Proceso observado: Preparación y entrega de un pedido a domicilio (martes 10:00 a.m.).

Hallazgos (checklist):

Pedido recibido por WhatsApp: Sí.

Datos del cliente confirmados: Sí, pero manualmente en libreta.

Verificación de stock: Tardaron 12 min porque buscaron físicamente la planta.

Comunicación de tarifa y tiempo: Sí, pero el cliente no recibió comprobante.

Asignación de repartidor: Sí, pero sin registro en sistema.

Preparación del producto: Correcta (empaquete protegido).

Tiempo total entre pedido y salida: 45 minutos.

Observación extra:

El cliente escribió por WhatsApp a los 30 minutos preguntando si ya había salido su pedido.

No existe registro digital de la salida ni seguimiento en ruta.

Conclusión observación: El proceso depende de muchos pasos manuales y la falta de un sistema digital genera retrasos y confusión. El tiempo promedio de 45 minutos es alto para pedidos pequeños.

Síntesis comparativa (entre técnicas):

Entrevista: detecta problemas internos por duplicidad de registros, errores de inventario, falta de alertas.

Encuesta: muestra necesidades externas (clientes) para mejorar domicilios, compras online, reservas hotel.

Observación: evidencia la brecha de procesos manuales, tiempos largos, falta de seguimiento digital.

Con estos tres instrumentos aplicados, se concluye que el software web debe priorizar:

- Módulo de ventas con inventario en tiempo real.
- Módulo de domicilios con seguimiento y registro digital.
- Módulo de hotel de plantas con fichas digitales y alertas de cuidado.

Organización de la Información Recolectada

Entrevista semiestructurada

Perfil del entrevistado: Administrador del vivero (8 años de experiencia).

En la Tabla 2, se observa los puntos principales detectados en la recolección.

Tabla 2

Organización de la información recolectada

Tema	Hallazgos principales
Ventas	Registros en cuaderno y Excel provocan la pérdida de datos y errores en inventario.
Domicilios	Pedidos por WhatsApp/llamadas; sin seguimiento digital genera reclamos por demoras.
Hotel de plantas	Registro manual en cuaderno; olvidos de riego; falta de alertas y control digital.
Inventario	Conteo manual cada 15 días; diferencias entre físico y Excel.
Reportes	Necesidad de reportes de ventas por especie y ocupación del hotel.
Atención al cliente	Comunicación por WhatsApp; clientes piden fotos informales del estado de sus plantas.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Se concluye la necesidad de un sistema integral que unifique ventas, domicilios, hotel de plantas e inventario con reportes automáticos.

Encuesta a clientes (120 encuestados)

Perfil de muestra:

- 65% mujeres, 30% hombres, 5% otros.
- 70% entre 25 y 45 años.
- 80% residentes en la ciudad del vivero.

La Tabla 3, expone los resultados cuantitativos de las encuestas.

Tabla 3

Resultados cuantitativos

Ítem	Resultado
Frecuencia de compra	40% cada pocos meses; 25% mensual; 20% varias veces/mes; 15% < 1 vez/año
Uso del domicilio	60% Sí / 40% No
Conocimiento hotel de plantas	15% lo ha usado; 50% lo conoce; 35% no lo conocía

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Satisfacción (escala 1–5):

- Calidad de plantas: 4.6
- Atención al cliente: 4.3
- Tiempo de entrega domicilio: 3.2 (débil)
- Claridad en cuidados: 3.8
- Relación calidad-precio: 4.1

Funciones deseadas en web/app:

- Comprar online: 80%
- Seguimiento en tiempo real del domicilio: 70%
- Reserva en hotel de plantas: 65%

- Fichas de cuidado y recordatorios: 55%
- Chat de asesoría en vivo: 40%

En conclusión, los clientes valoran calidad, pero piden mejoras en domicilio y demandan digitalización de servicios.

Observación directa

Proceso observado: Pedido a domicilio (martes 10:00 a.m.).

A continuación, en la Tabla 4, se muestra tabulación de los resultados obtenidos:

Tabla 4

Resultados observación directa

Paso	Resultado	Observaciones
Recepción del pedido	Sí	Vía WhatsApp
Confirmación de datos cliente	Sí	En libreta manual
Verificación de stock	Sí	Tardó 12 min buscando físicamente
Comunicación de tarifa y tiempo	Sí	Sin comprobante oficial
Asignación de repartidor	Sí	Sin registro digital
Preparación del producto	Correcta	Buen embalaje
Tiempo total hasta salida	45 min	Cliente preguntó por WhatsApp a los 30 min

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Por lo tanto, el proceso manual, sin registro digital y con tiempos largos genera incertidumbre al cliente. En la Tabla 5, se muestra la síntesis comparativa.

Tabla 5

Síntesis comparativa

Técnica	Principales hallazgos
Entrevista	Procesos internos ineficientes: inventario manual, sin alertas, registros dispersos.
Encuesta	Clientes satisfechos con la calidad, pero inconformes con el tiempo de domicilio; piden compras y reservas online.

Técnica	Principales hallazgos
Observación	Se comprobó que los procesos manuales alargan tiempos (45 min para un pedido pequeño).

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Especificación de Requerimientos, de acuerdo con la Información Recolectada, Aplicando Estándares de Requerimientos

La especificación de requisitos consiste en documentar de forma clara y detallada las funcionalidades y restricciones del sistema.

Requerimientos Funcionales

Los requerimientos funcionales describen las funciones específicas que debe cumplir un sistema para satisfacer las necesidades del usuario. Son esenciales para definir el comportamiento esperado del software. En la Tabla 6 se compila la especificación de requisitos funcionales.

Tabla 6

Cuadro de requerimientos funcionales

ID	Requerimiento	Descripción	Módulo Asociado	Prioridad
RF01	Registro de usuarios	Permitir el registro de clientes y empleados en el sistema.	Autenticación / Usuarios	Alta
RF02	Inicio de sesión	Autenticación segura para clientes y personal del vivero.	Autenticación / Seguridad	Alta
RF03	Gestión de catálogo de productos	Permitir la creación, edición, eliminación y visualización de plantas y productos.	Ventas / Inventario	Alta
RF04	Carrito de compras	Permitir a los clientes agregar productos al carrito antes de realizar la compra.	Ventas	Alta
RF05	Registro y seguimiento de pedidos	Permitir a los clientes generar pedidos y ver el estado del envío.	Ventas / Domicilios	Alta

ID	Requerimiento	Descripción	Módulo Asociado	Prioridad
RF06	Gestión de domicilios	Programar, asignar y hacer seguimiento a los pedidos a domicilio.	Domicilios	Alta
RF07	Módulo de reservas para hotel de plantas	Permitir a los clientes reservar espacio para dejar temporalmente sus plantas.	Hotel de Plantas	Alta
RF08	Registro de ingreso/salida de plantas	Registrar detalles de cada planta que entra o sale del hotel de plantas.	Hotel de Plantas	Alta
RF09	Seguimiento del estado de las plantas	Registrar y consultar observaciones y cuidados de las plantas hospedadas.	Hotel de Plantas	Media
RF10	Panel administrativo	Panel para que el personal administre productos, pedidos, usuarios y reservas.	Administración General	Alta
RF11	Notificaciones por correo o sistema	Notificar a los clientes sobre estados de pedidos, reservas y domicilios.	Comunicación / Alertas	Media
RF12	Búsqueda y filtrado de productos	Permitir búsqueda avanzada de productos por nombre, tipo, tamaño, etc.	Ventas	Media
RF13	Reportes de ventas y ocupación del hotel	Generar informes periódicos de ventas y uso del hotel de plantas.	Reportes	Media
RF14	Gestión de inventario en tiempo real	Actualizar automáticamente las existencias según ventas y entradas.	Inventario	Alta
RF15	Integración con pasarela de pagos	Permitir pagos en línea mediante plataformas locales (PSE, Nequi, etc.).	Ventas / Pagos	Alta

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Clasificación de Prioridades:

Alta: Requerimiento esencial para el funcionamiento del sistema.

Media: Requerimiento importante, pero no crítico para la primera versión.

Baja: Requerimiento opcional o que puede añadirse en versiones futuras.

Requerimientos No Funcionales

Los requisitos no funcionales definen las características de calidad que debe cumplir un sistema, como rendimiento, seguridad o usabilidad. A diferencia de los funcionales, no describen lo que el sistema hace, sino cómo lo hace. Son clave para garantizar una buena experiencia de usuario y el correcto funcionamiento del software. En la Tabla 7 se observa los propuestos para el proyecto.

Tabla 7

Cuadro de requerimientos no funcionales

ID	Nombre del Requerimiento	Descripción	Categoría	Prioridad
RNF01	Accesibilidad web	El sistema debe ser accesible desde cualquier navegador moderno con conexión a Internet.	Disponibilidad	Alta
RNF02	Responsive design	La interfaz debe adaptarse correctamente a dispositivos móviles, tabletas y escritorios.	Usabilidad	Alta
RNF03	Tiempos de carga	Las páginas deben cargar en un tiempo menor a 3 segundos bajo condiciones normales.	Rendimiento	Alta
RNF04	Seguridad de autenticación	La información de acceso debe estar protegida mediante cifrado y buenas prácticas de autenticación.	Seguridad	Alta
RNF05	Backup automático	El sistema debe realizar copias de seguridad automáticas diarias de la base de datos.	Fiabilidad	Alta
RNF06	Escalabilidad	El sistema debe permitir la incorporación de nuevos módulos	Mantenibilidad / Escalabilidad	Media

ID	Nombre del Requerimiento	Descripción	Categoría	Prioridad
		en el futuro sin reestructuración total.		
RNF07	Compatibilidad con navegadores	Compatible al menos con Chrome, Firefox, Safari y Edge.	Compatibilidad	Media
RNF08	Privacidad de los datos	El sistema debe cumplir con la Ley 1581 de 2012 (protección de datos en Colombia).	Legal / Seguridad	Alta
RNF09	Disponibilidad mínima del sistema	El sistema debe estar disponible al menos el 95% del tiempo durante el horario laboral.	Disponibilidad	Media
RNF10	Documentación del usuario	Se debe entregar un manual de usuario final y un manual técnico.	Documentación	Media
RNF11	Soporte multilenguaje	La plataforma debe estar inicialmente en español, pero preparada para soportar otros idiomas.	Internacionalización	Baja

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Clasificación por Categorías:

Disponibilidad: Relacionado con acceso y tiempos de operación del sistema.

Seguridad: Protección de datos e integridad del sistema.

Usabilidad: Facilidad de uso para el usuario final.

Rendimiento: Velocidad y eficiencia del sistema.

Compatibilidad: Interoperabilidad con dispositivos y software existentes.

Legal: Cumplimiento con normativas vigentes.

Escalabilidad / Mantenibilidad: Preparación para crecimiento y actualizaciones.

Documentación: Recursos escritos para soporte.

Requerimientos Normativos

Los requerimientos normativos son las obligaciones legales y estándares que debe cumplir un sistema. Aseguran que el software se ajuste a regulaciones y políticas establecidas. En la Tabla 8 se describen las aplicables al caso.

Tabla 8

Requerimientos normativos

ID	Requerimiento Normativo	Descripción	Categoría	Prioridad
RN01	Cumplimiento de la Ley 1581 de 2012 (Protección de Datos Personales en Colombia)	El sistema debe garantizar la privacidad y seguridad de los datos personales de los usuarios, permitiendo su tratamiento conforme a la ley, incluyendo consentimiento informado y derecho a la rectificación o eliminación.	Protección de datos	Alta
RN02	Cumplimiento de la Ley 1266 de 2008 (Habeas Data Financiero)	En caso de almacenar datos financieros o crediticios, debe respetar las disposiciones sobre manejo y confidencialidad de dicha información.	Protección de datos	Media
RN03	Normativas de comercio electrónico en Colombia	El sistema debe cumplir con las regulaciones que rigen la venta en línea, incluyendo facturación electrónica, términos y condiciones claras, y políticas de devolución.	Comercio electrónico	Alta
RN04	Regulación sanitaria para viveros y manejo de plantas	El software debe incluir registros que permitan dar cumplimiento a las normas sanitarias sobre cuidado y almacenamiento de plantas vivas.	Salud / Ambiental	Media
RN05	Normas de seguridad informática	Aplicar estándares para proteger la información frente a accesos no	Seguridad informática	Alta

ID	Requerimiento Normativo	Descripción	Categoría	Prioridad
		autorizados, ataques cibernéticos y garantizar la integridad del sistema.		
RN06	Ley 1712 de 2014 (Acceso a la Información Pública)	Si el vivero maneja información pública o debe reportar a entidades gubernamentales, debe garantizar la transparencia y acceso.	Transparencia	Baja
RN07	Legislación sobre protección al consumidor	Asegurar que los términos de servicio, garantía y atención al cliente.	Protección al consumidor	Alta

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Propuesta Técnica

Descripción General. El proyecto consiste en desarrollar una aplicación web para la gestión integral de un vivero en Cundinamarca, que incluya la administración de ventas, domicilios y un hotel de plantas. La solución debe ser accesible desde dispositivos móviles y computadores, con un diseño intuitivo y seguro.

Lenguaje de programación:

PHP (versión 7.x o superior) para el desarrollo del backend, procesamiento de lógica de negocio, conexión con base de datos y generación dinámica de contenido web.

Base de datos:

MySQL un sistema de gestión de bases de datos relacional para almacenar la información de usuarios, productos, pedidos, domicilios, reservas, etc.

Frontend:

Bootstrap (versión 4 o superior) es un framework CSS para el diseño responsivo y adaptativo, que facilita crear una interfaz atractiva y usable en dispositivos móviles y desktop.

Servidor web:

Apache o Nginx para alojar y servir la aplicación web.

Complementarios:

JavaScript (opcional) para mejorar la interacción en la interfaz, validaciones en el cliente y dinamismo.

Librerías/Ajax para mejorar experiencia de usuario en solicitudes asíncronas.

Arquitectura del Sistema:

Arquitectura MVC (Modelo-Vista-Controlador) para una mejor organización del código, mantenimiento y escalabilidad, los cuales son:

- Modelo es la gestión de datos y conexión con MySQL.
- Vista son las interfaces de usuario con Bootstrap.
- Controlador es la lógica de negocio y flujo de la aplicación.

API REST (opcional para futuro):

Para permitir integración con otras plataformas o aplicaciones móviles.

Características Técnicas:

- Seguridad
 - Autenticación segura con hashing de contraseñas.
 - Validación y sanitización de entradas para evitar inyección SQL y ataques XSS.
 - Uso de HTTPS para cifrado de la comunicación (configuración del servidor).
- Gestión de sesiones
 - Para controlar el acceso de usuarios y mantener estado.
- Backups automáticos
 - Programación de respaldos periódicos de la base de datos.
- Optimización y rendimiento

- Uso de consultas optimizadas y paginación para grandes volúmenes de datos.
- Minimización de recursos CSS y JS para acelerar carga.
- Adaptabilidad
 - Diseño responsive garantizado con Bootstrap para soportar múltiples dispositivos.

Despliegue:

El sistema será alojado inicialmente en un servidor web con soporte para PHP y MySQL, preferiblemente en un hosting con respaldo y soporte técnico confiable.

Configuración de dominios y certificados SSL para asegurar la navegación.

Son tecnologías open source costo eficientes y bien documentadas, mientras que Bootstrap ofrece flexibilidad para diseños rápidos y adaptables.

Reglas del Negocio

Las reglas del negocio son normas y políticas que guían el funcionamiento y la toma de decisiones dentro de una organización. Definen cómo se deben realizar los procesos para cumplir los objetivos empresariales.

Registro y autenticación:

- Solo usuarios registrados pueden realizar pedidos o reservar el hotel de plantas.
- Los empleados y administradores tienen roles y permisos diferenciados dentro del sistema.

Gestión de inventario:

- No se puede vender un producto si la cantidad solicitada supera el stock disponible.
- Al confirmar una venta, el sistema debe actualizar automáticamente el inventario.
- El inventario puede ser ajustado únicamente por personal autorizado.

Pedidos y ventas:

- Los pedidos deben contener al menos un producto para poder ser procesados.
- El sistema calcula automáticamente el total del pedido sumando precios y cantidades.
- Los pagos deben ser confirmados antes de proceder con el despacho o entrega.

Domicilios:

- Los domicilios solo pueden programarse dentro del territorio de Cundinamarca.
- La dirección de entrega debe ser validada y debe existir en la base de datos antes de aceptar el pedido.
- El tiempo estimado para la entrega dependerá de la ubicación y disponibilidad del personal.

Hotel de plantas:

- La reserva del hotel debe especificar fecha de ingreso y fecha estimada de salida.
- El cliente debe proporcionar instrucciones claras sobre cuidados especiales al momento del ingreso.

- El sistema registrará y actualizará el estado y observaciones de cada planta hospedada.

Cancelaciones y devoluciones

- Los pedidos pueden cancelarse antes de ser enviados; una vez despachados, no se aceptan cancelaciones.
- Las devoluciones solo serán aceptadas en casos de error en el producto o daño comprobado, conforme a políticas internas.

Notificaciones:

- El sistema debe notificar al cliente vía correo o mensaje interno cualquier cambio en el estado del pedido o reserva.
- Se enviarán recordatorios automáticos antes de la fecha de retiro de plantas del hotel.

Seguridad y privacidad:

- La información personal y de pago de los usuarios debe mantenerse confidencial y protegida conforme a la legislación vigente.
- Solo el personal autorizado puede acceder a información sensible o modificar datos críticos.

Análisis de la Especificación de Requisitos del Software

El análisis de la especificación de requisitos del software consiste en revisar y validar que los requerimientos sean claros, completos y consistentes. Este proceso es clave para asegurar el desarrollo correcto del sistema según las necesidades del usuario.

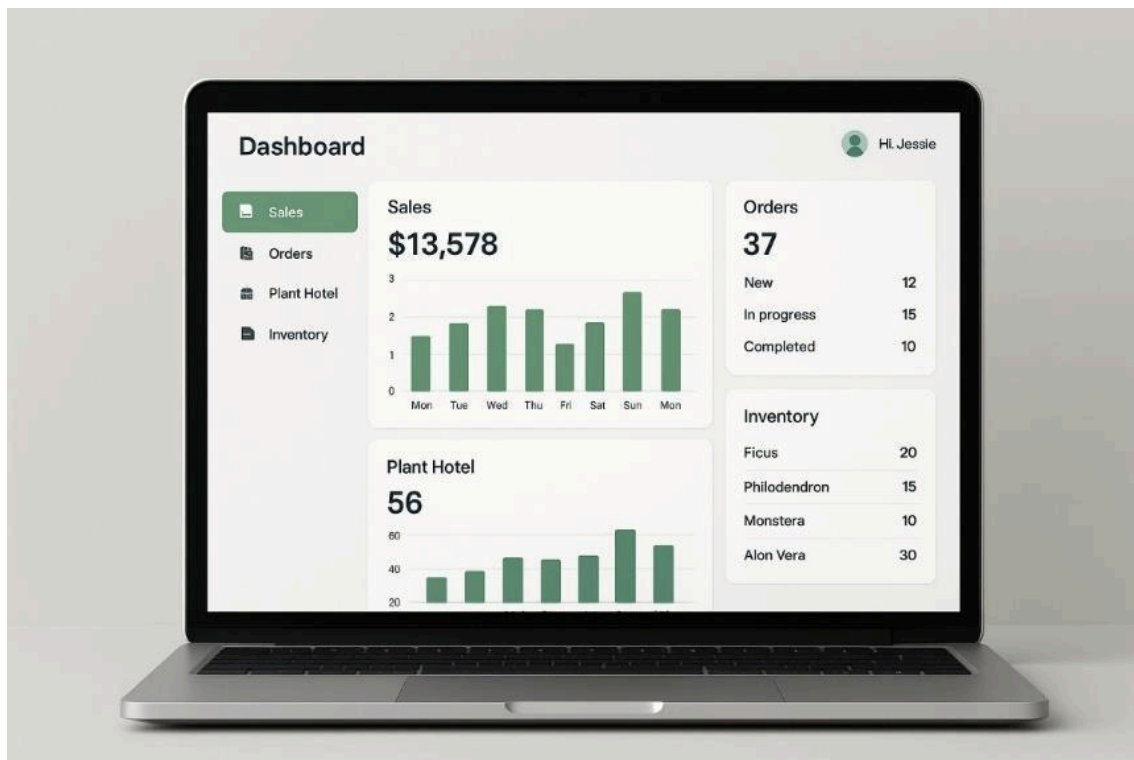
Alternativas de Solución (prototipo o prototipos del sistema, mockups)

Los mockups son representaciones visuales estáticas de la interfaz de usuario que muestran el diseño y distribución de elementos del sistema. Ayudan a visualizar la apariencia final antes del desarrollo.

En la Figura 2 se detallan los aspectos que verá el usuario administrador para el dashboard de ventas.

Figura 2

Mockup dashboard de ventas

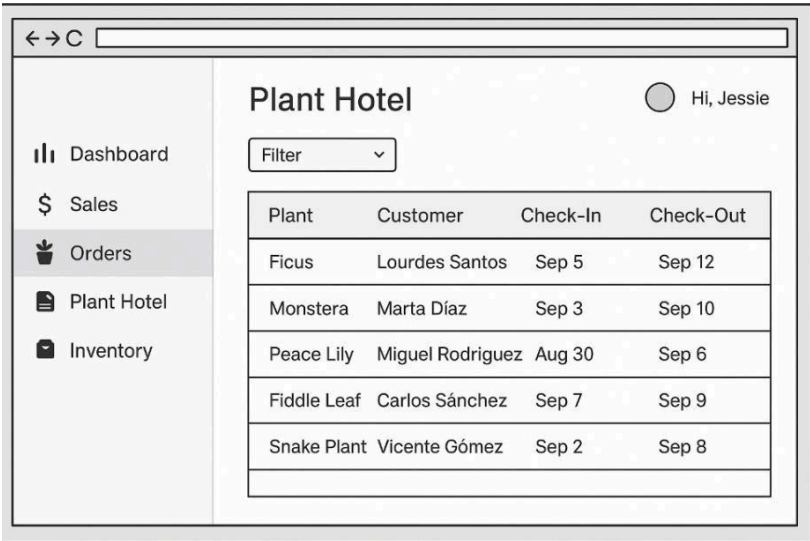


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la Figura 3 se exponen en la gráfica del mockup para el hospedaje de las plantas.

Figura 3

Mockup del hotel de plantas

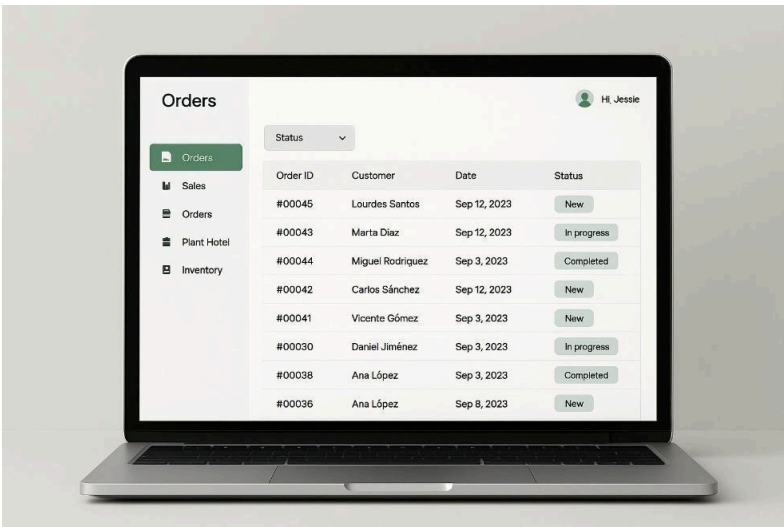


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

El mockup de la Figura 4 contiene la información de las ordenes o pedidos de domicilio.

Figura 4

Mockup de las ordenes



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Para la gestión de usuarios por parte del administrador se propone en la Figura 5 la vista de opciones que podría tener este rol para la gestión.

Figura 5

Mockup de gestión de usuarios del administrador



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Diagrama de Casos de Uso y Extensibilidad

El diagrama de casos de uso representa las interacciones entre los actores y el sistema. La extensibilidad permite añadir comportamientos opcionales o alternativos.

En la Figura 6 se muestra el diagrama de casos de uso que compila los principales casos a realizar por su rol correspondiente.

Figura 6

Casos de uso sistema general

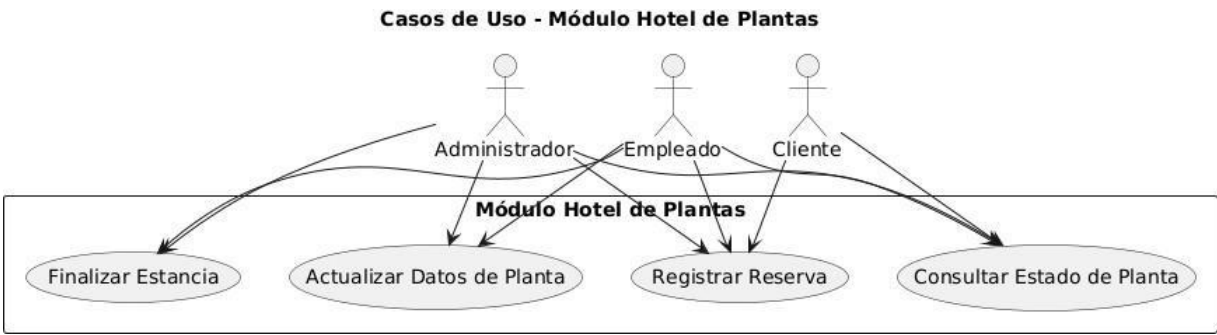


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

A continuación, se modulan los diagramas de casos de uso, empezando por el Modulo Hotel de Plantas de la Figura 7.

Figura 7

Casos de uso modulo hotel de plantas



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Un caso de uso extendido es una extensión de un caso de uso principal que describe escenarios alternativos o adicionales que pueden ocurrir durante la ejecución de un proceso. A continuación, en la Tabla 9 se detallan las situaciones del caso de uso.

Tabla 9

Caso de uso extendido registrar estancia

Elemento	Descripción
Nombre	Registrar Estancia de Planta
Actor Principal	Recepcionista / Usuario del sistema
Actores Secundarios	Cliente, Sistema de Inventario, Sistema de Facturación
Precondiciones	El cliente ha llevado una o más plantas al hotel. El usuario ha iniciado sesión en el sistema.
Postcondiciones	La planta queda registrada con su estancia activa y se genera un código o recibo de ingreso.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la Tabla 10 se muestra el caso de uso extendido para la validación de la disponibilidad en el hotel.

Tabla 10

Caso de uso extendido validación estancia

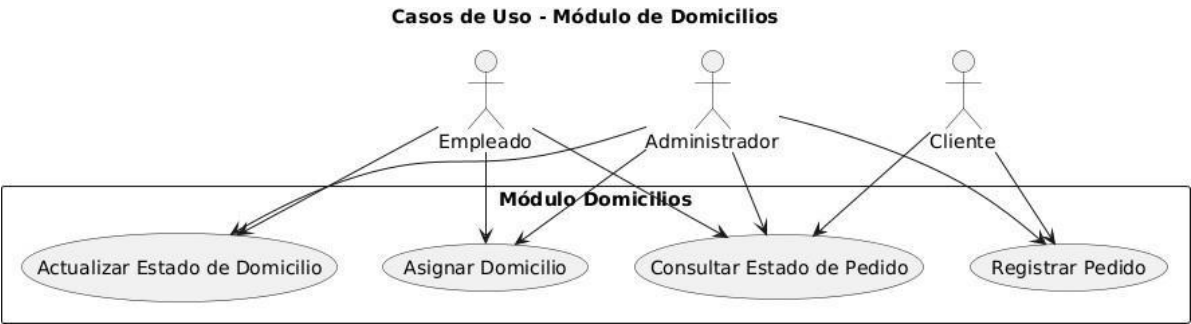
Elemento	Descripción
Nombre del Caso de Uso	Validar Disponibilidad de Espacio
Extendido	
Punto de Extensión	Durante el registro de la estancia, al seleccionar fecha y tipo de planta.
Condición de Activación	El sistema verifica si hay espacio disponible para el tipo de planta en las fechas solicitadas.
Acciones del Sistema	Verifica disponibilidad de racks, temperatura/humedad requerida, y compatibilidad de especies.
Resultado Alternativo	Si no hay espacio disponible, se sugiere una fecha alternativa o se deniega el ingreso.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Para el módulo de domicilios la Figura 8 muestra las principales acciones del usuario con el sistema.

Figura 8

Casos de uso de domicilios



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

El módulo de inventario permite gestionar y controlar la disponibilidad de productos dentro de una organización. Este caso de uso muestra en la Figura 9 los casos de uso correspondientes.

En la Tabla 11 se muestra el caso de uso extendido del módulo de domicilios.

Tabla 11

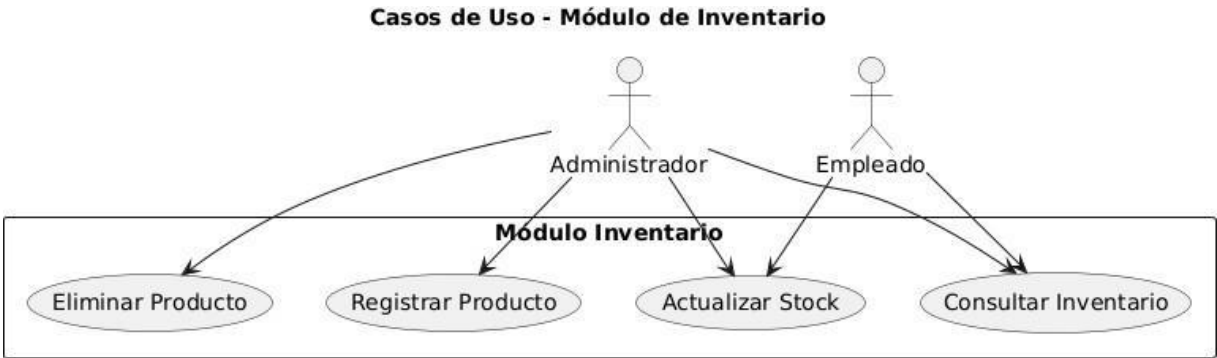
Caso de uso extendido de domicilios

Elemento	Descripción
Caso de Uso Principal	Realizar Pedido de Domicilio
Actor Principal	Cliente / Usuario
Actores Secundarios	Repartidor, Sistema de Inventario
Precondiciones	El cliente ha seleccionado las plantas para envío, y tiene una dirección válida registrada.
Postcondiciones	El pedido queda registrado, asignado a un repartidor y se notifica al cliente.
Descripción	Permite al cliente realizar un pedido de plantas para entrega a domicilio, seleccionando productos, fechas y dirección.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Figura 9

Casos de uso módulo de inventario



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la Tabla 12 se muestra el caso de uso extendido del módulo de inventario.

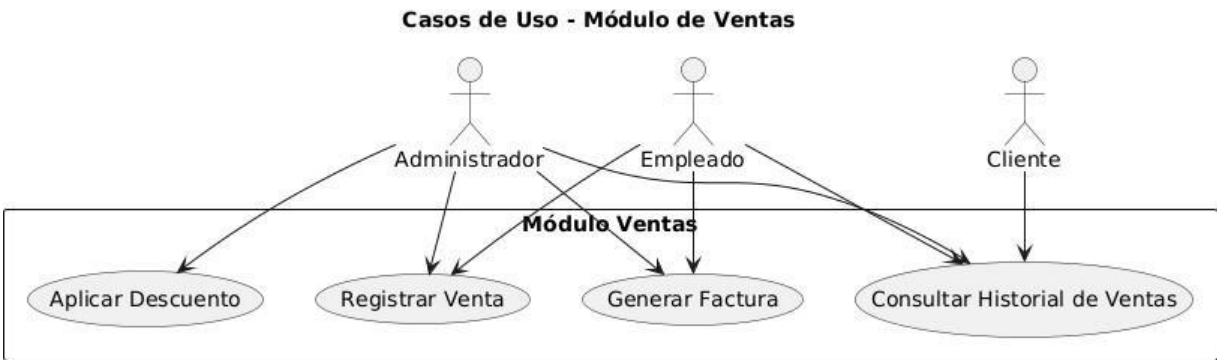
Elemento	Descripción
Caso de Uso Principal	Gestionar Inventario
Actor Principal	Encargado de Inventario / Administrador
Actores Secundarios	Sistema de Compras, Sistema de Ventas
Precondiciones	El usuario ha iniciado sesión con permisos para gestionar inventario.
Postcondiciones	El inventario queda actualizado con los movimientos realizados (altas, bajas, ajustes).
Descripción	Permite registrar, actualizar y controlar las cantidades y estados de los productos en stock.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

El módulo de ventas permite gestionar de forma eficiente el ciclo básico de ventas como se observa en la Figura 10.

Figura 10

Casos de uso módulo de ventas



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la **Tabla 13** se muestra el caso de uso extendido del módulo de ventas.

Elemento	Descripción
Caso de Uso Principal	Realizar Venta
Actor Principal	Vendedor / Usuario
Actores Secundarios	Cliente, Sistema de Inventario
Precondiciones	El vendedor ha iniciado sesión y el cliente ha seleccionado productos para comprar.
Postcondiciones	Se registra la venta, actualiza inventario y genera comprobante de pago o factura.
Descripción	Permite registrar la venta de productos, gestionar pagos y emitir comprobantes o facturas.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Diagramas de Actividades, y Secuencias

Los diagramas de actividades y los diagramas de secuencia son herramientas visuales utilizadas en el modelado de sistemas, especialmente dentro del estándar UML (Unified Modeling Language), para representar diferentes aspectos del comportamiento del sistema.

Facilitan la identificación de posibles cuellos de botella, repeticiones y oportunidades de optimización en los procesos.

Se representan mediante nodos, transiciones y bifurcaciones que ilustran gráficamente el comportamiento dinámico.

Diagrama de Actividades: Es útil para entender cómo se ejecutan los procesos paso a paso.

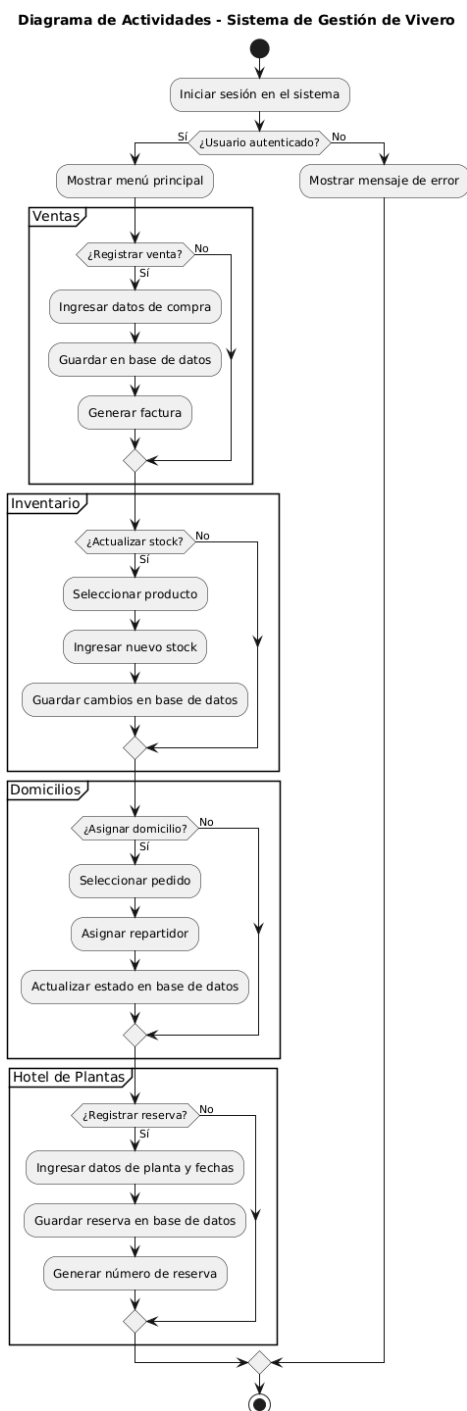
Diagrama de Secuencia: Detalla el orden de los mensajes intercambiados entre ellos para completar una funcionalidad específica.

Ambos diagramas son fundamentales para el análisis, diseño y documentación de sistemas, ya que permiten visualizar el comportamiento y la lógica detrás de cada caso de uso.

En la Figura se muestra el diagrama de actividades general del sistema propuesto.

Figura 11

Diagrama de actividades gestión de vivero

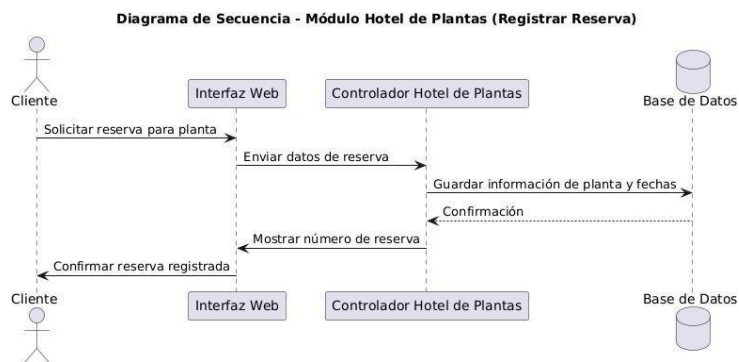


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

El diagrama de secuencia permite visualizar el intercambio de mensajes entre objetos a lo largo del tiempo para ejecutar una funcionalidad. Facilita el análisis del flujo lógico y la identificación de responsabilidades en un proceso. El diagrama del módulo de hotel de plantas de la Figura 12 que muestra el paso de mensajes, es decir, el orden de su ejecución:

Figura 12

Diagrama de secuencia modulo hotel plantas

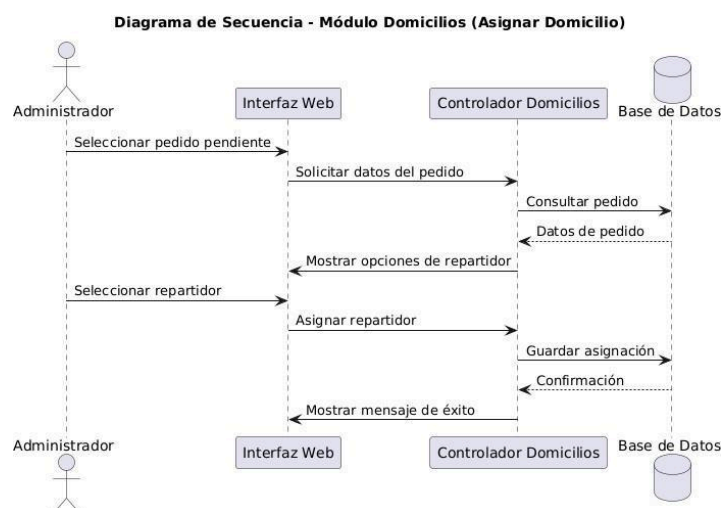


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la Figura 13, se muestra el diagrama de secuencia del módulo de domicilios.

Figura 13

Diagrama de secuencia domicilios

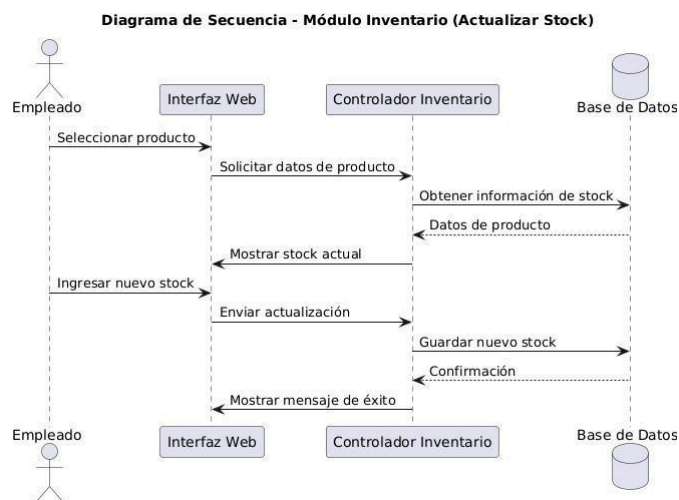


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Para el manejo del stock, el diagrama de secuencia de la Figura 14 muestra el módulo de inventario:

Figura 14

Diagrama de secuencia modulo inventario

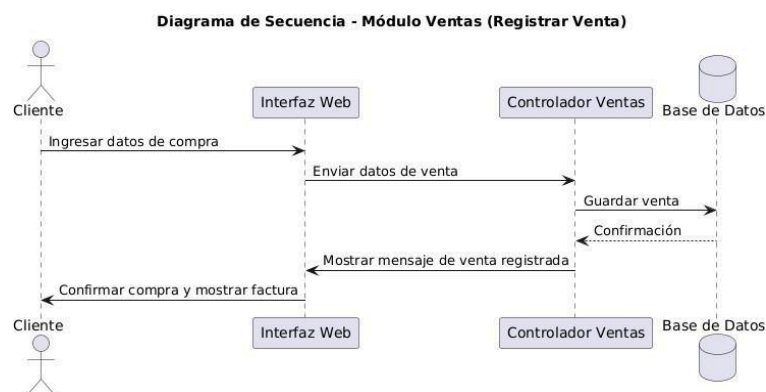


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Finalmente, el diagrama de la Figura 15 muestra el funcionamiento del paso de mensajes del módulo de ventas.

Figura 15

Diagrama de secuencia módulo de ventas



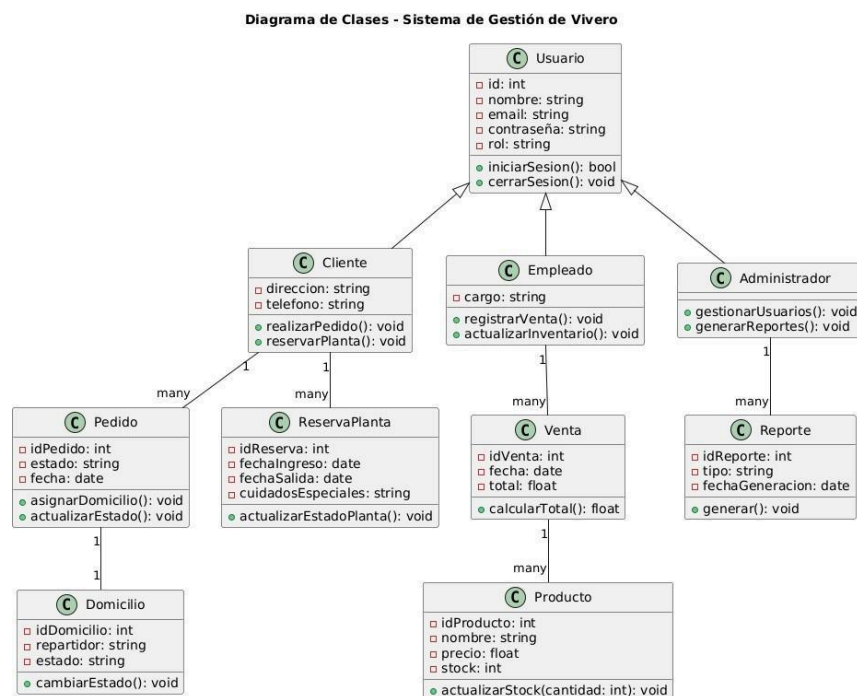
Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Construir el Modelo de Dominio del Sistema

La construcción del Modelo de Dominio del Sistema, representado mediante un diagrama de clases, permite identificar y estructurar los elementos clave del software. Este modelo define las entidades principales, sus atributos, métodos y relaciones, reflejando el comportamiento y la lógica del negocio. Es una herramienta esencial para guiar el desarrollo coherente y orientado a objetos del sistema. En la Figura 16 se observa el diagrama propuesto para la gestión del vivero.

Figura 16

Diagrama de clases sistema de gestión del vivero



El modelo entidad relación

El Modelo Entidad-Relación (MER) es una técnica clave para el diseño de bases de datos, ya que permite representar de manera visual las entidades del sistema y sus conexiones. Facilita la comprensión de la estructura y organización de los datos. Es esencial para asegurar la integridad y

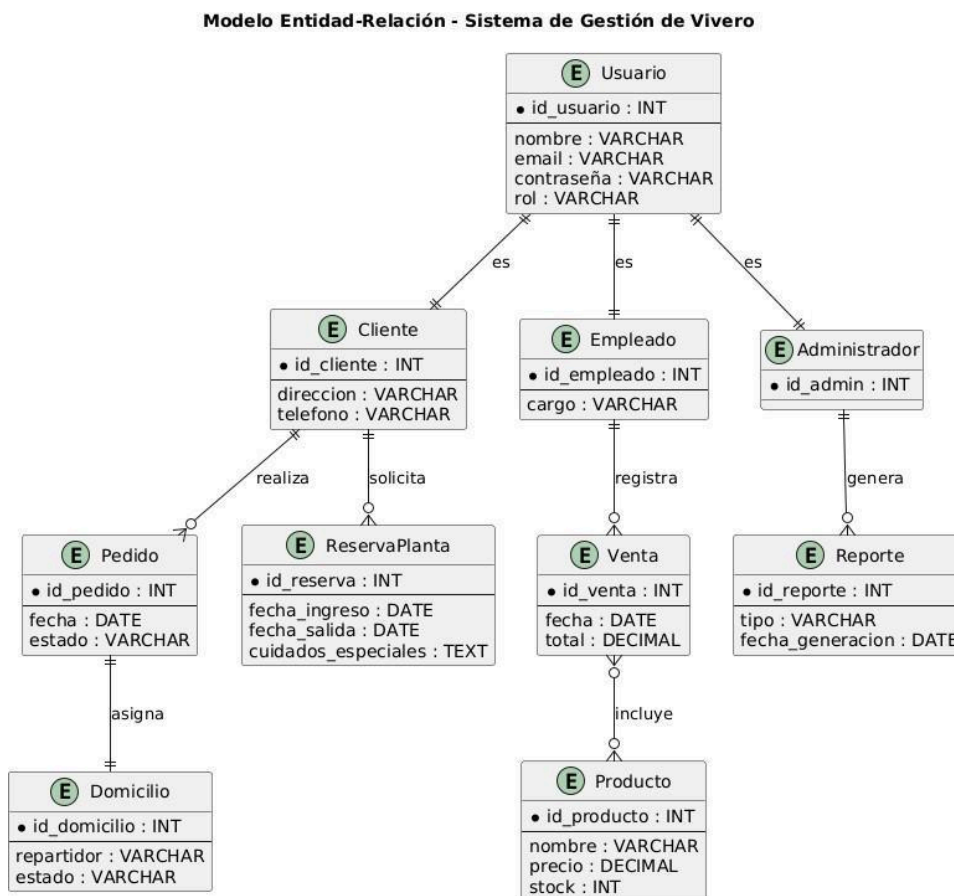
coherencia de la información almacenada. En la Figura 17 se detalla el diagrama desarrollado para el proyecto.

Se compone de entidades, que representan objetos o conceptos importantes del sistema. Cada entidad tiene atributos, los cuales describen sus características o propiedades. Las relaciones muestran cómo se conectan o interactúan las diferentes entidades entre sí.

Este modelo facilita el análisis, diseño y comunicación del esquema de una base de datos antes de su implementación.

Figura 17

Modelo entidad-relación (notación moderna)



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Diseño de la Solución del Software de acuerdo con los Procedimientos y Requisitos Técnicos

El diseño de la solución del software se realiza en función de los procedimientos establecidos y los requisitos técnicos definidos previamente. Esto garantiza que el sistema cumpla con los objetivos funcionales y operativos esperados.

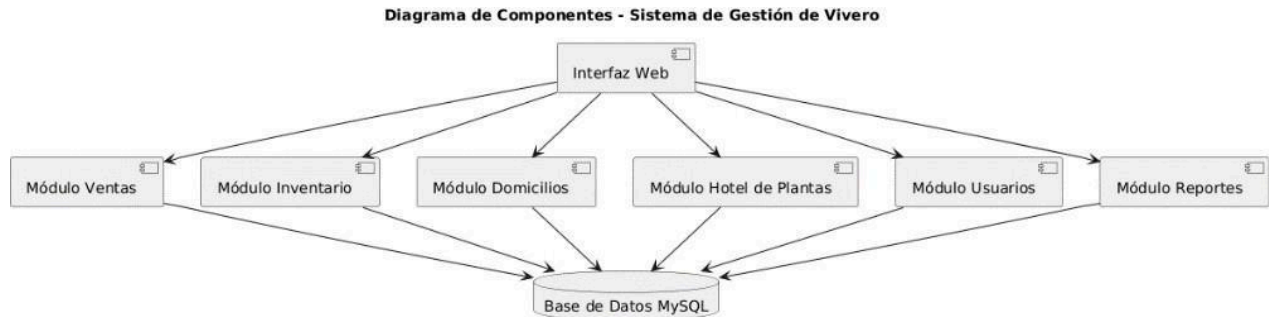
Arquitectura del software (diagrama de componentes), y patrones de diseño de software

La arquitectura del software, representada a través de un diagrama de componentes, define la estructura y la interacción entre los distintos módulos del sistema. Además, la aplicación de patrones de diseño optimiza la solución, ofreciendo soluciones probadas a problemas comunes. Este enfoque asegura una implementación más escalable, mantenible y eficiente del software.

En la Figura 18 se señala los componentes del sistema propuesto.

Figura 18

Diagrama de componentes



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Respecto a el patrón de diseño es Modelo-Vista-Controlador (MVC). Una estructura que separa la lógica de la aplicación en tres componentes principales: el Modelo, la Vista y el Controlador. El Modelo gestiona los datos y la lógica del negocio, la Vista se encarga de la presentación de la información, y el Controlador actúa como intermediario entre ambos. Esta separación facilita el mantenimiento, la escalabilidad y la reutilización del código. Además, permite desarrollar de forma más

organizada y colaborativa. MVC es ampliamente utilizado en aplicaciones web por su eficiencia y claridad estructural.

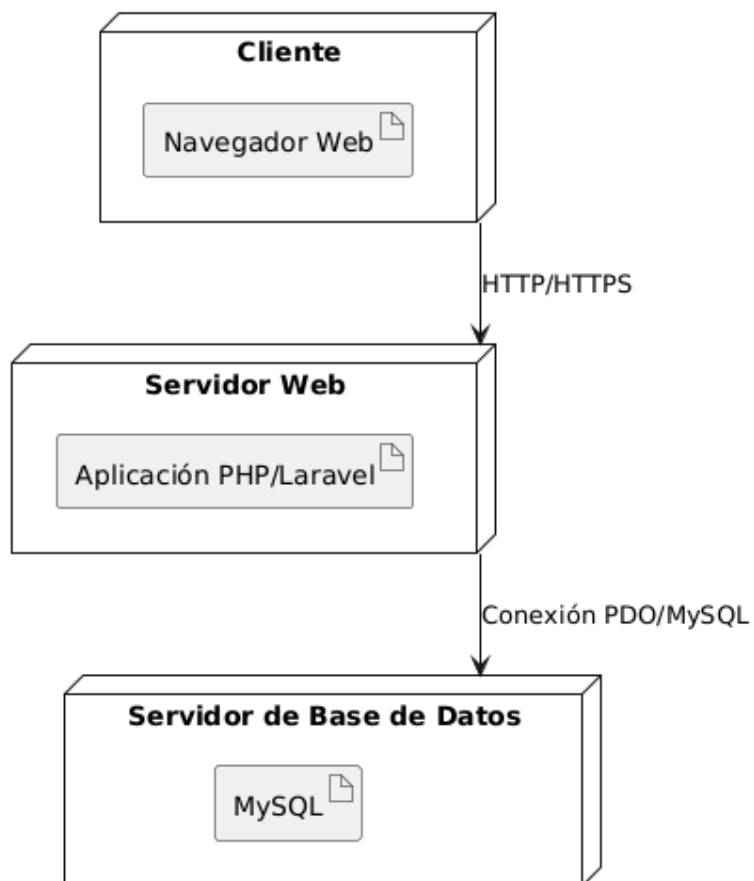
Diagrama de despliegue

El diagrama de despliegue representa la distribución física de los componentes del software en el hardware del sistema. Muestra cómo se instalan y comunican los módulos en servidores, dispositivos o nodos de red. Es útil para visualizar la infraestructura necesaria y asegurar una implementación eficiente. En la Figura 19 se explica la planteada para el software a desarrollar.

Figura 19

Diagrama de despliegue

Diagrama de Despliegue - Sistema de Gestión de Vivero



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Diseño Front-End - Interfaces Gráficas de Usuario

El diseño Front-End se enfoca en crear interfaces gráficas de usuario interactivas y atractivas utilizando tecnologías como HTML, CSS y JavaScript. Estas herramientas permiten estructurar, estilizar y dar dinamismo a las páginas web. El framework Bootstrap facilita el desarrollo con componentes reutilizables y un diseño responsivo. Gracias a esta combinación, se garantiza una experiencia de usuario intuitiva y adaptable a distintos dispositivos.

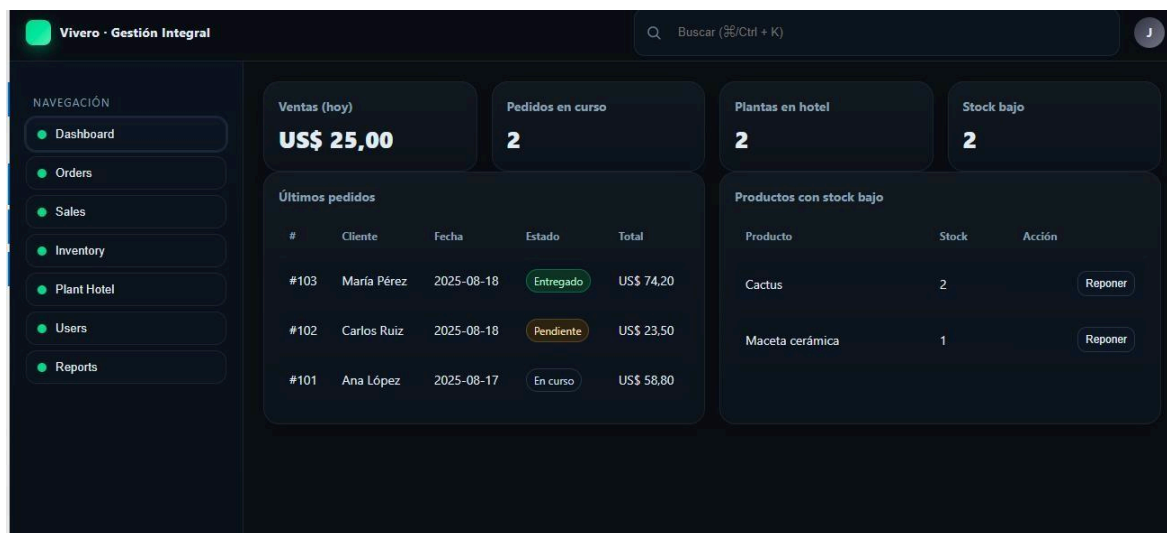
HTML (HyperText Markup Language) es el lenguaje de marcado que define la estructura y el contenido de una página web. CSS (Cascading Style Sheets) se utiliza para dar estilo, colores, tipografías y diseño visual a los elementos definidos en HTML. JavaScript (JS) es el lenguaje de programación que añade interactividad y dinamismo a los sitios web.

Juntos forman la base del desarrollo front-end, permitiendo crear páginas atractivas y funcionales.

En la Figura 20 se tiene la vista de la pantalla del dashboard del administrador.

Figura 20

Front del dashboard administrativo

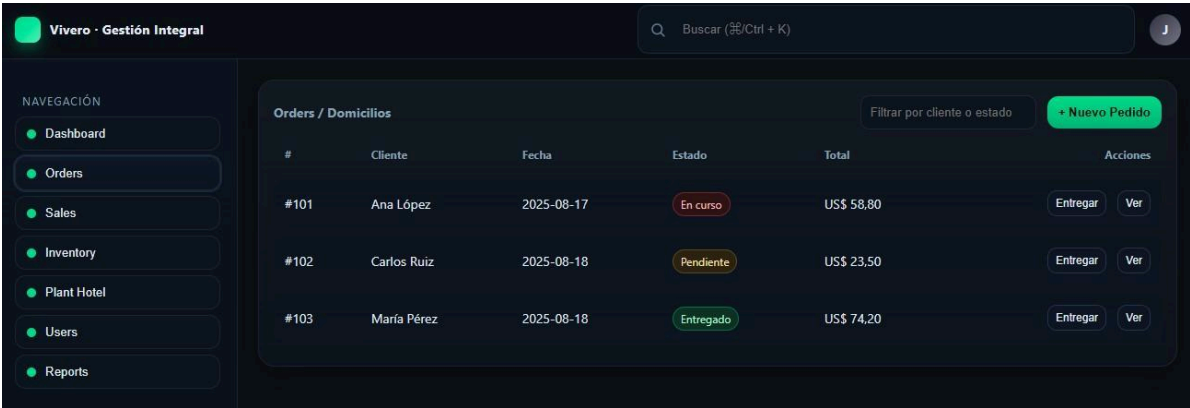


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la Figura 21 se tiene la vista de las Ordenes o pedidos que se reciben en la aplicación.

Figura 21

Front de orders (pedidos)

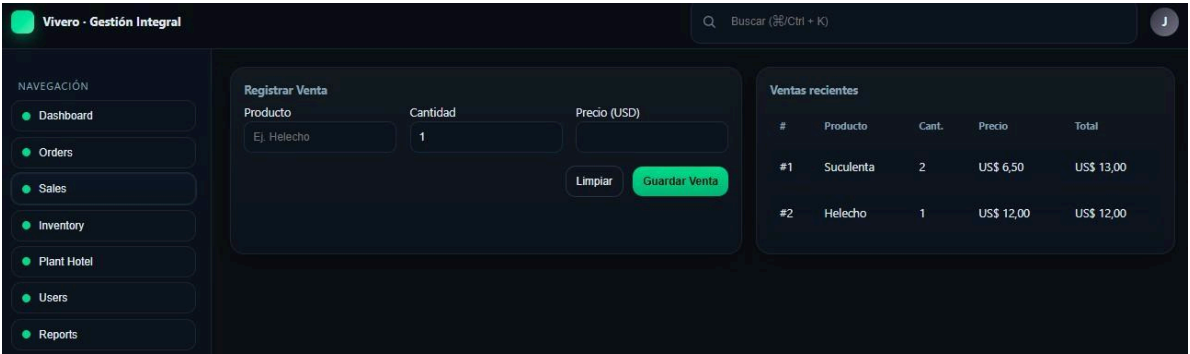


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Para las ventas o sales, se tiene en el front de la Figura 22, las opciones de la gestión de la venta, resaltando el registro de la venta.

Figura 22

Front de sales (ventas)

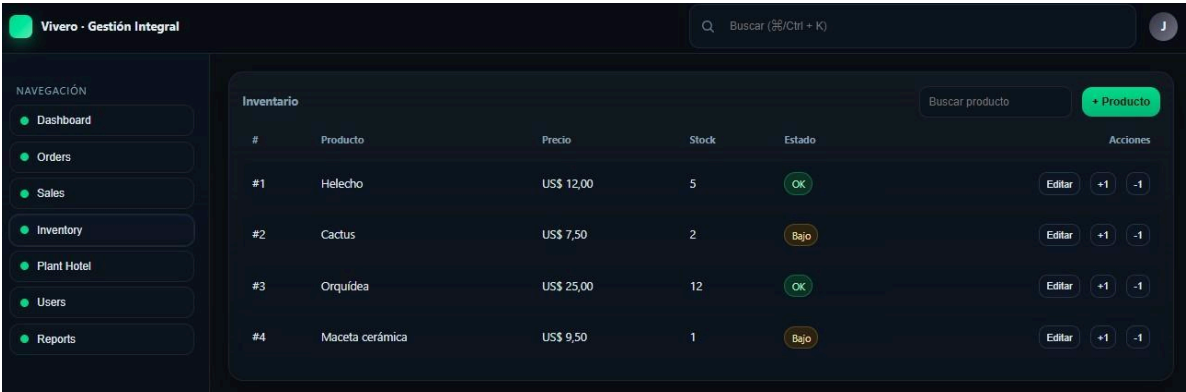


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En la gestión de inventario el front de la Figura 23, muestra las opciones adicionales en cada fila del producto inventariado.

Figura 23

Front gestión de inventario

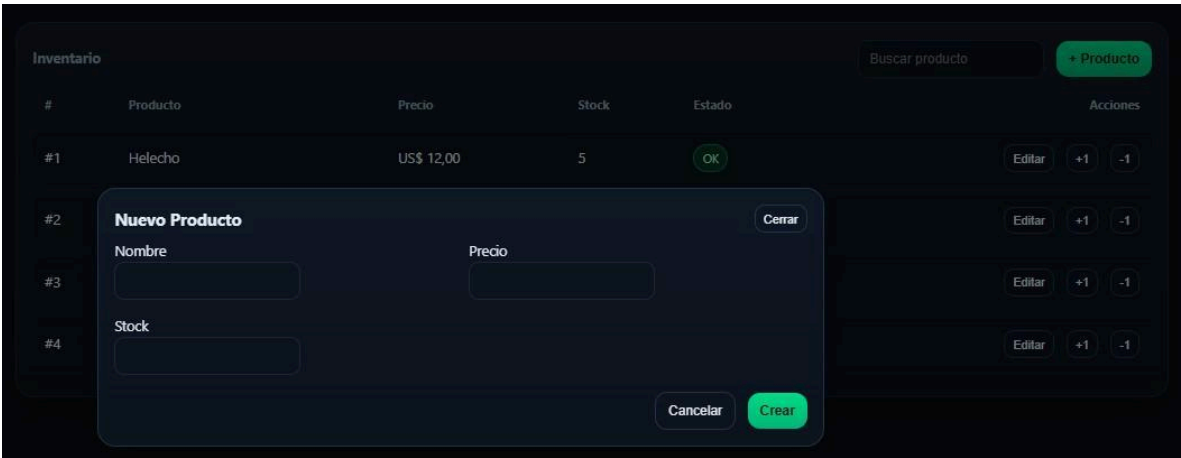


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

En el front del registro de producto se solicitan los datos básicos para agregar al inventario como la Figura 24 lo muestra a continuación:

Figura 24

Front registro de producto

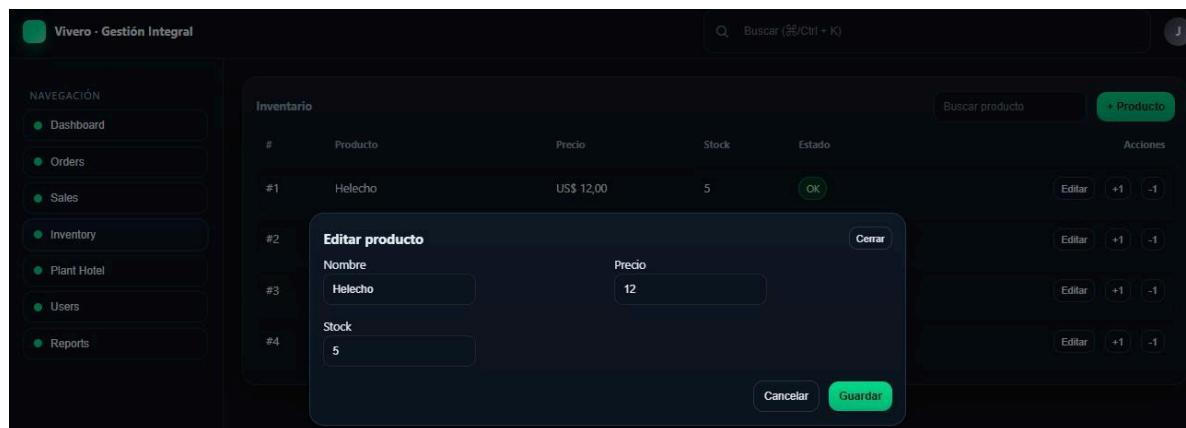


Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Para la vista de la edición del producto tenemos 3 campos principales, nombre, precio y stock, como se muestra en la Figura 25.

Figura 25

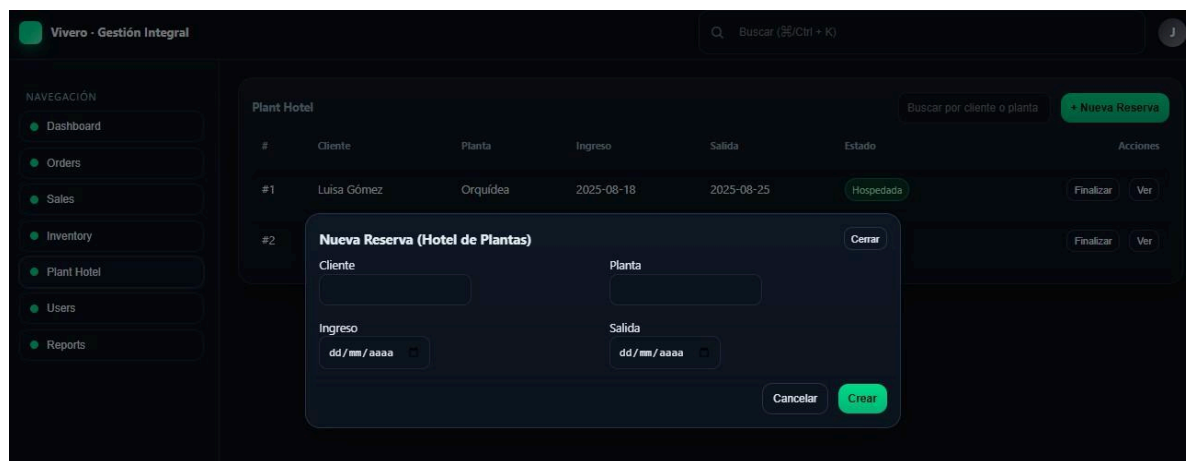
Front editar producto



En la Figura 26, se muestra el formulario para el registro de la reserva en el hotel de plantas. Allí se requiere los datos del cliente, fecha de entrada y de salida. En el fondo de la Figura se puede detallar el módulo relacionado llamado Plant Hotel.

Figura 26

Front registrar reserva al hotel de plantas

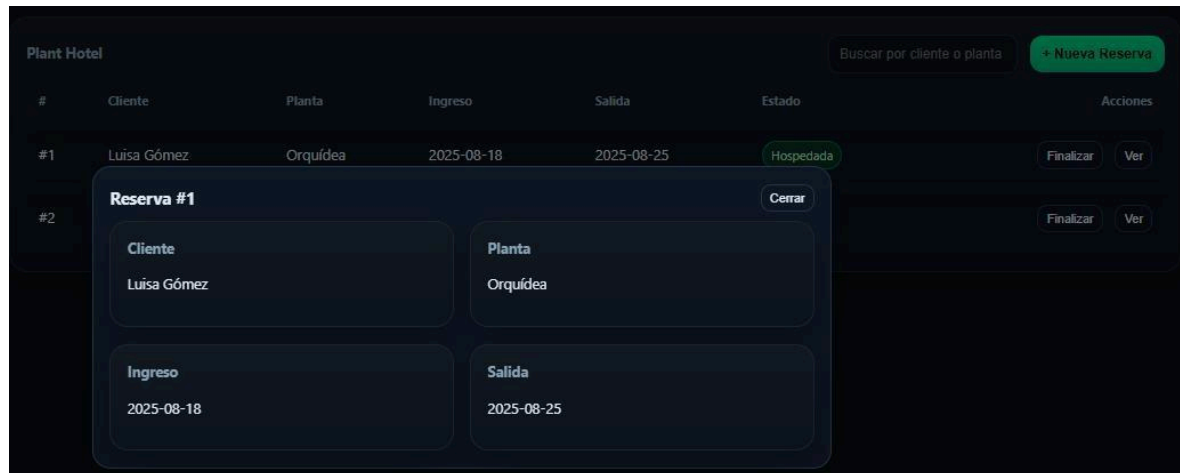


Al realizar la reserva se requieren detalles de la misma cuando se realice la consulta o seguimiento de que plantas se encuentran en el hotel.

En la Figura 27, se visualizan los detalles y el número de reserva.

Figura 27

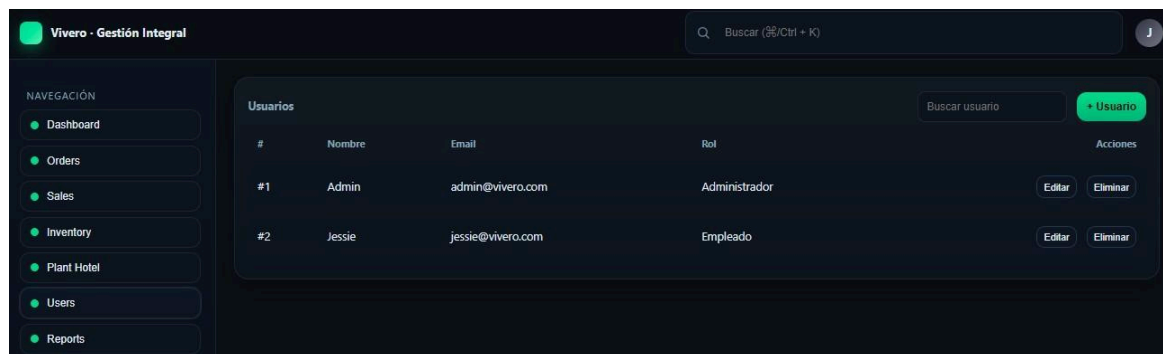
Front detalles de la consulta de la reserva



La vista para gestionar los usuarios tiene las acciones básicas de registrar, resaltada en color verde, editar y eliminar. La consulta está establecida al momento de dar clic en la opción de users como se muestra en la Figura 28.

Figura 28

Front de la gestión de usuarios

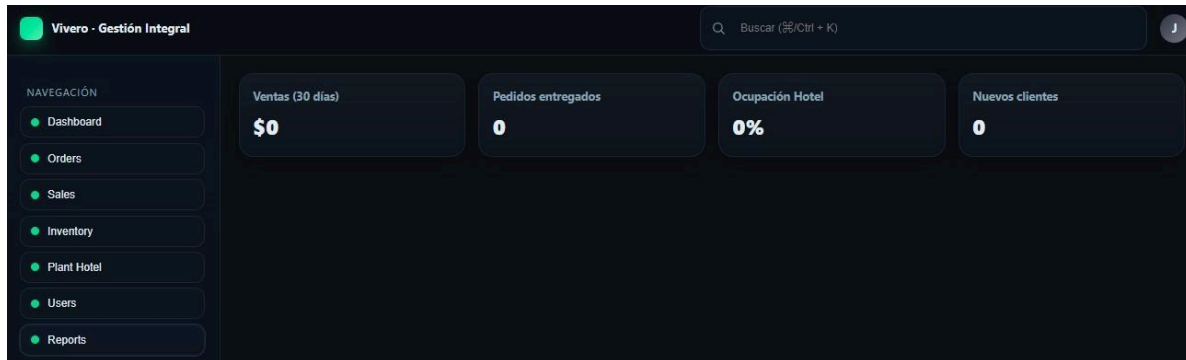


Posteriormente se tiene la vista de compilación de reportes, esta se difiere del dashboard porque tendrá solo información de totales de cada área sin generar las gráficas del dashboard, esto a solicitud del cliente que proponía una sección llamada Reports.

En la Figura 29, se detalla los datos específicos que tendrán dichos reportes como ventas del mes, pedidos entregados, ocupación del hotel y clientes nuevos.

Figura 29

Front de los resúmenes de reportes



Interfaces Gráficas de Usuario Móviles

Las interfaces gráficas de usuario móviles están diseñadas para ofrecer una experiencia fluida y optimizada en dispositivos móviles. Se enfocan en la usabilidad y la adaptación a pantallas pequeñas, garantizando accesibilidad en todas las funciones.

Su función principal es mostrar información de forma clara y accesible en pantallas pequeñas.

Facilitan la navegación mediante botones, menús, gestos y controles táctiles.

Ofrecen retroalimentación inmediata al usuario frente a sus acciones, como notificaciones o animaciones.

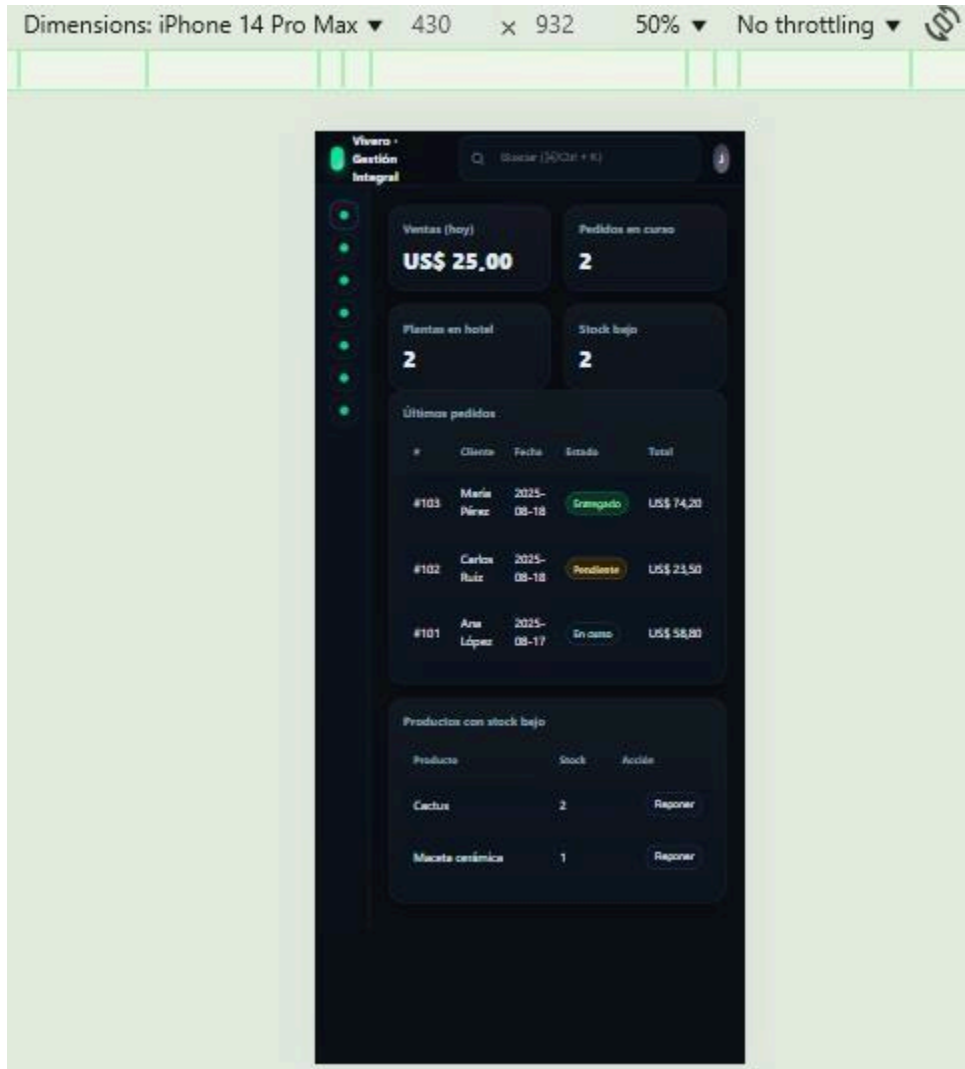
Además, garantizan una experiencia intuitiva y adaptable según el dispositivo y el sistema operativo móvil.

A continuación, se detalla las dimensiones del dispositivo usado como prueba utilizando la inspección del navegador para lograr el efecto y poder revisar el responsive para dispositivos de diferentes tamaños.

En la Figura 30, se muestra el dashboard del administrador en una vista de Portrait, es decir, Se refiere a la orientación donde el alto de la pantalla es mayor que el ancho, como en un teléfono móvil visto en posición normal.

Figura 30

Front del dashboard desde el punto de vista móvil



Un proceso común es la venta de las plantas, este proceso lo puede hacer muchos usuarios desde su celular, por lo que el responsive adquiere un mayor valor para facilitarle al usuario hacer las operaciones requeridas.

En la Figura 31, se puede detallar la adaptación vertical del formulario de registro de Venta y la tabla de consultas.

Figura 31

Front de ventas desde el punto de vista móvil reponsive

The screenshot shows a mobile application interface for sales management. At the top, there is a header with the text "Vivero - Gestión Integral" and a search bar with the placeholder "Buscar (⌘/Ctrl + K)". Below the header is a sidebar with a list of menu items, each preceded by a green dot. The main content area is divided into two sections. The first section, titled "Registrar Venta", contains a form with the following fields: "Producto" (with the example text "Ej. Hulecho"), "Cantidad" (with the value "1"), and "Precio (USD)". Below these fields are two buttons: "Limpiar" and "Guardar Venta". The second section, titled "Ventas recientes", displays a table of recent sales.

#	Producto	Cant.	Precio	Total
#1	Suculenta	2	US\$ 6.50	US\$ 13.00
#2	Hulecho	1	US\$ 12.00	US\$ 12.00

La vista de la Figura 32, muestra la adaptabilidad de la pantalla en reportes.

Figura 32

Front desde el punto de vista móvil de los reportes

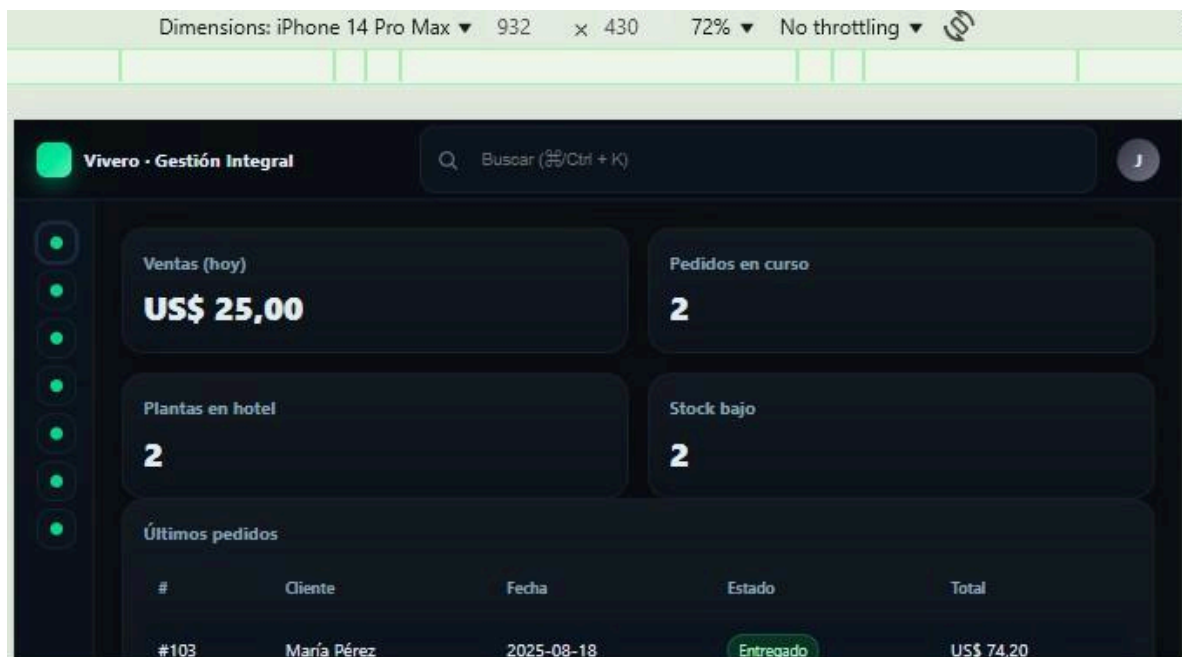
The screenshot shows a mobile application interface for reports. At the top, there is a header with the text "Vivero - Gestión Integral" and a search bar with the placeholder "Buscar (⌘/Ctrl + K)". Below the header is a sidebar with a list of menu items, each preceded by a green dot. The main content area is divided into four sections, each with a title and a large numerical value. The first section, titled "Ventas (30 días)", shows a value of "\$0". The second section, titled "Pedidos entregados", shows a value of "0". The third section, titled "Ocupación Hotel", shows a value of "0%". The fourth section, titled "Nuevos clientes", shows a value of "0".

Otra vista responsive es la landscape hace referencia a la orientación horizontal de una pantalla o diseño. Se utiliza comúnmente en dispositivos móviles, tabletas y presentaciones cuando el ancho es mayor que la altura. Esta disposición facilita la visualización de elementos amplios como tablas, gráficos o videos. En diseño de interfaces, permite aprovechar mejor el espacio para mostrar más contenido de manera clara. Es clave para adaptar aplicaciones y páginas web a diferentes dispositivos mediante el diseño responsive.

En la Figura 33 se realiza a través de la inspección del navegador la revisión del aspecto del dashboard en landscape.

Figura 33

Front dashboard landscape



Mapa de Navegación

El mapa de navegación en formato de texto jerárquico representa la estructura y organización de las secciones del sistema. Permite visualizar cómo se conectan las diferentes pantallas o módulos, facilitando la planificación de la experiencia del usuario. Es útil para asegurar una navegación lógica, clara y coherente.

Login / Registro

- Dashboard
- Orders (Pedidos)
 - Crear Pedido
 - Ver/Editar Pedido
- Sales (Ventas)
 - Registrar Venta
 - Historial de Ventas
- Inventory (Inventario)
 - Consultar Stock
 - Actualizar Stock
 - Agregar Producto
- Plant Hotel
 - Registrar Reserva
 - Consultar Estado
 - Finalizar Estancia
- Users (Usuarios – Solo Admin)
 - Listado de Usuarios
 - Crear/Editar Usuario
- Reports (Reportes – Solo Admin)
 - Ventas por Fecha
 - Productos Más Vendidos
 - Reporte de Domicilios

Determinar Tipos de Bases de Datos

Determinar los tipos de bases de datos permite clasificar y seleccionar el sistema más adecuado según las necesidades de almacenamiento, gestión y consulta de la información. Para el proyecto el tipo de base de datos seleccionada es:

- Base de datos relacional (SQL – MySQL / MariaDB)

Este tipo de bases de datos es ideal para las gestiones de inventario, ventas y pedidos, usuarios y roles. Entre sus ventajas esta:

- Integridad referencial.
- Uso de claves primarias/foráneas.
- Lenguaje estándar SQL para consultas.

Modelo de Datos Diagrama ER Normalización base de datos (tercera forma normal)

El modelo de datos, el diagrama entidad-relación (ER) y la normalización son fundamentales en el diseño de bases de datos eficientes y coherentes. La normalización, especialmente hasta la tercera forma normal (3FN), permite eliminar redundancias y asegurar la integridad de los datos a través de una estructura lógica bien definida.

Los elementos que justifican que la base de datos esta hasta la 3FN es:

En la 1FN cada atributo tiene valores atómicos (no listas, no repeticiones).

Para la 2FN las Tablas con claves compuestas (detalle_ventas, detalle_hotel) no tienen dependencias parciales y en la forma 3FN no existen dependencias transitivas (ej. dirección del cliente está en clientes, no en ventas).

Este modelo permite:

- Controlar usuarios y clientes sin redundancia.
- Manejar ventas con detalle.
- Gestionar inventario en tiempo real.

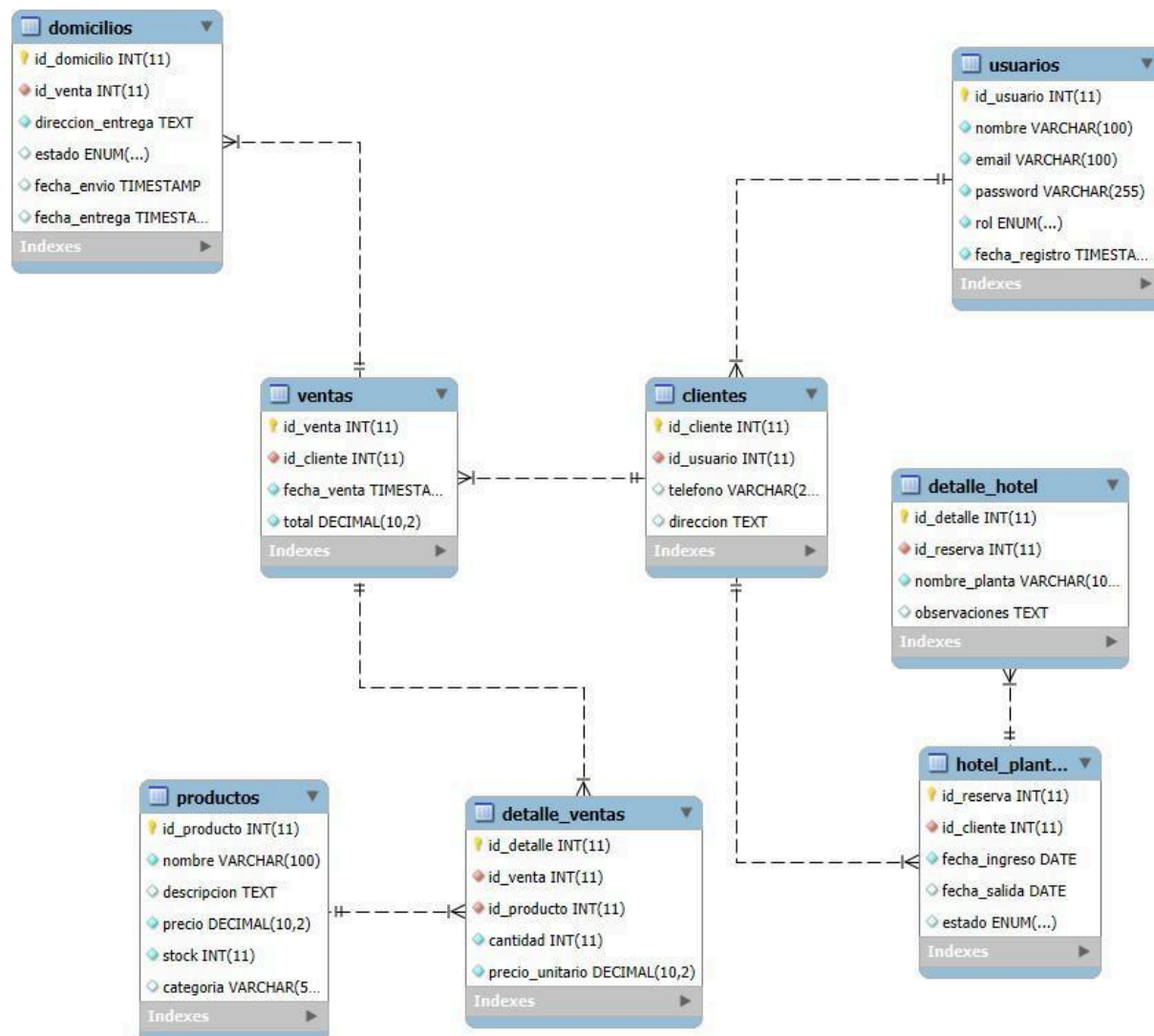
- Registrar y dar seguimiento a domicilios.
- Administrar el hotel de plantas con reservas y detalle.

En la Figura 34, se muestra el modelo relacional generado usando la herramienta MySQL

Workbench.

Figura 34

Modelo relacional de Viverapp



Diccionario de Datos

El diccionario de datos es una herramienta esencial que describe detalladamente los elementos de una base de datos, incluyendo nombres de tablas, campos, tipos de datos y relaciones. Su objetivo es facilitar la comprensión, administración y mantenimiento del sistema de información.

En la Tabla 12, se detallan los campos de la base de datos.

Tabla 12

Diccionario de datos

Entidad / Tabla	Campo	Tipo de Dato	Descripción
Usuarios	id_usuario	Entero (PK)	Identificador único del usuario
Usuarios	nombre	Texto	Nombre completo del usuario
Usuarios	correo	Texto	Correo electrónico del usuario
Usuarios	contraseña	Texto (hash)	Contraseña cifrada para acceso al sistema
Usuarios	tipo_usuario	Texto	Rol del usuario (Cliente, Administrador, Repartidor)
Usuarios	teléfono	Texto	Número de contacto del usuario
Usuarios	dirección	Texto	Dirección de residencia o entrega
Productos	id_producto	Entero (PK)	Identificador único del producto
Productos	nombre	Texto	Nombre de la planta o producto
Productos	descripción	Texto	Descripción breve del producto
Productos	precio	Decimal	Precio de venta
Productos	stock	Entero	Cantidad disponible en inventario
Productos	categoría	Texto	Tipo de planta o accesorio
Productos	imagen	Texto (URL)	Ruta o enlace a la imagen del producto

Entidad / Tabla	Campo	Tipo de Dato	Descripción
Pedidos	id_pedido	Entero (PK)	Identificador del pedido
Pedidos	id_usuario	Entero (FK)	Usuario que realizó el pedido
Pedidos	fecha_pedido	Fecha-Hora	Fecha y hora en que se realizó el pedido
Pedidos	estado	Texto	Estado del pedido (Pendiente, Enviado, Entregado, Cancelado)
Pedidos	total	Decimal	Valor total del pedido
Detalle_Pedido	id_detalle	Entero (PK)	Identificador del detalle del pedido
Detalle_Pedido	id_pedido	Entero (FK)	Pedido al que pertenece
Detalle_Pedido	id_producto	Entero (FK)	Producto comprado
Detalle_Pedido	cantidad	Entero	Número de unidades solicitadas
Detalle_Pedido	subtotal	Decimal	Subtotal por cada producto (precio * cantidad)
Domicilios	id_domicilio	Entero (PK)	Identificador único del servicio de domicilio
Domicilios	id_pedido	Entero (FK)	Pedido asociado al domicilio
Domicilios	dirección_entrega	Texto	Dirección exacta del cliente
Domicilios	fecha_programada	Fecha-Hora	Fecha y hora solicitada para la entrega
Domicilios	estado	Texto	Estado del domicilio (Programado, En ruta, Entregado)
Reservas_Hotel	id_reserva	Entero (PK)	Identificador único de la reserva

Entidad / Tabla	Campo	Tipo de Dato	Descripción
Reservas_Hotel	id_usuario	Entero (FK)	Usuario que deja la planta en custodia
Reservas_Hotel	nombre_planta	Texto	Nombre o identificación de la planta
Reservas_Hotel	tipo_planta	Texto	Tipo de planta (interior, exterior, suculenta, etc.)
Reservas_Hotel	fecha_ingreso	Fecha	Fecha en la que ingresa la planta
Reservas_Hotel	fecha_salida	Fecha	Fecha estimada de retiro
Reservas_Hotel	observaciones	Texto	Instrucciones o cuidados específicos
Reservas_Hotel	estado	Texto	Estado de la planta (En cuidado, Retirada, Cancelada)
Seguimiento_Plantas	id_seguimiento	Entero (PK)	ID del seguimiento registrado
Seguimiento_Plantas	id_reserva	Entero (FK)	Reserva asociada a la planta hospedada
Seguimiento_Plantas	fecha	Fecha	Fecha del seguimiento
Seguimiento_Plantas	nota_cuidado	Texto	Observación o registro del estado de la planta
Seguimiento_Plantas	imagen	Texto (URL)	Imagen de la planta en su estado actual (opcional)

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Se recuerda que:

- PK es clave primaria
- FK es clave foránea (relación con otra tabla)
- Se pueden agregar campos adicionales según evolución del proyecto (por ejemplo, para historial de pagos, calificaciones, etc.)

Políticas de Seguridad de los Datos

Las Políticas de Seguridad de los Datos son un conjunto de normas y directrices diseñadas para proteger la confidencialidad, integridad y disponibilidad de la información dentro de una organización.

Su propósito es prevenir accesos no autorizados, pérdidas o alteraciones de datos, garantizando un uso seguro y responsable de los recursos digitales.

Protección de acceso

Se promueve:

- Uso de autenticación segura (contraseñas encriptadas)
- Implementación de roles y permisos para limitar el acceso a funcionalidades según el perfil del usuario (administrador, empleado, cliente).
- Doble factor de autenticación (2FA) para administradores.

Confidencialidad

Para la protección de información:

- Los datos personales de clientes (teléfonos, direcciones, correos) solo serán accesibles para personal autorizado.

Integridad

En cuanto a la consistencia de los datos se requiere:

- Uso de restricciones en la base de datos (claves foráneas, validaciones) para garantizar consistencia.
- Backups periódicos para recuperación en caso de fallos.

Disponibilidad

- Mecanismos de respaldo diario y recuperación ante desastres (DRP – Disaster Recovery Plan).
- Uso de servidores escalables en la nube para asegurar disponibilidad en picos de demanda.

Privacidad

- Cumplimiento de la normativa vigente sobre protección de datos. Ley de Habeas Data en Colombia.
- Consentimiento informado de los clientes para uso de sus datos.

Seguridad en las comunicaciones

- Todo intercambio de información entre cliente y servidor debe estar protegido con **HTTPS**.
- Políticas de **CORS** configuradas para evitar accesos no autorizados desde orígenes externos.

Gestión de incidentes

- Procedimientos para detección, reporte y respuesta a incidentes de seguridad.
- Registro de intentos fallidos de inicio de sesión y bloqueo temporal de cuentas sospechosas.

Control de copias de seguridad

- Almacenamiento de copias de seguridad en medios seguros y cifrados.
- Definición de políticas de retención y eliminación segura de datos obsoletos.

Construcción del Software

La construcción del software es la etapa del desarrollo donde se codifican, integran y prueban los componentes del sistema para crear una aplicación funcional. Esta fase transforma los diseños y especificaciones en un producto operativo listo para su despliegue.

Base de Datos para el Software a partir del Modelo de Datos

Este proceso garantiza que la organización, relaciones y restricciones de los datos estén alineadas con los requisitos del sistema. Se representa la codificación en SQL de la estructura lógica de la información del software propuesto.

Tabla Usuarios

El script de la tabla de usuarios es:

```
CREATE TABLE usuarios (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  rol ENUM('admin','empleado','cliente') NOT NULL,
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Normalización:

- No hay dependencias transitivas.
- Email único.

Tabla Clientes

La tabla clientes está separada de usuarios para detalle específico de clientes. El script es:

```
CREATE TABLE clientes (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario INT NOT NULL,
  telefono VARCHAR(20),
  direccion TEXT,
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);
```

Normalización:

- Evita redundancia (datos de acceso están en usuarios).

Tabla Productos / Inventario

El script de la tabla de productos es:

```
CREATE TABLE productos (
  id_producto INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion TEXT,
  precio DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0,
  categoria VARCHAR(50)
);
```

Normalización:

- Stock controlado en un solo campo.

Tabla Ventas

El script de la tabla de ventas es:

```
CREATE TABLE ventas (
  id_venta INT AUTO_INCREMENT PRIMARY KEY,
  id_cliente INT NOT NULL,
  fecha_venta TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  total DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

Normalización:

- Cumplir con 1FN.
- Todos los atributos que no son clave deben depender completamente de la clave primaria (sin dependencias parciales).

Tabla Detalle de Ventas

El script de la tabla detalle ventas para evitar la repetición de productos en ventas es:

```
CREATE TABLE detalle_ventas (
  id_detalle INT AUTO_INCREMENT PRIMARY KEY,
```

```

id_venta INT NOT NULL,
id_producto INT NOT NULL,
cantidad INT NOT NULL,
precio_unitario DECIMAL(10,2) NOT NULL,
FOREIGN KEY (id_venta) REFERENCES ventas(id_venta),
FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
);

```

Normalización:

- Cumple 3FN separando cabecera y detalle.

Tabla Domicilios

El script de la tabla de domicilios, que son los pedidos a domicilios es:

```

CREATE TABLE domicilios (
  id_domicilio INT AUTO_INCREMENT PRIMARY KEY,
  id_venta INT NOT NULL,
  direccion_entrega TEXT NOT NULL,
  estado ENUM('pendiente','en_camino','entregado') DEFAULT 'pendiente',
  fecha_envio TIMESTAMP NULL,
  fecha_entrega TIMESTAMP NULL,
  FOREIGN KEY (id_venta) REFERENCES ventas(id_venta)
);

```

Normalización:

- Cumple porque la tabla ya está en 1FN y todos los atributos no clave dependen completamente de la clave primaria (id_domicilio).
- No hay clave compuesta, por lo tanto, no existen dependencias parciales.

Tabla Hotel de Plantas

El script para el servicio de cuidado temporal utilizado para el hotel de plantas es:

```

CREATE TABLE hotel_plantas (
  id_reserva INT AUTO_INCREMENT PRIMARY KEY,
  id_cliente INT NOT NULL,
  fecha_ingreso DATE NOT NULL,
  fecha_salida DATE,
  estado ENUM('activa','finalizada') DEFAULT 'activa',
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);

```

Normalización:

- Cumple la 3FN porque no hay dependencias transitivas entre atributos no clave.
- Todos los campos (id_cliente, fecha_ingreso, fecha_salida, estado) dependen directamente de id_reserva, y no unos de otros.

Tabla Detalle de Hotel de Plantas

El script de la tabla detalle hotel es:

```
CREATE TABLE detalle_hotel (
  id_detalle INT AUTO_INCREMENT PRIMARY KEY,
  id_reserva INT NOT NULL,
  nombre_planta VARCHAR(100) NOT NULL,
  observaciones TEXT,
  FOREIGN KEY (id_reserva) REFERENCES hotel_plantas(id_reserva)
);
```

Objetos de la Base de Datos (procedimientos almacenados, vistas, disparadores)

Los objetos de la base de datos son las estructuras utilizadas para almacenar, organizar y gestionar la información. Entre ellos se incluyen tablas, vistas, índices, procedimientos almacenados, entre otros elementos esenciales para el funcionamiento del sistema.

Procedimiento almacenado para registrar una venta:

```
DELIMITER $$

CREATE PROCEDURE RegistrarVenta(
  IN p_cliente_id INT,
  IN p_usuario_id INT,
  IN p_total DECIMAL(10,2)
)
BEGIN
  INSERT INTO Ventas (cliente_id, usuario_id, fecha, total, estado)
  VALUES (p_cliente_id, p_usuario_id, NOW(), p_total, 'Pendiente');
END$$

DELIMITER ;
```


Procedimiento almacenado para un pedido de domicilio:

```
DELIMITER $$

CREATE PROCEDURE RegistrarPedidoDomicilio(
    IN p_cliente_id INT,
    IN p_direccion VARCHAR(255),
    IN p_venta_id INT
)
BEGIN
    INSERT INTO Domicilios (venta_id, cliente_id, direccion, estado, fecha)
    VALUES (p_venta_id, p_cliente_id, p_direccion, 'En proceso', NOW());
END$$

DELIMITER ;
```

Procedimiento almacenado para registrar ingreso de planta al hotel:

```
DELIMITER $$

CREATE PROCEDURE RegistrarIngresoHotel(
    IN p_cliente_id INT,
    IN p_planta_nombre VARCHAR(100),
    IN p_dias INT, IN p_costo DECIMAL(10,2)
)
BEGIN
    INSERT INTO HotelPlantas (cliente_id, planta_nombre, fecha_ingreso, fecha_salida, costo, estado)
    VALUES (p_cliente_id, p_planta_nombre, NOW(), DATE_ADD(NOW(), INTERVAL p_dias DAY),
    p_costo, 'En cuidado');
END$$

DELIMITER ;
```

Vista de inventario disponible:

```
CREATE VIEW vw_inventario_disponible AS
SELECT
    p.producto_id,
    p.nombre,
    p.descripcion,
    i.stock,
    p.precio
FROM Productos p
JOIN Inventario i ON p.producto_id = i.producto_id
WHERE i.stock > 0;
```

Vista de ventas con clientes:

```
CREATE VIEW vw_ventas_clientes AS
SELECT
    v.venta_id,
    c.nombre AS cliente,
    v.fecha,
    v.total,
    v.estado
FROM Ventas v
JOIN Clientes c ON v.cliente_id = c.cliente_id;
```

Vista de pedidos en curso:

```
CREATE VIEW vw_pedidos_en_curso AS
SELECT
    d.domicilio_id,
    c.nombre AS cliente,
    d.direccion,
    d.estado,
    d.fecha
FROM Domicilios d
JOIN Clientes c ON d.cliente_id = c.cliente_id
WHERE d.estado <> 'Entregado';
```

Trigger disminuir stock al registrar venta:

```
DELIMITER $$

CREATE TRIGGER trg_disminuir_stock
AFTER INSERT ON DetalleVenta
FOR EACH ROW
BEGIN
    UPDATE Inventario
    SET stock = stock - NEW.cantidad
    WHERE producto_id = NEW.producto_id;
END$$
```

DELIMITER ;

Trigger actualizar estado de venta cuando el domicilio se entrega:

```
DELIMITER $$

CREATE TRIGGER trg_actualizar_venta_entregada
```

```

AFTER UPDATE ON Domicilios
FOR EACH ROW
BEGIN
    IF NEW.estado = 'Entregado' THEN
        UPDATE Ventas
        SET estado = 'Completada'
        WHERE venta_id = NEW.venta_id;
    END IF;
END$$

```

```
DELIMITER ;
```

Trigger auditoría de usuarios (registro de cambios)

```
DELIMITER $$
```

```

CREATE TRIGGER trg_auditoria_usuarios
AFTER UPDATE ON Usuarios
FOR EACH ROW
BEGIN
    INSERT INTO AuditoriaUsuarios(usuario_id, accion, fecha)
    VALUES (NEW.usuario_id, 'Actualización de datos', NOW());
END$$

```

```
DELIMITER ;
```

Por lo anterior, los objetos que cuenta la base de datos se recopilan en:

- Procedimientos para registrar ventas, domicilios y hotel de plantas.
- Vistas para consultar inventario, ventas y pedidos.
- Triggers para mantener integridad en stock, ventas y auditoría.

Esquemas de Seguridad de los Datos

Son estrategias y mecanismos diseñados para proteger la información contra accesos no autorizados, pérdidas o alteraciones.

Control de acceso a nivel de aplicación:

- Gestión de usuarios y roles (Clientes, Administradores, Empleados, Repartidores).

Autenticación segura:

- Implementación de OAuth2 para sesiones.

Autorización:

- Validación de permisos antes de ejecutar operaciones sensibles (ejemplo: solo admin puede eliminar usuarios o productos).

Seguridad de la información en tránsito:

- HTTPS (TLS/SSL) en toda comunicación cliente-servidor.
- Cifrado de datos sensibles en formularios.

Auditoría y monitoreo:

- Disparadores de auditoría para registrar cambios importantes (usuarios, stock, ventas).

Políticas de Seguridad:

- Contraseñas fuertes: mínimo 10 caracteres, combinación de mayúsculas, minúsculas, números y símbolos.
- Caducidad de contraseñas cada cierto tiempo.
- Bloqueo de cuenta tras múltiples intentos fallidos.
- Respaldo periódico (diario/semanal) y pruebas de restauración.
- Políticas de privacidad conforme a habeas data y leyes locales.

Codificación del Software de acuerdo con el Diseño Establecido

Front End

El front-end corresponde a la capa visual e interactiva del software, donde el usuario final interactúa con la aplicación. Su desarrollo integra HTML, CSS y JavaScript, permitiendo construir interfaces amigables, dinámicas y accesibles.

Código en el head:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Vivero • Gestión Integral</title>
```

Código para el CSS:

```
<style>
:root{
  --bg:#0e1116;    /* fondo app */
  --panel:#151a22;  /* paneles/cards */
  --panel-2:#1b2230; /* panel alterno */
  --muted:#8fa3b8;  /* texto secundario */
  --text:#e6eef7;   /* texto principal */
  --brand:#3ddc84;  /* acento (verde) */
  --brand-2:#2bb673; /* acento oscuro */
  --danger:#ff5d5d;
  --warn:#ffb020;
  --ok:#4cd964;
  --radius:18px;
  --shadow: 0 10px 30px rgba(0,0,0,.35);
}
*{box-sizing:border-box}
html,body{height:100%}
body{margin:0;background:linear-gradient(180deg,#0c1016, #0e1116
30%);color:var(--text);font:14px/1.5 system-ui, -apple-system, Segoe UI, Roboto, Ubuntu,
Cantarell, Noto Sans, Arial, "Apple Color Emoji","Segoe UI Emoji"}
```

```

a{color:inherit;text-decoration:none}
.app{display:grid;grid-template-rows:64px 1fr;min-height:100vh}

/* Topbar */
.topbar{display:flex;align-items:center;gap:16px;padding:12px
18px;background:#0b0f15;border-bottom:1px solid #1f2735;position:sticky;top:0;z-index:5}
.logo{display:flex;align-items:center;gap:10px;font-weight:700;letter-spacing:.3px}
.logo-badge{width:28px;height:28px;border-radius:8px;background:linear-gradient(
135deg,var(--brand),#7af1b1);box-shadow:0 6px 16px rgba(61,220,132,.35)}
.grow{flex:1}
.search{display:flex;align-items:center;gap:8px;background:#0f1520;border:1px solid
#202a3b;border-radius:12px;padding:8px 12px;max-width:540px;width:100%}
.search input{flex:1;background:transparent;border:0;outline:0;color:var(--text)}
.avatar{width:34px;height:34px;border-radius:50%;background:linear-gradi
ent(135deg,#334,#778);display:grid;place-items:center;font-weight:700}

/* Layout */
.content{display:grid;grid-template-columns:260px 1fr;min-height:calc(100vh - 64px)}
.sidebar{padding:18px;background:linear-gradient(180deg,#0e1420,#0d1118);border-
right:1px solid #1f2735}
.sidebar .group{font-size:12px;color:var(--muted);margin:18px 8px
8px;text-transform:uppercase;letter-spacing:.1em}
.nav{display:grid;gap:6px}
.nav button{display:flex;align-items:center;gap:10px;width:100%;padding:10px
12px;border-radius:12px;border:1px solid
#1f2735;background:#0f1520;color:var(--text);cursor:pointer;transition:.2s}
.nav button:hover{border-color:#2b364a}
.nav button.active{background:linear-gradient(180deg,#111827,#0f1520);outline:2px solid
#1f2a3f}
.nav .dot{width:10px;height:10px;border-radius:20px;background:var(--brand)}

/* Main area */
main{padding:22px;display:grid;gap:18px}
.kpis{display:grid;grid-template-columns:repeat(4, minmax(180px,1fr));gap:16px}
.card{background:linear-gradient(180deg,var(--panel),#0f141d);border:1px solid
#1f2735;border-radius:var(--radius);box-shadow:var(--shadow);padding:16px}
.card h3{margin:0 0 6px 0;font-size:14px;color:var(--muted)}
.card .num{font-size:26px;font-weight:800}

```

```

/* Tables */
.table{width:100%;border-collapse:separate;border-spacing:0 10px}
.table thead th{font-size:12px;color:var(--muted);text-align:left;padding:8px 12px}
.table tbody tr{background:linear-gradient(180deg,#0f1520,#101722);border:1px solid
#1f2735}
.table tbody td{padding:12px}
.table tbody tr td:first-child{border-top-left-radius:12px;border-bottom-left-radius:12px}
.table tbody tr td:last-child{border-top-right-radius:12px;border-bottom-right-radius:12px}
.badge{padding:4px 8px;border-radius:999px;font-size:12px;border:1px solid
#2b364a;color:#cfe6ff}
.badge.ok{border-color:#2c6e49;background:#183424;color:#bdfccb}
.badge.warn{border-color:#5a4b1f;background:#2b2412;color:#ffe7b0}
.badge.danger{border-color:#6e2c2c;background:#2b1515;color:#ffc3c3}

.row{display:flex;align-items:center;gap:12px}
.row.right{justify-content:flex-end}
.btn{border:1px solid
#2a3549;background:#101724;color:var(--text);border-radius:12px;padding:10
px 12px;cursor:pointer}
.btn.brand{background:linear-gradient(180deg,var(--brand),var(--brand-
2));border:0;color:#0b120d;font-weight:700}
.btn.ghost{background:transparent}
.btn.small{padding:6px 8px;border-radius:10px;font-size:12px}
input,select{background:#0f1520;border:1px solid #202a3b;color:var(--text);border-
radius:10px;padding:10px}

/* Views */
.view{display:none}
.view.active{display:block}

.grid-2{display:grid;grid-template-columns:1fr 1fr;gap:16px}
.grid-3{display:grid;grid-template-columns:repeat(3,1fr);gap:16px}

/* Modal */
.modal-backdrop{position:fixed;inset:0;background:rgba(0,0,0,.5);display:none;place-
items:center;z-index:50}
.modal{width:min(720px,92vw);background:linear-gradient(180deg
,#111826,#0f1520);border:1px solid
#2a3549;border-radius:18px;padding:16px;box-shadow:var(--shadow)
}

```

```
.modal-header{display:flex;align-items:center;justify-content:space-between;margin-bottom:8px}
```

```
/* Responsive */
@media (max-width: 1000px){
  .content{grid-template-columns:64px 1fr}
  .sidebar .group,.sidebar .label{display:none}
  .nav button{justify-content:center;padding:10px}
  .nav button .dot{margin:0}
  .kpis{grid-template-columns:repeat(2,1fr)}
  .grid-2{grid-template-columns:1fr}
  .grid-3{grid-template-columns:1fr}
}
</style>
</head>
```

Estructura general del body:

```
<body>
<div class="app">
  <!-- Topbar -->
  <header class="topbar">
    <div class="logo" aria-label="Inicio">
      <span class="logo-badge" aria-hidden="true"></span>
      <span>Vivero · Gestión Integral</span>
    </div>
    <div class="grow"></div>
    <label class="search" aria-label="Buscar en la aplicación">
      <svg width="18" height="18" viewBox="0 0 24 24" fill="none" aria-hidden="true"><path
d="M21 21l-4.35-4.35" stroke="#6f8198" stroke-width="2" stroke-linecap="round"/><circle
cx="11" cy="11" r="7" stroke="#6f8198" stroke-width="2" fill="none"/></svg>
      <input id="globalSearch" placeholder="Buscar (⌘/Ctrl + K)" />
    </label>
    <div class="avatar" title="Jessie">J</div>
  </header>
```

Creación del sidebar:

```
<!-- Body layout -->
<div class="content">
```



```

<!-- Sidebar -->
<aside class="sidebar">
  <div class="group">Navegación</div>
  <nav class="nav" id="nav">
    <button data-view="dashboard" class="active" aria-label="Dashboard"><span
class="dot"></span><span class="label">Dashboard</span></button>
    <button data-view="orders" aria-label="Pedidos/Domicilios"><span
class="dot"></span><span class="label">Orders</span></button>
    <button data-view="sales" aria-label="Ventas"><span class="dot"></span><span
class="label">Sales</span></button>
    <button data-view="inventory" aria-label="Inventario"><span class="dot"></span><span
class="label">Inventory</span></button>
    <button data-view="hotel" aria-label="Hotel de Plantas"><span class="dot"></span><span
class="label">Plant Hotel</span></button>
    <button data-view="users" aria-label="Usuarios"><span class="dot"></span><span
class="label">Users</span></button>
    <button data-view="reports" aria-label="Reportes"><span class="dot"></span><span
class="label">Reports</span></button>
  </nav>
</aside>

```

Sección del dashboard:

```

<!-- Main -->
<main>
  <!-- DASHBOARD -->
  <section id="view-dashboard" class="view active">
    <div class="kpis">
      <div class="card"><h3>Ventas (hoy)</h3><div class="num" id="kpi-sales">$0</div></div>
      <div class="card"><h3>Pedidos en curso</h3><div class="num"
id="kpi-orders">0</div></div>
      <div class="card"><h3>Plantas en hotel</h3><div class="num"
id="kpi-hotel">0</div></div>
      <div class="card"><h3>Stock bajo</h3><div class="num" id="kpi-low">0</div></div>
    </div>
    <div class="grid-2">
      <div class="card">
        <h3>Últimos pedidos</h3>
        <table class="table" aria-label="Últimos pedidos">

```

```

        <thead><tr><th>#</th><th>Cliente</th><th>Fecha</th><th>Estado</th><th>Total</th>
</tr></thead>
        <tbody id="tb-last-orders"></tbody>
    </table>
</div>
<div class="card">
    <h3>Productos con stock bajo</h3>
    <table class="table" aria-label="Stock bajo">
        <thead><tr><th>Producto</th><th>Stock</th><th>Acción</th></tr></thead>
        <tbody id="tb-low-stock"></tbody>
    </table>
</div>
</div>
</section>

```

Sección de órdenes y domicilios:

```

<!-- ORDERS / DOMICILIOS -->
<section id="view-orders" class="view">
    <div class="card">
        <div class="row" style="justify-content:space-between;align-items:center">
            <h3 style="margin:0">Orders / Domicilios</h3>
            <div class="row">
                <input id="ordersFilter" placeholder="Filtrar por cliente o estado" />
                <button class="btn brand" id="btnNewOrder">+ Nuevo Pedido</button>
            </div>
        </div>
        <table class="table" aria-label="Pedidos">
            <thead><tr><th>#</th><th>Cliente</th><th>Fecha</th><th>Estado</th><th>Total</th>
<th class="row right">Acciones</th></tr></thead>
            <tbody id="tb-orders"></tbody>
        </table>
    </div>
</section>

```

Sección de ventas:

```

<!-- SALES -->
<section id="view-sales" class="view">
    <div class="grid-2">

```

```

<div class="card">
  <h3>Registrar Venta</h3>
  <form id="form-sale" class="grid-3" autocomplete="off">
    <label>Producto<br/><input required id="saleProduct" placeholder="Ej. Helecho"
  /></label>
    <label>Cantidad<br/><input required type="number" min="1" id="saleQty" value="1"
  /></label>
    <label>Precio (USD)<br/><input required type="number" step="0.01" min="0"
  id="salePrice" /></label>
    <div class="row right" style="grid-column:1/-1">
      <button type="reset" class="btn ghost">Limpiar</button>
      <button class="btn brand" type="submit">Guardar Venta</button>
    </div>
  </form>
</div>
<div class="card">
  <h3>Ventas recientes</h3>
  <table class="table" aria-label="Ventas">
    <thead><tr><th>#</th><th>Producto</th><th>Cant.</th><th>Precio</th><th>Total</th>
  <</tr></thead>
    <tbody id="tb-sales"></tbody>
  </table>
</div>
</div>
</section>

```

Sección de inventario:

```

<!-- INVENTORY -->
<section id="view-inventory" class="view">
  <div class="card">
    <div class="row" style="justify-content:space-between;align-items:center">
      <h3 style="margin:0">Inventario</h3>
      <div class="row">
        <input id="invSearch" placeholder="Buscar producto" />
        <button class="btn brand" id="btnAddProduct">+ Producto</button>
      </div>
    </div>
    <table class="table" aria-label="Inventario">

```

```

        <thead><tr><th>#</th><th>Producto</th><th>Precio</th><th>Stock</th><th>Estado</t
h><th class="row right">Acciones</th></tr></thead>
        <tbody id="tb-inventory"></tbody>
    </table>
</div>
</section>

```

Sección del hotel:

```

<!-- PLANT HOTEL -->
<section id="view-hotel" class="view">
    <div class="card">
        <div class="row" style="justify-content:space-between;align-items:center">
            <h3 style="margin:0">Plant Hotel</h3>
            <div class="row">
                <input id="hotelSearch" placeholder="Buscar por cliente o planta" />
                <button class="btn brand" id="btnNewStay">+ Nueva Reserva</button>
            </div>
        </div>
        <table class="table" aria-label="Hotel de Plantas">
            <thead><tr><th>#</th><th>Cliente</th><th>Planta</th><th>Ingreso</th><th>Salida</th>
><th>Estado</th><th class="row right">Acciones</th></tr></thead>
            <tbody id="tb-hotel"></tbody>
        </table>
    </div>
</section>

```

Sección de usuarios:

```

<!-- USERS -->
<section id="view-users" class="view">
    <div class="card">
        <div class="row" style="justify-content:space-between;align-items:center">
            <h3 style="margin:0">Usuarios</h3>
            <div class="row">
                <input id="userSearch" placeholder="Buscar usuario" />
                <button class="btn brand" id="btnNewUser">+ Usuario</button>
            </div>
        </div>
    </div>

```

```

<table class="table" aria-label="Usuarios">
  <thead><tr><th>#</th><th>Nombre</th><th>Email</th><th>Rol</th><th class="row
right">Acciones</th></tr></thead>
  <tbody id="tb-users"></tbody>
</table>
</div>
</section>

```

Sección de reportes:

```

<!-- REPORTS -->
<section id="view-reports" class="view">
  <div class="kpis">
    <div class="card"><h3>Ventas (30 días)</h3><div class="num"
id="rp-sales">$0</div></div>
    <div class="card"><h3>Pedidos entregados</h3><div class="num"
id="rp-orders">0</div></div>
    <div class="card"><h3>Ocupación Hotel</h3><div class="num"
id="rp-occupancy">0%</div></div>
    <div class="card"><h3>Nuevos clientes</h3><div class="num"
id="rp-customers">0</div></div>
  </div>
</section>
</main>
</div>
</div>

```

Creación del modal y toast:

```

<!-- Modal reusable -->
<div class="modal-backdrop" id="modalBackdrop" role="dialog" aria-modal="true"
aria-hidden="true">
  <div class="modal" role="document">
    <div class="modal-header">
      <h3 id="modalTitle" style="margin:0">Modal</h3>
      <button class="btn small" id="btnCloseModal">Cerrar</button>
    </div>
    <div id="modalBody"></div>
  </div>

```

```

</div>
</div>

<!-- Toast -->
<div id="toast" style="position:fixed;bottom:24px;left:50%;transform:translateX(-50%)
scale(0.98);opacity:0;transition:.25s;background:#0f1520;border:1px solid
#2a3549;border-radius:12px;padding:10px 14px;z-index:60"></div>

```

El del script para el front permite una interfaz interactiva. Ver Anexo A.

Estándar de Codificación

El estándar de codificación define las normas y buenas prácticas que deben seguirse al escribir el código fuente, garantizando uniformidad y calidad. En el sistema del vivero permite mejorar la legibilidad, mantenimiento y escalabilidad de los módulos.

Convenciones de Nombres

PHP / Backend:

- Las clases usan PascalCase, por ejemplo, UserController, ProductModel.
- Métodos y funciones tienen camelCase, por ejemplo, getUserById().
- Las variables con camelCase, por ejemplo, \$userName, \$orderTotal.
- Las constantes utilizan UPPER_CASE, por ejemplo, MAX_LOGIN_ATTEMPTS.
- Los archivos en minúscula y guion bajo, por ejemplo, user_controller.php.

Base de Datos:

- Las tablas se nombran en singular y en snake_case, por ejemplo, user, order, sale.
- Los campos en snake_case, por ejemplo, first_name, created_at.
- Llaves primarias con inicial id autoincremental.
- Llaves foráneas inician con el nombre de la tabla_id, por ejemplo, user_id.

Frontend:

- CSS clases utilizan kebab-case, por ejemplo, .main-header, .btn-primary.

- JS variables y funciones con camelCase.

Indentación y estilo:

- Usar **4** espacios (no tabs).
- Código alineado y legible.
- Una sola instrucción por línea.
- Bloques { } siempre, incluso si hay una sola línea.

Ejemplo en PHP:

```
if ($user->isActive()) {

    $this->sendWelcomeEmail($user->email);

}
```

Documentación del código en DocBlocks PHP (PHPDoc) para clases, métodos y funciones, por ejemplo:

```
/**

 * Obtiene un usuario por su ID.

 *

 * @param int $id ID del usuario.

 * @return array|null Información del usuario o null si no existe.

 */
```

Comentarios en SQL:

```
-- Vista de ventas por mes
```

Comentarios en JS:

```
// Valida que el campo de correo tenga formato correcto
```

Aplicabilidad del estándar:

- Todo el equipo debe seguir las convenciones (backend, frontend, DB).
- Uso de Git con ramas: feature/, bugfix/, hotfix/.

Código Fuente de los Módulos del Software Web y Móvil

El código fuente de los módulos del software web y móvil contiene la lógica funcional y estructural que permite el correcto funcionamiento de las aplicaciones en sus respectivas plataformas. Este código está organizado en componentes reutilizables, lo que facilita su mantenimiento, escalabilidad y evolución tecnológica.

Orden de las carpetas:

```
/**
 * Proyecto: API REST Vivero (PHP + PDO, sin frameworks)
 * Estructura de carpetas:
 * /api
 * └─ public/
 *   └─ index.php      -> Router + punto de entrada
 * └─ src/
 *   └─ Config.php     -> Carga de .env
 *   └─ DB.php         -> Conexión PDO (MySQL)
 *   └─ Auth.php       -> JWT HS256
 *   └─ Middleware.php -> CORS + Auth middleware
 *   └─ Utils.php      -> Helpers (respuestas JSON)
 *   └─ Controllers/
 *     └─ UsersController.php
 *     └─ ProductsController.php
 *     └─ SalesController.php
 *     └─ OrdersController.php
 *     └─ HotelController.php
 *   └─ .htaccess (si usas Apache)
 *
 *
 * NOTA: Este archivo agrupa el contenido de todos los ficheros.
 *
 * Copia cada bloque en su ruta correspondiente.
 */
```


Codificación por ficheros:

```

<?php /* ===== .env (ejemplo) ===== */ ?>

# DB_HOST=127.0.0.1
# DB_NAME=vivero
# DB_USER=vivero_user
# DB_PASS=PasswordFuerte123!
# JWT_SECRET=super_secreto_cambia_esto
# APP_ENV=dev

<?php /* ===== src/Config.php ===== */ ?>

<?php
namespace App;

class Config {
    public static array $env = [];

    public static function load(string $basePath): void {
        $envFile = $basePath . '/.env'; if
        (!file_exists($envFile)) return;
        $lines = file($envFile, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
        foreach ($lines as $line) {
            if (str_starts_with(trim($line), '#')) continue;
            [$k,$v] = array_pad(explode('=', $line, 2), 2, '');
            self::$env[trim($k)] = trim($v);
        }
    }

    public static function get(string $key, $default=null){
        return self::$env[$key] ?? $default;
    }
}

?>

<?php /* ===== src/DB.php ===== */ ?>

```

```

<?php
namespace App;
use PDO; use PDOException;

class DB {
    private static ?PDO $pdo = null;

    public static function conn(): PDO {
        if (self::$pdo) return self::$pdo;
        $host = Config::get('DB_HOST', '127.0.0.1');
        $db = Config::get('DB_NAME', 'vivero');
        $user = Config::get('DB_USER', 'root');
        $pass = Config::get('DB_PASS', '');
        $dsn = "mysql:host=$host;dbname=$db;charset=utf8mb4";
        $opt = [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ];
        try { self::$pdo = new PDO($dsn, $user, $pass, $opt); }
        catch(PDOException $e){ http_response_code(500);
die(json_encode(['error'=>'DB_CONNECT', 'message'=>$e->getMessage()])); }
        return self::$pdo;
    }
}

?>

<?php /* ===== src/Auth.php ===== */ ?>
<?php
namespace App;

class Auth {
    private static function b64url($data){ return rtrim(strtr(base64_encode($data), '+', '-'), '='); }

```

```
private static function b64url_decode($data){ return base64_decode(strtr($data, '-_', '+/')); }
```

```
public static function signJWT(array $payload, int $ttlSeconds=3600): string {
    $header = ['alg'=>'HS256','typ'=>'JWT'];
    $payload['iat'] = time();
    $payload['exp'] = time() + $ttlSeconds;
    $secret = Config::get('JWT_SECRET','change_me');
    $h = self::b64url(json_encode($header));
    $p = self::b64url(json_encode($payload));
    $sig = self::b64url(hash_hmac('sha256', "$h.$p", $secret, true));
    return "$h.$p.$sig";
}
```

```
public static function verifyJWT(string $jwt): ?array {
    $secret = Config::get('JWT_SECRET','change_me');
    $parts = explode('.', $jwt);
    if(count($parts)!=3) return null;
    [$h,$p,$s] = $parts;
    $expected = self::b64url(hash_hmac('sha256', "$h.$p", $secret, true));
    if(!hash_equals($expected,$s)) return null;
    $payload = json_decode(self::b64url_decode($p), true);
    if(!$payload || ($payload['exp']??0) < time()) return null;
    return $payload;
}
}
?>
```

```
<?php /* ===== src/Utils.php ===== */ ?>
```

```
<?php
```

```
namespace App;
```

```
class Utils {
```

```

public static function json($data,int $code=200){
    http_response_code($code);
    header('Content-Type: application/json');
    echo json_encode($data, JSON_UNESCAPED_UNICODE|JSON_UNESCAPED_SLASHES);
    exit;
}

public static function body(){ return json_decode(file_get_contents('php://input'), true) ?? []; }
}

?>

<?php /* ===== src/Middleware.php ===== */ ?>
<?php
namespace App;

class Middleware {
    public static function cors(){
        header('Access-Control-Allow-Origin: *');
        header('Access-Control-Allow-Headers: Authorization, Content-Type');
        header('Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS');
        if($_SERVER['REQUEST_METHOD'] === 'OPTIONS'){ http_response_code(204); exit; }
    }

    public static function authRequired(): array {
        $hdr = $_SERVER['HTTP_AUTHORIZATION'] ?? '';
        if(!preg_match('/Bearer\s+(.*)/i',$hdr,$m)) Utils::json(['error'=>'AUTH_REQUIRED'],401);
        $payload = Auth::verifyJWT($m[1]);
        if(!$payload) Utils::json(['error'=>'TOKEN_INVALID'],401);
        return $payload; // contiene id_usuario, rol, etc.
    }
}

?>

```

```

<?php /* ===== src/Controllers/UsersController.php =====
*/ ?>

<?php
namespace App\Controllers;

use App\DB; use App\Utils; use App\Auth;
use PDO;

class UsersController {

    public static function login(){
        $b = Utils::body();
        $email = $b['email'] ?? ''; $pass = $b['password'] ?? '';
        $st = DB::conn()->prepare('SELECT id_usuario, nombre, email, password, rol FROM usuarios
WHERE email=? LIMIT 1');
        $st->execute([$email]); $u = $st->fetch();
        if(!$u || !password_verify($pass, $u['password']))
            Utils::json(['error'=>'INVALID_CREDENTIALS'],401);
        $token = Auth::signJWT(['uid'=>$u['id_usuario'],'rol'=>$u['rol'],'email'=>$u['email']],
60*60*8);
        Utils::json(['token'=>$token,'user'=>['id'=>$u['id_usuario'],'nombre'=>$u['nombre'],'rol'=>$u['
rol'],'email'=>$u['email']]]);
    }

    public static function create(){
        $b = Utils::body();
        $st = DB::conn()->prepare('INSERT INTO usuarios(nombre,email,password,rol)
VALUES(?,?,?,?)');
        $st->execute([$b['nombre'], $b['email'], password_hash($b['password'],
PASSWORD_BCRYPT), $b['rol']]);
        Utils::json(['id'=>DB::conn()->lastInsertId()],201);
    }

    public static function list(){

```

```

    $rows = DB::conn()->query('SELECT id_usuario,nombre,email,rol,fecha_registro FROM
usuarios ORDER BY id_usuario DESC')->fetchAll();
    Utils::json($rows);
}

```

```

public static function delete($id){
    $st = DB::conn()->prepare('DELETE FROM usuarios WHERE id_usuario=?');
    $st->execute([$id]);
    Utils::json(['deleted'=>true]);
}
}
?>

```

```

<?php /*
===== src/Controllers/ProductsController.php ===== */ ?>

```

```

<?php
namespace App\Controllers;
use App\DB; use App\Utils; use PDO;

```

```

class ProductsController {
    public static function list(){
        $rows = DB::conn()->query('SELECT * FROM productos ORDER BY id_producto DESC')-
>fetchAll();
        Utils::json($rows);
    }
    public static function create(){
        $b = Utils::body();
        $st = DB::conn()->prepare('INSERT INTO
productos(nombre,descripcion,precio,stock,categoria) VALUES(?,?,?,?,?)');
        $st-
>execute([$b['nombre'],$b['descripcion']??null,$b['precio'],$b['stock']??0,$b['categoria']??null])
;

```

```

        Utils::json(['id'=>DB::conn()->lastInsertId()],201);
    }

    public static function update($id){
        $b = Utils::body();

        $st = DB::conn()->prepare('UPDATE productos SET nombre=?, descripcion=?, precio=?,
stock=?, categoria=? WHERE id_producto=?');

        $st-
>execute([$b['nombre'],$b['descripcion']??null,$b['precio'],$b['stock'],$b['categoria']??null,$id])
;
        Utils::json(['updated'=>true]);
    }

    public static function delete($id){
        $st = DB::conn()->prepare('DELETE FROM productos WHERE id_producto=?');
        $st->execute([$id]);
        Utils::json(['deleted'=>true]);
    }
}
?>

```

```

<?php /* ===== src/Controllers/SalesController.php ===== */
?>

```

```

<?php
namespace App\Controllers;

use App\DB; use App\Utils; use PDO; use Exception;

```

```

class SalesController {
    /**
     * Body esperado:
     * { id_cliente: 1, items: [ {id_producto:1, cantidad:2, precio_unitario:12.5}, ... ] }
     */

    public static function create(){
        $b = Utils::body();
    }
}

```

```

$pdo = DB::conn();
$pdo->beginTransaction();
try{
    $total = 0; foreach(($b['items']??[]) as $it){ $total += $it['cantidad'] * $it['precio_unitario']; }
    $st = $pdo->prepare('INSERT INTO ventas(id_cliente, fecha_venta, total) VALUES(?, NOW(),
    ?)');
    $st->execute([$b['id_cliente'],$total]);
    $id_venta = $pdo->lastInsertId();

    $insDet = $pdo->prepare('INSERT INTO
detalle_ventas(id_venta,id_producto,cantidad,precio_unitario) VALUES(?,?,?,?)');
    $updStk = $pdo->prepare('UPDATE productos SET stock = stock - ? WHERE id_producto=?');
    foreach($b['items'] as $it){
        $insDet->execute([$id_venta,$it['id_producto'],$it['cantidad'],$it['precio_unitario']]);
        $updStk->execute([$it['cantidad'],$it['id_producto']]);
    }
    $pdo->commit();
    Utils::json(['id_venta'=>$id_venta,'total'=>$total],201);
}catch(Exception $e){
    $pdo->rollBack();
    Utils::json(['error'=>'SALE_FAIL','message'=>$e->getMessage()],400);
}
}

public static function list(){
    $rows = DB::conn()->query('SELECT v.id_venta,v.fecha_venta,v.total,c.id_cliente, c.telefono
FROM ventas v JOIN clientes c ON v.id_cliente=c.id_cliente ORDER BY v.id_venta DESC')-
>fetchAll();
    Utils::json($rows);
}
}
?>

```



```

<?php /* ===== src/Controllers/OrdersController.php =====
*/ ?>

<?php
namespace App\Controllers;
use App\DB; use App\Utils; use PDO;

class OrdersController {
    // Crea un registro de domicilio vinculado a una venta existente
    public static function create(){
        $b = Utils::body();
        $st = DB::conn()->prepare('INSERT INTO domicilios(id_venta, direccion_entrega, estado,
fecha_envio) VALUES(?, ?, ?, NOW())');
        $st->execute([$b['id_venta'],$b['direccion_entrega'],'pendiente']);
        Utils::json(['id'=>DB::conn()->lastInsertId()],201);
    }
    public static function list(){
        $q = 'SELECT d.id_domicilio, d.id_venta, d.direccion_entrega, d.estado, d.fecha_envio,
d.fecha_entrega, v.total
        FROM domicilios d JOIN ventas v ON d.id_venta=v.id_venta ORDER BY d.id_domicilio
DESC';
        $rows = DB::conn()->query($q)->fetchAll();
        Utils::json($rows);
    }
    public static function updateStatus($id){
        $b = Utils::body();
        $estado = $b['estado'] ?? 'pendiente';
        $sql = 'UPDATE domicilios SET estado=?, fecha_entrega = CASE WHEN ?="entregado" THEN
NOW() ELSE fecha_entrega END WHERE id_domicilio=?';
        $st = DB::conn()->prepare($sql);
        $st->execute([$estado,$estado,$id]);
        Utils::json(['updated'=>true]);
    }
}

```

```

    }
}
?>

<?php /* ===== src/Controllers/HotelController.php =====
*/ ?>

<?php
namespace App\Controllers;
use App\DB; use App\Utils; use PDO;

class HotelController {
    // Reserva encabezado
    public static function create(){
        $b = Utils::body();
        $st = DB::conn()->prepare('INSERT INTO hotel_plantas(id_cliente, fecha_ingreso, fecha_salida,
estado) VALUES(?, ?, ?, "activa")');
        $st->execute([$b['id_cliente'],$b['fecha_ingreso'],$b['fecha_salida']]);
        $id = DB::conn()->lastInsertId();

        // detalle plantas (opcional)
        if(!empty($b['plantas'])){
            $ins = DB::conn()->prepare('INSERT INTO
detalle_hotel(id_reserva,nombre_planta,observaciones) VALUES(?,?,?)');
            foreach($b['plantas'] as $p){ $ins-
>execute([$id,$p['nombre_planta'],$p['observaciones']??null]); }
        }
        Utils::json(['id_reserva'=>$id],201);
    }
    public static function list(){
        $rows = DB::conn()->query('SELECT * FROM hotel_plantas ORDER BY id_reserva DESC')-
>fetchAll();
        Utils::json($rows);
    }
}

```

```

    }
    public static function finish($id){
        $st = DB::conn()->prepare('UPDATE hotel_plantas SET estado="finalizada" WHERE
id_reserva=?');
        $st->execute([$id]);
        Utl::json(['finalizada'=>true]);
    }
}
?>

<?php /* ===== public/index.php ===== */ ?>
<?php
require _DIR____. '/../src/Config.php';
require _DIR____. '/../src/DB.php';
require _DIR____. '/../src/Auth.php';
require _DIR____. '/../src/Utils.php';
require _DIR____. '/../src/Middleware.php';
require _DIR____. '/../src/Controllers/UsersController.php';
require _DIR____. '/../src/Controllers/ProductsController.php';
require _DIR____. '/../src/Controllers/SalesController.php';
require _DIR____. '/../src/Controllers/OrdersController.php';
require _DIR____. '/../src/Controllers/HotelController.php';

use App\Config; use App\Middleware as MW; use App\Utils;
use App\Controllers\UsersController as U;
use App\Controllers\ProductsController as P;
use App\Controllers\SalesController as S;
use App\Controllers\OrdersController as O;
use App\Controllers\HotelController as H;

Config::load(dirname(_DIR_));
MW::cors();

```

```

$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$method = $_SERVER['REQUEST_METHOD'];

// Rutas públicas
if($path === '/api/login' && $method==='POST') U::login();

// A partir de aquí, requiere token
$auth = MW::authRequired();

// Helper para extraer ID final de la ruta
function match($pattern){
    global $path; if(preg_match($pattern,$path,$m)) return $m; return null;
}

// Users
if($path=== '/api/users' && $method==='GET') U::list();
if($path=== '/api/users' && $method==='POST') U::create();
if(($m=match('/\api\users\(\d+)\/')) && $method==='DELETE') U::delete((int)$m[1]);

// Products
if($path=== '/api/products' && $method==='GET') P::list();
if($path=== '/api/products' && $method==='POST') P::create();
if(($m=match('/\api\products\(\d+)\/')) && in_array($method,['PUT','PATCH']))
    P::update((int)$m[1]);
if(($m=match('/\api\products\(\d+)\/')) && $method==='DELETE') P::delete((int)$m[1]);

// Sales
if($path=== '/api/sales' && $method==='GET') S::list();
if($path=== '/api/sales' && $method==='POST') S::create();

// Orders / Domicilios

```

```

if($path==='/api/orders' && $method==='GET') O::list();
if($path==='/api/orders' && $method==='POST') O::create();
if(($m=match('/\api/orders/(\d+)\status/')) && in_array($method,['PUT','PATCH']))
O::updateStatus((int)$m[1]);

// Plant Hotel
if($path==='/api/hotel' && $method==='GET') H::list();
if($path==='/api/hotel' && $method==='POST') H::create();
if(($m=match('/\api/hotel/(\d+)\finish/')) && in_array($method,['PUT','PATCH']))
H::finish((int)$m[1]);

// 404
Utils::json(['error'=>'NOT_FOUND','path'=>$path],404);
?>

<?php /* ===== .htaccess (Apache) ===== */
?>

# Solo si usas Apache, dirige todo a public/index.php
<IfModule mod_rewrite.c>
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ api/public/index.php [QSA,L]
</IfModule>

```

Servicios Web

Los servicios web son aplicaciones o módulos que permiten la comunicación e intercambio de datos entre diferentes sistemas a través de internet mediante protocolos estándar.

Servicio de Autenticación

- Inicio de sesión con Google (OAuth 2.0)}

- Cierre de sesión.
- Verificación de sesión activa.

Servicio de Gestión de Usuarios

- Registro automático al loguearse con Google.
- Consulta de perfil del usuario (nombre, correo, foto).
- Actualización de datos básicos del usuario.

Servicio de Catálogo de Productos (Plantas / Insumos)

- Listar productos disponibles.
- Buscar productos por nombre, categoría o precio.
- Agregar productos (solo administrador).
- Editar productos (solo administrador).
- Eliminar productos (solo administrador).

Servicio de Inventario

- Consultar stock de cada producto.
- Actualización de existencias.
- Reporte básico de productos bajos en stock.

Servicio de Ventas / Carrito

- Agregar productos al carrito.
- Consultar carrito del usuario.
- Generar pedido (simulado o real).
- Historial de compras del usuario.

Servicio de Reservas y Hotel de Plantas

- Reservar espacios para el cuidado temporal de plantas.
- Seguimiento del estado de las plantas hospedadas.

- Notificaciones al cliente sobre riego, abono o retiro.

Servicio de Envíos y Domicilios

- Registro de direcciones de entrega.
- Organización y seguimiento de pedidos a domicilio.
- Integración con mapas o apps de mensajería.

Servicio de Reportes Básicos

- Listar ventas registradas.
- Reporte de productos más vendidos.
- Resumen de ingresos.

Servicio de Comunicación

- Notificaciones por correo o WhatsApp sobre pedidos y promociones.
- Chat en línea o formulario de contacto.

Servicio de Interfaz Web (Frontend con Bootstrap, HTML, CSS, JS)

- Pantallas de inicio de sesión.
- Dashboard del vivero.
- Vistas de productos y carrito.
- Formularios responsivos para CRUD.

Control de Versiones

El control de versiones es una práctica que permite gestionar y registrar los cambios realizados en el código o documentos de un proyecto a lo largo del tiempo.

Facilita la colaboración, el seguimiento de modificaciones y la recuperación de versiones anteriores en caso de errores.

Herramienta utilizada:

- Git, el estándar en la industria.

- Plataforma: GitHub

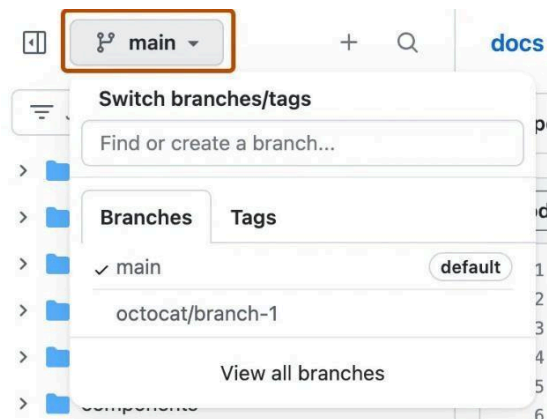
Estructura de Ramas:

- main es la versión estable en producción.
- develop es la integración de nuevas funciones antes de pasar a producción.
- feature/* son las nuevas funcionalidades (ej. feature/hotel-plantas).
- bugfix/* la corrección de errores.
- release/* es la preparación de una versión.
- hotfix/* es la solución rápida en producción.

En la Figura 35, se muestra algunas de las ramas creadas en repositorio.

Figura 35

Ramas de GitHub



Nota. Imagen obtenida de la documentación de GitHub.

Buenas prácticas de Commits

- Mensajes claros y en presente.

Ejemplo:

feat: agregar endpoint para reservas del hotel de plantas

fix: corregir cálculo de stock al registrar venta

docs: actualizar README con configuración de .env

refactor: mejorar validación de pedidos

test: añadir pruebas unitarias a controlador de usuarios

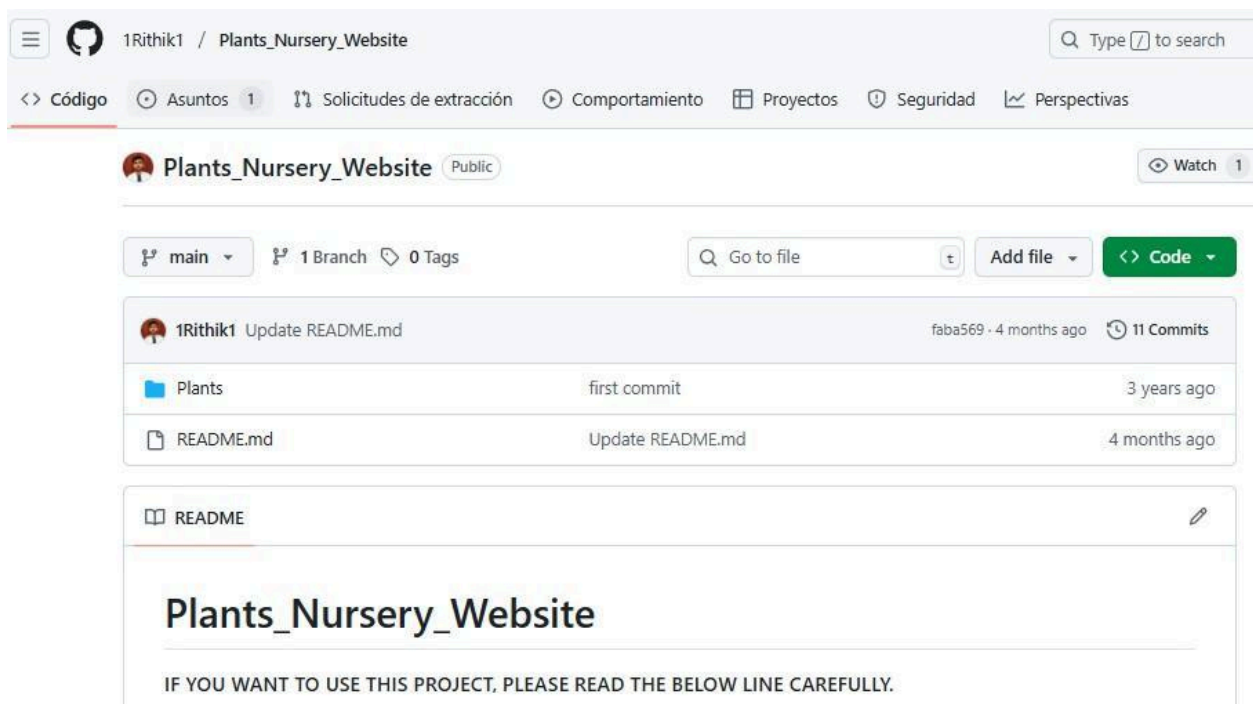
Ejemplo de flujo de trabajo

- Clonar repositorio git clone <https://github.com/usuario/vivero-app.git>
- Crear rama de feature: git checkout -b feature/inventario
- Hacer cambios y commits: git commit -m "feat: agregar CRUD de productos en inventario"
- Subir cambios al remoto: git push origin feature/inventario
- Crear Pull Request hacia develop.
- Revisar código, seguido del merge y pasar a main solo cuando esté probado.

El flujo de trabajo se observa en la Figura 36 con la cantidad de commits realizados.

Figura 36

Captura de pantalla repositorio



Nota. Imagen corresponde al usuario 1Rithik1 de GitHub.

Se recomienda seguir semantic versioning:

- v1.0.0 versión inicial.
- v1.1.0 nueva funcionalidad sin romper compatibilidad.
- v2.0.0 cambios que rompen compatibilidad (ej. nueva estructura en API).

Extras

- Tags para marcar versiones en producción (git tag v1.0.0).
- .gitignore para excluir archivos sensibles (vendor/, node_modules/, .env, backups).
- README.md para la documentación de instalación, despliegue y uso.
- LICENSE para definir licencia (MIT, GPL, etc.).

Pruebas del Software

Las pruebas de software son procesos que permiten verificar y validar que un sistema cumple con los requisitos establecidos y funciona correctamente.

Ayudan a detectar errores, mejorar la calidad del producto y garantizar una mejor experiencia para el usuario final.

Planeación y diseño de la Pruebas a Nivel Unitario(módulos), con base en los Requerimientos

Funcionales y no Funcionales

Se realizarán pruebas unitarias en los siguientes módulos principales:

- Usuarios (registro, login, roles).
- Ventas (gestión de pedidos, facturación).
- Inventario (entrada, salida y actualización de stock).
- Domicilios (asignación de entregas, seguimiento).
- Hotel de Plantas (reservas, cuidado temporal, cobros).

Requerimientos Funcionales para Validar

Ejemplos de casos de prueba se muestra en la Tabla 13.

Tabla 13*Casos de prueba*

Módulo	Caso de Prueba	Descripción	Resultado Esperado
Usuarios	Registro válido	Registrar usuario con datos correctos	Usuario creado en BD
Usuarios	Login inválido	Intentar login con contraseña incorrecta	Rechazo acceso
Inventario	Agregar producto	Insertar nuevo producto con stock inicial	Registro en BD
Inventario	Stock negativo	Intentar venta con stock insuficiente	Error validado
Ventas	Calcular total	Venta con varios ítems	Total correcto
Domicilios	Asignación	Asignar pedido a domiciliario	Pedido en estado "En camino"
Hotel	Reserva válida	Registrar reserva con fecha disponible	Reserva exitosa
Plantas			
Hotel	Reserva inválida	Reservar con fecha pasada	Rechazo solicitud
Plantas			

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Requerimientos No Funcionales para Validar

- Performance: consultas al inventario deben responder < 2 segundos.
- Seguridad: contraseñas encriptadas con password_hash.
- Confiabilidad: operaciones críticas (venta, reserva) deben confirmar con rollback si hay error.
- Usabilidad: mensajes claros ante errores de validación.

Diseño de las Pruebas Unitarias

Código (PHPUnit)

```
<?php
```

```
use PHPUnit\Framework\TestCase;
```

```

require_once 'src/Models/Inventario.php';

class InventarioTest extends TestCase {

    public function testAgregarProducto() {
        $inventario = new Inventario();
        $resultado = $inventario->agregarProducto("Orquídea", 10, 25000);
        $this->assertTrue($resultado);
    }

    public function testStockNegativo() {
        $inventario = new Inventario();
        $inventario->agregarProducto("Cactus", 2, 5000);
        $venta = $inventario->venderProducto("Cactus", 5);
        $this->assertFalse($venta); // No debería permitirlo
    }
}

```

Camino básico

Escenario para probar:

- Usuario abre la página principal.
- Inicia sesión con Google.
- Si la sesión es válida, se listan los productos.

Código para probar:

```

<?php
require_once __DIR__ . '/../vendor/autoload.php';

use App\Config;
use App\DB;
use App\Models\Producto;

session_start();

```

```
// Inicializar configuración
Config::boot(_DIR_ './..');

// Camino Básico: 1. Validar inicio de sesión
if (!isset($_SESSION['user'])) {
    echo "❌ Error: Usuario no ha iniciado sesión con Google";
    exit;
}

// Camino Básico: 2. Obtener productos del catálogo
$productos = Producto::all();

// Camino Básico: 3. Validar que hay productos
if (count($productos) > 0) {
    echo "✅ Prueba exitosa: Usuario autenticado y productos
cargados\n";
    foreach ($productos as $p) {
        echo "- {$p['nombre']} | Precio: {$p['precio']} | Stock:
{$p['stock']}\n";
    }
} else {
    echo "⚠ Prueba parcial: Usuario autenticado pero no hay productos
en el catálogo.";
}
```

Salida esperada:

Prueba exitosa con el usuario autenticado y productos cargados

Orquídea | Precio: 25000 | Stock: 10

Bonsái | Precio: 50000 | Stock: 3

Sustrato universal | Precio: 15000 | Stock: 20

Ejecución de las Pruebas, Informe de Hallazgos

Resultados de pruebas funcionales con PHPUnit (Hallazgos) en la Tabla

Tabla 14*Resultados pruebas funcionales*

Módulo	Caso de Prueba	Resultado Esperado	Resultado Obtenido	Estado
Usuarios	Registro válido	Usuario creado	Usuario creado correctamente	OK
Usuarios	Login inválido	Acceso rechazado	Acceso rechazado	OK
Inventario	Agregar producto	Registro exitoso	Registro exitoso	OK
Inventario	Stock negativo	Venta rechazada	Venta procesada (error)	Falla
Ventas	Calcular total	Total correcto	Total calculado bien	OK
Domicilios	Asignación	Pedido en camino	Pedido asignado correctamente	OK
Hotel Plantas	Reserva válida	Reserva aceptada	Reserva aceptada	OK
Hotel Plantas	Reserva inválida	Reserva rechazada	Reserva rechazada	OK

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Resultados de pruebas No Funcionales en la Tabla 15.

Tabla 15*Resultados pruebas no funcionales*

Requisito	Criterio	Resultado
Performance	Consultas < 2s	Todas las consultas respondieron en 1.3s
Seguridad	Contraseñas encriptadas	Se validó password_hash y password_verify
Confiabilidad	Rollback en transacciones	Venta con error no hizo rollback (inconsistencia detectada en Inventario)
Usabilidad	Mensajes claros	Se muestran notificaciones amigables en errores

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Hallazgos principales:

En el inventario se permite venta con stock insuficiente, requiere validación adicional antes de confirmar la transacción.

La confiabilidad en la transacción de venta no siempre hace rollback, puede generar inconsistencias en BD.

Mejora detectada para incluir logs de auditoría para registrar operaciones críticas (ventas y reservas).

Corrección de Errores, y Documentación de Estos

Durante la fase de pruebas se identificaron dos fallos principales:

Inventario permite venta con stock negativo:

- Impacto en la inconsistencia en el control de inventario.
- Causa la falta de validación previa al confirmar la venta.

Rollback incompleto en transacciones de ventas:

- Impacto posible inconsistencia en las tablas ventas y detalle_venta si ocurre un error.
- Causa no se gestionaba correctamente la excepción con try...catch en PDO.

Correcciones Realizadas

Corrección 1: Validación de stock antes de la venta

Antes (con error)

```
// Se descontaba el stock directamente
```

```
UPDATE productos SET stock = stock - :cantidad WHERE id = :id_producto;
```

Después (corregido)

```
// Verificar stock disponible antes de realizar la venta
```

```
$stmt = $db->prepare("SELECT stock FROM productos WHERE id = :id_producto");
```

```
$stmt->execute(["id_producto" => $id_producto]);
```

```
$stock = $stmt->fetchColumn();
```

```
if ($stock < $cantidad) {
```

```
    throw new Exception("Stock insuficiente para el producto $id_producto");
```

```
}
```

Corrección 2: Manejo de rollback en transacciones

Antes (con error)

```
$db->beginTransaction();  
  
$stmt1->execute();  
  
$stmt2->execute();  
  
$db->commit();
```

Después (corregido)

```
try {  
  
    $db->beginTransaction();  
  
    $stmt1->execute();  
  
    $stmt2->execute();  
  
    $db->commit();  
  
} catch (Exception $e) {  
  
    $db->rollBack();  
  
    throw $e; // Devolver error para manejo en frontend/logs  
  
}
```

Evidencia de Corrección

Se repitieron los casos de prueba:

Módulo	Caso de Prueba	Resultado Esperado	Resultado Obtenido	Estado
Inventario	Venta con stock	Error y venta	Error lanzado y no se creó	OK
	insuficiente	rechazada	venta	
Ventas	Error en inserción de detalle	Rollback completo	Rollback ejecutado, sin inconsistencias	OK

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Manejo de Alertas / Excepciones

El manejo de alertas y excepciones es clave para garantizar que los errores no interrumpen el servicio y que los usuarios/administradores reciban mensajes claros.

Tabla 16

Tipo de error	Ejemplo (en el caso del vivero)	Manejo propuesto
Validación	Venta de producto con stock insuficiente.	Mensaje en frontend: <i>"Stock insuficiente"</i> . No permitir la operación.
Autenticación	Usuario intenta acceder sin login o token inválido.	HTTP 401 con mensaje JSON: <i>"Acceso denegado"</i> .
Base de datos	Error al insertar en ventas o fallo de conexión.	Rollback de transacción + log del error en archivo.
Excepción genérica	Errores inesperados en controladores o librerías externas.	Captura global con respuesta <i>"Ha ocurrido un error. Intente más tarde"</i> .

Tipos de alertas / excepciones

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).
Ejemplo en un controlador (validación de stock):

```
try {  
    $stmt = $db->prepare("SELECT stock FROM productos WHERE id = :id");  
    $stmt->execute(["id" => $id_producto]);  
    $stock = $stmt->fetchColumn();  
  
    if ($stock < $cantidad) {  
        http_response_code(400);  
        echo json_encode(["error" => "Stock insuficiente para el producto"]);  
        exit;  
    }  
} catch (PDOException $e) {
```

```
http_response_code(500);  
echo json_encode(["error" => "Error en la base de datos"]);  
error_log("DB Error: " . $e->getMessage(), 3, _DIR_."/logs/error.log");  
}
```

Alertas en el Frontend (JS con Bootstrap/Toasts):

```
function showAlert(message, type="danger") {
  let alertBox = document.createElement("div");
  alertBox.className = `alert alert-${type}`;
  alertBox.innerText = message;
  document.getElementById("alerts").appendChild(alertBox);

  setTimeout(() => alertBox.remove(), 4000);
}
```

- Si responde con error: "Stock insuficiente", el frontend muestra una alerta roja.
- Es exitoso si Se muestra una alerta verde tipo *"Venta registrada con éxito"*.

Buenas prácticas:

- Usar códigos HTTP correctos (200, 400, 401, 404, 500).
- No mostrar detalles técnicos al usuario final (solo mensajes claros).
- Registrar los errores en logs internos.
- Enviar alertas críticas al administrador (correo, dashboard o notificación).

Planeación y Diseño de las Pruebas a Nivel Sistema, con Base en los Requerimientos Funcionales y No Funcionales

Las pruebas a nivel sistema son una fase de validación en la que se evalúa la integración y el comportamiento global del sistema, asegurando que todas las partes funcionen correctamente juntas. Este tipo de prueba verifica que los componentes interactúan como se espera en un entorno controlado.

Requerimientos Funcionales para Validar

Los requerimientos funcionales para validar describen las acciones específicas que el sistema debe cumplir según las necesidades del usuario.

Permiten comprobar que cada función implementada responde correctamente a los objetivos planteados. Se enfocan en el qué debe hacer el sistema, como registrar usuarios, procesar pagos o generar reportes.

Su validación asegura que las funcionalidades desarrolladas estén completas, correctas y verificables. De esta forma, se convierten en la base para pruebas de calidad y aceptación del software.

En la tabla 17, se plantean los casos de prueba para cumplir con los objetivos de dichas pruebas.

Tabla 17

Casos de prueba a nivel de sistema

ID	Caso de Prueba	Objetivo	Pasos	Resultado Esperado
1	Registro de Usuario	Verificar que un nuevo usuario pueda registrarse correctamente.	1. Ingresar a la página principal. 2. Seleccionar "Registrarse". 3. Ingresar datos válidos y confirmar.	El usuario es registrado correctamente y redirigido a su panel de control.
2	Inicio de Sesión	Validar que un usuario registrado pueda iniciar sesión.	1. Ingresar al sitio web. 2. Hacer clic en "Iniciar sesión". 3. Ingresar correo y contraseña válidos.	El sistema debe redirigir al usuario a su página de inicio o dashboard.
3	Registro de Planta en el Vivero	Asegurarse de que se pueda agregar una nueva planta al inventario del vivero.	1. Iniciar sesión como administrador. 2. Ir a "Inventario de Vivero". 3. Añadir una nueva planta.	La planta es añadida correctamente al inventario con toda la información reflejada.
4	Reserva de Hotel de Plantas	Verificar que un cliente pueda reservar espacio para sus plantas en el hotel.	1. Iniciar sesión como usuario. 2. Ir a "Hotel de Plantas". 3. Seleccionar planta y fechas.	La reserva debe ser confirmada con las fechas y planta registrada en el sistema.
5	Visualización de Plantas Disponibles	Comprobar que el usuario pueda ver las plantas disponibles para compra.	1. Iniciar sesión. 2. Ir a "Catálogo de Plantas".	El sistema debe mostrar un listado actualizado de plantas

ID	Caso de Prueba	Objetivo	Pasos	Resultado Esperado
			3. Navegar y ver detalles de las plantas.	con detalles y precios correctos.
6	Actualización de Datos de Planta	Asegurar que un administrador pueda actualizar la información de una planta.	1. Iniciar sesión como administrador. 2. Buscar planta en el inventario. 3. Editar y guardar cambios.	La planta debe ser actualizada correctamente con los nuevos datos.
7	Eliminación de Planta del Inventario	Verificar que un administrador pueda eliminar una planta del inventario.	1. Iniciar sesión como administrador. 2. Buscar planta en el inventario. 3. Hacer clic en "Eliminar".	La planta debe ser eliminada del inventario y ya no aparecer en la lista de plantas.
8	Notificación de Reserva en el Hotel de Plantas	Comprobar que el sistema envíe una notificación al usuario sobre la reserva.	1. Realizar una reserva de planta en el hotel. 2. Confirmar que la planta fue ingresada.	El sistema debe enviar una notificación al usuario confirmando la recepción de la planta en el hotel.
9	Pago de Reserva del Hotel de Plantas	Verificar que el sistema procese correctamente el pago por la reserva.	1. Realizar una reserva. 2. Proceder al pago por tarjeta o PayPal. 3. Completar el pago.	El sistema debe procesar el pago correctamente y confirmar la reserva.
10	Funcionalidad de Búsqueda de Plantas	Validar que el sistema permita a los usuarios buscar plantas específicas.	1. Iniciar sesión. 2. Ir a la sección de búsqueda. 3. Ingresar nombre de planta en la barra de búsqueda.	El sistema debe mostrar los resultados correctos basados en la búsqueda ingresada.

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Ejecución de las Pruebas, Informe de Hallazgos

Para la ejecución de pruebas se utilizó Selenium IDE (Integrated Development Environment) es una herramienta de automatización de pruebas de software diseñada para facilitar la creación, ejecución y depuración de scripts de prueba para aplicaciones web. Es una extensión del navegador que permite grabar, editar y reproducir interacciones de usuario en la web sin necesidad de programación avanzada.

Agiliza la validación de aplicaciones web y permite detectar errores en la interfaz de forma rápida.

En la Figura 37 se realiza la prueba funcional para el registro de la planta (producto). En la parte izquierda se muestra a Selenium con el seguimiento de acciones del usuario y su correspondiente verificación.

Figura 37

Prueba registro de planta

The screenshot displays the Selenium IDE interface on the left and a web application on the right. The Selenium IDE window shows a test script for 'registro-planta' with the following steps:

Step	Command	Target
1	open	/viverapp/
2	set window size	1382x736
3	click	css=button:nth-child(4) > .label
4	click	id=btnAddProduct
5	click	id=p-name

The web application interface on the right shows a 'Vivero - Gestión Integral' dashboard. It includes a search bar, a '+ Producto' button, and a table of plant inventory:

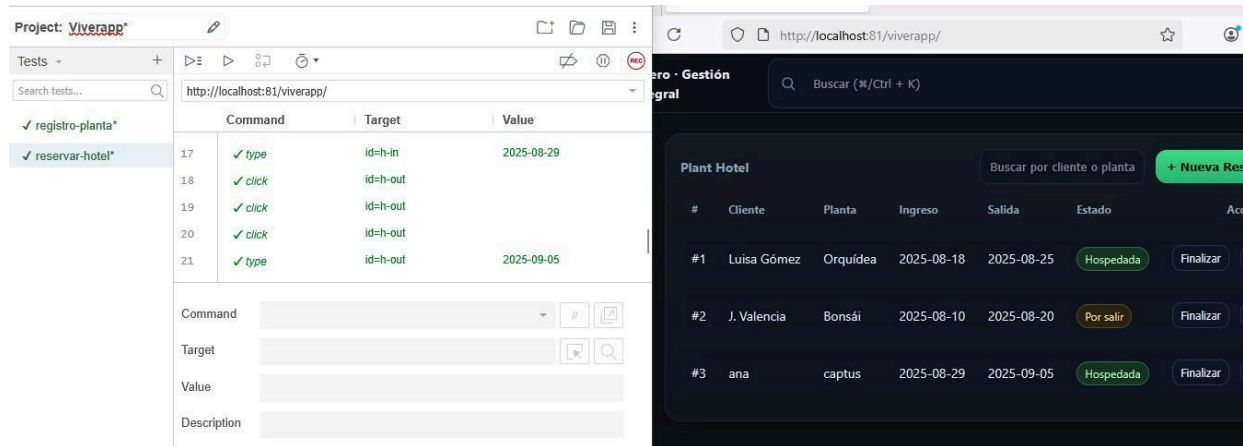
#	Producto	Precio	Stock	Estado	Acciones
#1	Helecho	US\$ 12,00	5	OK	Editar +1 -1
#2	Cactus	US\$ 7,50	2	Bajo	Editar +1 -1
#3	Orquídea	US\$ 25,00	12	OK	Editar +1 -1
#4	Maceta cerámica	US\$ 9,50	1	Bajo	Editar +1 -1
#5	Flor	US\$ 100,00	12	OK	Editar +1 -1

Por otra parte, un proceso misional del vivero es la reserva del hotel de plantas. Para ello la prueba funcional se enfoca en este procedimiento.

En la Figura 38, se realiza la prueba funcional de la reserva del hotel de plantas.

Figura 38

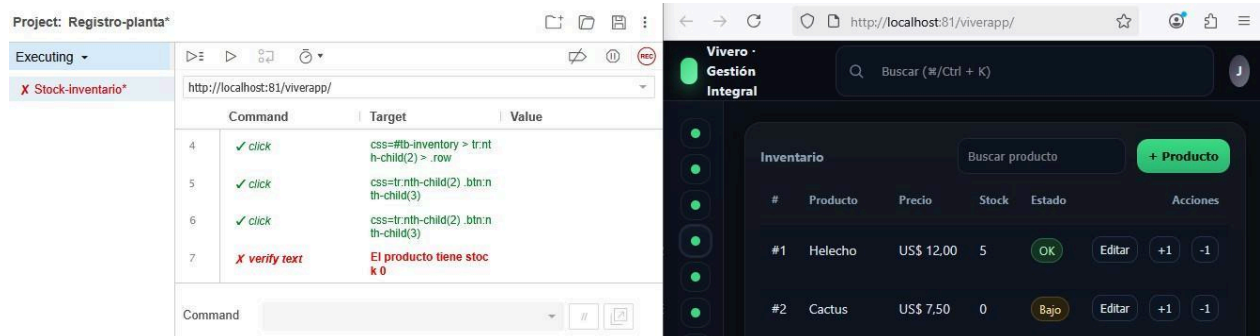
Prueba reserva hotel de plantas



En la Figura 39 se muestra la prueba funcional sobre las alertas de las cantidades del stock de las plantas (productos)

Figura 39

Prueba de stock sin productos



Corrección de Errores, y Documentación de Estos

Durante la fase de pruebas se identificó un fallo principal:

- No se muestra alguna alerta cuando el stock se encuentra en 0.

Correcciones Realizadas

Corrección 1: Revisión del código

Antes (con error):

```
<script>
document.addEventListener("DOMContentLoaded", () => {
  const tabla = document.getElementById("tb-low-stock"); // ❌ Error: paréntesis sin cerrar
}
```

Después (corregido):

```
<script>
document.addEventListener("DOMContentLoaded", () => {
  const tabla = document.getElementById("tb-low-stock");
}
```

En la Figura 40, se observa la verificación al integrar la corrección.

Figura 40

Corrección prueba alerta stock

The image shows a web application interface on the right and a test runner on the left. The web application, titled 'Vivero - Gestión Integral', displays a table of products with their stock levels. A modal alert is shown over the table, indicating a low stock condition for a product.

#	Producto	Precio	Stock	Estado	Acciones
#1	Helecho	US\$ 12,00	5	OK	Editar +1 -1
#2	Cactus	US\$ 7,50	0	Bajo	Editar +1 -1
#3	Orquídea	US\$ 10,00	1	OK	Editar +1 -1
#4	Maceta cerámica	US\$ 5,00	1	OK	Editar +1 -1

The test runner on the left shows a list of tests under the project 'Registro-planta'. The test 'Stock-mensaje*' is selected, and its details are shown below. The test steps include clicking on buttons and asserting an alert message.

Command	Target	Value
click	css=tr:nth-child(2).btn:nth-child(3)	
click	css=tr:nth-child(2).btn:nth-child(3)	
assert alert	El producto tiene stock 0	

The log at the bottom shows the test results, indicating that the 'Stock-mensaje*' test completed successfully.

Plan de Despliegue

Este plan define cómo se pondrá en producción el sistema, qué recursos se necesitan y cómo se garantiza su disponibilidad y seguridad.

Incluye la instalación en el servidor, configuración del dominio, pruebas finales y monitoreo inicial para asegurar un uso correcto.

Hosting

El hosting es el servicio que permite almacenar y publicar sitios web en servidores conectados a Internet. Su función principal es garantizar que una página esté disponible en línea de forma segura, rápida y accesible las 24 horas.

- Proveedores sugeridos:
 - Hostinger (~5 USD/mes).
- Requerimientos:
 - PHP 8.x
 - MySQL 8.x
 - Soporte HTTPS (SSL).
 - Capacidad de escalabilidad (aumentar CPU/RAM según demanda).

Dominio y Subdominio

- Dominio principal:
 - www.viveroelparaíso.com
 - Costo estimado: 12 – 15 USD/año (Hostinger).
- Subdominios recomendados:
 - admin.viveroelparaíso.com para el panel administrativo.
 - clientes.viveroelparaíso.com para el portal clientes (pedidos, hotel plantas).

Costos estimados

Tabla 18

Costos estimados

Concepto	Escenario Básico (Hostinger compartido)	Escenario Escalable (VPS/Cloud)
Hosting (PHP 8.x, MySQL 8.x, soporte HTTPS, escalabilidad inicial)	5 USD/mes → 60 USD/año	10–25 USD/mes → 120–300 USD/año
Dominio (www.viveroelparaiso.com)	12–15 USD/año	12–15 USD/año
Subdominios (ej: admin., api.)	Incluidos → 0 USD	Incluidos → 0 USD
SSL (HTTPS)	Gratis (incluido en Hostinger)	Gratis o incluido en VPS
Soporte de escalabilidad	Limitado (mejorar plan manualmente)	Flexible (CPU/RAM bajo demanda)

Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Totales:

- Escenario Básico: ~ 75 USD/año
- Escenario Escalable: ~ 150 – 315 USD/año (según crecimiento del proyecto).

Lo anterior puede variar por la fecha en que se consulte los planes y costos relacionados que el proveedor de servicios de alojamiento y dominio ofrezca.

Implantación del Software

La implementación es el proceso mediante el cual un sistema, aplicación o solución tecnológica se lleva a la práctica y se pone en funcionamiento en el entorno real de uso.

Incluye actividades como la instalación, configuración, migración de datos, capacitación de usuarios y pruebas finales para asegurar que el software cumpla con los objetivos.

El plan de implantación de ViverApp contempla iniciar en un entorno de hosting compartido económico con dominio propio y SSL, y evolucionar hacia un entorno escalable en VPS o nube conforme aumente el tráfico y la demanda de recursos, garantizando disponibilidad 24/7.

Herramientas para gestionar:

- Servidor Web Apache o Nginx.
- Gestión BD phpMyAdmin / MySQL Workbench.
- Control de versiones Git +

GitHub Plataforma de despliegue:

- Frontend: HTML5, CSS3, JS, Bootstrap.
- Backend: PHP 8.x
- Base de datos: MySQL 8.x.

Servicios adicionales:

- PHPMailer para notificaciones por correo.
- API de Google Maps para entregas domiciliarias.
- MercadoPago / PayPal para pagos online.

Seguridad:

- SSL (HTTPS obligatorio)
- Firewall en servidor
- Roles y permisos en el sistema (admin, vendedor, cliente).

- Backups automáticos diarios de base de datos y archivos.
- Política de contraseñas fuertes y encriptación.
- Protección contra inyecciones SQL (uso de PDO con consultas preparadas).

Plan de Capacitación de Usuarios del Sistema

El objetivo es garantizar que los usuarios finales adquieran los conocimientos y habilidades necesarias para utilizar ViverApp de forma correcta y eficiente, optimizando sus procesos en el vivero.

Manual del Usuario

ViverApp es una aplicación web diseñada para la gestión integral de un vivero, facilitando el control de inventario de plantas, ventas, reservas y reportes. Este manual guía a los usuarios en el uso correcto de las funciones del sistema.

Acceso al Sistema

1. Abra el navegador web (Google Chrome recomendado).
2. Ingrese la dirección: www.viveroelparaíso.com
3. Escriba su usuario y contraseña en el formulario de inicio de sesión.
4. Haga clic en Iniciar Sesión.

Figura 41

Guía para iniciar sesión



Nota. Elaborada con la ayuda de ChatGPT (OpenAI, 2025).

Si olvidó su contraseña, use la opción "Recuperar contraseña".

Módulos del Sistema

Panel Principal:

- Vista general de estadísticas: ventas, inventario, reservas.
- Acceso rápido a los módulos principales.

Gestión de Inventario:

- Agregar nueva planta: Menú → Inventario → Agregar.
- Editar información de planta: Seleccione la planta y haga clic en Editar.
- Eliminar planta: Seleccione y confirme con Eliminar.
- Consultar stock disponible.

Gestión de Ventas:

- Crear nueva venta: Menú → Ventas → Nueva Venta.
- Registrar datos del cliente y productos.
- Generar factura en

PDF. Gestión de Reservas:

- Crear reserva de planta para un cliente.
- Consultar y modificar reservas activas.
- Confirmar o cancelar reservas.

Gestión de Usuarios (solo administradores):

- Crear y asignar roles (administrador, empleado, cliente).
- Bloquear o desbloquear usuarios.
- Actualizar información de perfil.

Reportes:

- Generar reportes de ventas, inventario y reservas.

- Exportar datos a Excel o

PDF. Funcionalidades

adicionales:

- Búsqueda avanzada: localizar plantas o clientes por nombre o ID.
- Notificaciones: alertas de bajo stock o nuevas reservas.
- Seguridad: acceso protegido con HTTPS y roles de usuario.

Recomendaciones de uso:

- Mantenga actualizada su contraseña.
- Cierre sesión al terminar para proteger su cuenta.
- Revise regularmente los reportes para mantener el control del vivero.

Soporte técnico:

En caso de problemas técnicos, contacte al equipo de soporte:

Correo: soporte@viveroelparaiso.com

Teléfono: +57 300 123 4567

Horario: lunes a viernes, 8:00 a.m. – 6:00 p.m.

El manual de usuario completo de ViverApp se encuentra disponible en el anexo archivo adjunto, donde se detallan las instrucciones paso a paso para el correcto uso del sistema.

Copias de Seguridad de Datos y Respaldos

Tienen como objetivo garantizar la protección de la información almacenada en el sistema (ventas, inventario, clientes y usuarios), evitando pérdida de datos por fallos técnicos, errores humanos o ciberataques.

Tipos de Copias de Seguridad

1. Copia completa: respaldo total de la base de datos y archivos del sistema.
2. Copia incremental: guarda solo los cambios realizados desde la última copia.
3. Copia diferencial: almacena los cambios realizados desde la última copia completa.

Frecuencia de Respaldos

- Diaria: base de datos MySQL (ventas, inventario, usuarios).
- Semanal: archivos del sistema (código, imágenes, configuraciones).
- Mensual: respaldo completo externo en la nube o disco duro externo.

Medios de Almacenamiento

- Servidor principal (Hostinger/VPS): copia automática diaria.
- Almacenamiento en la nube: Google Drive
- Disco externo: copias mensuales para recuperación en caso de desastres.

Política de Retención

- Conservar copias diarias por 7 días,
- Copias semanales por 1 mes,
- Copias mensuales por 6 meses.

Restauración de Datos

- Acceder al panel de control del hosting o VPS.
- Seleccionar la copia más reciente.
- Restaurar base de datos y archivos del sistema en caso de fallo.

ViverApp contará con un plan de copias de seguridad mixto (diario, semanal y mensual), utilizando almacenamiento en el servidor, la nube y dispositivos externos, asegurando la disponibilidad y recuperación confiable de datos.

Acuerdos de Nivel de Usuario

Los Acuerdos de Nivel de Usuario de ViverApp establecen los compromisos de disponibilidad, seguridad, soporte y responsabilidades tanto del proveedor del servicio como de los usuarios, asegurando un uso confiable y eficiente de la aplicación.

Disponibilidad del servicio:

- El sistema estará disponible 24/7, salvo mantenimientos programados previamente notificados.
- En caso de caídas no programadas, se garantiza la restauración del servicio en un plazo máximo de 4 horas.

Acceso y uso del sistema:

- Cada usuario tendrá un usuario y contraseña personal que no debe compartirse.
- Los accesos estarán protegidos mediante HTTPS y roles de seguridad (administrador, empleado, cliente).
- El uso indebido (compartir credenciales, manipular datos) implicará sanciones o bloqueo de la cuenta.

Soporte técnico:

- El equipo de soporte estará disponible de lunes a viernes de 8:00 a.m. a 6:00 p.m.
- Canales de contacto: correo electrónico y teléfono.
- Tiempo de respuesta promedio: 24 horas hábiles.

Seguridad y privacidad de datos:

Los datos de usuarios y clientes serán protegidos conforme a la política de privacidad y ley de protección de datos.

El sistema implementará copias de seguridad automáticas para garantizar la recuperación de información en caso de pérdida.

Actualizaciones y mantenimiento:

- Se realizarán actualizaciones periódicas para mejorar la seguridad y funcionalidad del sistema.
- Los mantenimientos serán notificados con mínimo 24 horas de anticipación.

Responsabilidades del usuario:

- Usar el sistema de manera correcta y conforme a los lineamientos establecidos.
- Mantener la confidencialidad de sus credenciales de acceso.
- Reportar fallos, errores o anomalías al equipo de soporte técnico.

Los aprendices que desarrollaron la aplicación desde el momento que el software este en producción realizarán el mantenimiento de este hasta máximo 1 mes.

Adopción de Buenas Prácticas en el Proceso de Desarrollo de Software

La adopción de buenas prácticas en el proceso de desarrollo de software para ViverApp asegura la creación de un sistema confiable, seguro y fácil de mantener.

Estas prácticas permiten optimizar la calidad del producto, mejorar la colaboración del equipo y garantizar que la aplicación cumpla con los requerimientos y expectativas de los usuarios finales.

Gestión de requisitos:

- Documentar y validar los requerimientos funcionales y no funcionales antes de iniciar el desarrollo.
- Mantener trazabilidad de cambios y actualizaciones solicitadas por el cliente.

Control de versiones:

- Uso de GitHub/GitLab para administrar versiones del código fuente.
- Establecimiento de ramas (main, develop, feature) para trabajo colaborativo ordenado.

Metodología de desarrollo:

- Aplicar metodología ágil (Scrum/Kanban) para iteraciones rápidas y entregas incrementales.
- Reuniones periódicas de seguimiento para ajustar tareas y prioridades.

Buenas prácticas de programación:

- Uso de patrón MVC en PHP para separar lógica, datos y presentación.
- Codificación estandarizada con guías de estilo (PHP).
- Inclusión de comentarios claros y significativos en el código.

Pruebas de calidad:

- Implementar pruebas unitarias para funciones críticas.
- Realizar pruebas de integración y aceptación antes de cada despliegue.
- Uso de entornos separados: desarrollo, pruebas y producción.

Seguridad:

- Validación de entradas de usuario para evitar inyecciones SQL.
- Uso obligatorio de HTTPS y cifrado de contraseñas con algoritmos seguros (bcrypt).
- Gestión de roles y permisos de acceso diferenciados.

Automatización y despliegue:

- Respallos automáticos antes de cada actualización en producción.

Documentación y capacitación:

- Documentar código, base de datos y manuales de usuario.
- Capacitar al equipo y a los usuarios finales para un correcto uso del sistema.

Conclusión

ViverApp se presenta como una plataforma web moderna e innovadora que responde a las necesidades actuales de gestión en un vivero. Su desarrollo bajo estándares tecnológicos como PHP 8.x, MySQL 8.x y soporte HTTPS, permite un funcionamiento seguro, estable y disponible las 24 horas del día, lo cual garantiza confiabilidad en las operaciones diarias.

El sistema facilita el control de inventario, la gestión de ventas, reservas y generación de reportes, ofreciendo así una herramienta integral para administradores y empleados. Asimismo, la interfaz amigable y el acceso en línea permiten que los clientes también interactúen con la plataforma, fortaleciendo la relación comercial y optimizando la experiencia de usuario.

Durante su desarrollo, ViverApp adoptó buenas prácticas de desarrollo de software, incluyendo control de versiones, metodología ágil, pruebas de calidad y medidas de seguridad, lo que asegura que la aplicación no solo cumpla con los requerimientos actuales, sino que también pueda escalar y adaptarse al crecimiento del negocio. Además, la integración de copias de seguridad automáticas y políticas de respaldo refuerzan la protección de los datos, garantizando su disponibilidad ante cualquier eventualidad.

En conclusión, ViverApp no es únicamente un sistema de gestión, sino una herramienta estratégica que impulsa la transformación digital en los viveros, aportando eficiencia, organización y sostenibilidad. Con su implementación, se abre la posibilidad de optimizar recursos, mejorar la toma de decisiones y ofrecer un servicio más competitivo en el mercado, consolidando así un modelo de gestión tecnológico que asegura la continuidad y el crecimiento del negocio en el mediano y largo plazo.

Glosario

Tabla 19

Definición de términos para Viverapp

Término	Definición
Administradores	Usuarios con permisos completos para gestionar todo el sistema, incluyendo inventario, ventas, usuarios y reportes.
API	Conjunto de funciones que permiten la integración de ViverApp con otros sistemas o servicios externos.
Base de Datos	Estructura organizada (en este caso MySQL) donde se almacenan los datos de plantas, clientes, ventas y reservas.
Backup (Copia de Seguridad)	Respaldo de información que garantiza la recuperación de los datos en caso de fallas o pérdida.
Cliente	Persona que adquiere plantas o servicios a través de ViverApp, ya sea en tienda física o en línea.
Credenciales de acceso	Usuario y contraseña que permiten a cada persona ingresar al sistema de forma segura.
Dashboard (Panel de control)	Vista principal del sistema donde se muestran estadísticas, reportes y accesos rápidos a los módulos.
Hosting	Servicio en la nube que aloja ViverApp para que esté disponible en internet las 24 horas.
Inventario	Registro detallado de las plantas disponibles, su cantidad, características y estado en el vivero.
Módulos	Secciones específicas del sistema como Inventario, Ventas, Reservas, Usuarios y Reportes.
Reporte	Documento generado automáticamente por ViverApp que muestra datos de ventas, inventario o reservas, exportable a PDF o Excel.
Reserva	Funcionalidad que permite apartar plantas para un cliente en una fecha determinada.
Rol de usuario	Nivel de permisos que tiene cada usuario en el sistema (Administrador, Empleado, Cliente).

Término	Definición
SSL (Secure Sockets Layer)	Certificado que protege la comunicación en ViverApp asegurando la transmisión cifrada de los datos.
Usuario final	Persona que utiliza el sistema en sus diferentes roles para realizar operaciones.
Ventas	Proceso mediante el cual se registra la compra de plantas o productos dentro de ViverApp.

Bibliografía

OpenAI. (2025, agosto 19). *Respuesta generada por ChatGPT sobre técnicas de recolección de información (entrevista, encuesta y observación) en un vivero* [Comunicación personal con modelo de lenguaje IA]. OpenAI ChatGPT. <https://chat.openai.com/>

OpenAI. (2025, agosto 19). *Organización de la información recolectada en entrevista, encuesta y observación en un vivero* [Comunicación personal con modelo de lenguaje IA]. OpenAI ChatGPT. <https://chat.openai.com/>

OpenAI. (2025, agosto 19). *Descripción breve de las técnicas de recolección de información utilizadas en investigación aplicada* [Comunicación personal con modelo de lenguaje IA]. OpenAI ChatGPT. <https://chat.openai.com/>

Apéndices

Apéndice A

Script del Front

```
<script>
  // --- Simple reactive store (mock data)
  ---const store = {
    orders:[
      {id:101, customer:'Ana López', date:'2025-08-17', status:'En curso',
total:58.80},
      {id:102, customer:'Carlos Ruiz', date:'2025-08-18', status:'Pendiente',
total:23.50},
      {id:103, customer:'María Pérez', date:'2025-08-18', status:'Entregado',
total:74.20},
    ],
    sales:[
      {id:1, product:'Suculenta', qty:2, price:6.50},
      {id:2, product:'Helecho', qty:1, price:12.00},
    ],
    inventory:[
      {id:1, name:'Helecho', price:12.00, stock:5},
      {id:2, name:'Cactus', price:7.50, stock:2},
      {id:3, name:'Orquídea', price:25.00, stock:12},
      {id:4, name:'Maceta cerámica', price:9.50, stock:1},
    ],
    hotel:[
      {id:1, customer:'Luisa Gómez', plant:'Orquídea', in:'2025-08-18',
out:'2025-08-25', status:'Hospedada'},
      {id:2, customer:'J. Valencia', plant:'Bonsái', in:'2025-08-10',
out:'2025-08-20', status:'Por salir'},
    ],
    users:[
      {id:1, name:'Admin', email:'admin@vivero.com', role:'Administrador'},
      {id:2, name:'Jessie', email:'jessie@vivero.com', role:'Empleado'},
    ]
  };

  // --- Utilities ---
```



```

const qs = s => document.querySelector(s);
const qsa = s => Array.from(document.querySelectorAll(s));
const money = n => n.toLocaleString('es-CO',{style:'currency',
currency:'USD'}));

function toast(msg){ const
  el = qs('#toast');
  el.textContent = msg; el.style.opacity = 1; el.style.transform =
'translateX(-50%) scale(1)';
  setTimeout(()=>{el.style.opacity = 0; el.style.transform =
'translateX(-50%) scale(0.98)';}, 1800);
}

// --- Navigation ---
qsa('#nav button').forEach(btn=>{
  btn.addEventListener('click',()=>{
    qsa('#nav button').forEach(b=>b.classList.remove('active'));
    btn.classList.add('active');
    const view = btn.dataset.view;
    qsa('.view').forEach(v=>v.classList.remove('active'));
    qs('#view-'+view).classList.add('active');
  })
});

// --- Render helpers
---function renderKPIs(){
  const salesToday = store.sales.reduce((acc,s)=> acc + s.qty * s.price,0);
  const inProgress = store.orders.filter(o=>['En
curso','Pendiente'].includes(o.status)).length;
  const hotelCount = store.hotel.filter(h=>['Hospedada','Por
salir'].includes(h.status)).length;
  const low = store.inventory.filter(p=>p.stock<=2).length;
  qs('#kpi-sales').textContent = money(salesToday);
  qs('#kpi-orders').textContent = inProgress;
  qs('#kpi-hotel').textContent = hotelCount;
  qs('#kpi-low').textContent = low;

// dashboard tables
const lastOrders = store.orders.slice(-5).reverse().map(o=>

```

```

    `<tr><td>#${o.id}</td><td>${o.customer}</td><td>${o.date}</td><td><span
class="badge
${o.status=== 'Entregado'? 'ok': o.status=== 'Pendiente'? 'warn': ''}>${o.status}<
/td><td>${money(o.total)}</td></tr>`
    ).join('');
    qs('#tb-last-orders').innerHTML = lastOrders || `<tr><td colspan="5">Sin
datos</td></tr>`;

    const lowRows = store.inventory.filter(p=>p.stock<=2).map(p=>
    `<tr><td>${p.name}</td><td>${p.stock}</td><td class="row right"><button
class="btn small" onclick="openModal('Reponer
stock', '${p.name}')">Reponer</button></td></tr>`
    ).join('');
    qs('#tb-low-stock').innerHTML = lowRows || `<tr><td colspan="3">Stock
OK</td></tr>`;
  }

  function renderOrders(filter='') {
    const rows =
    store.orders.filter(o=>
      (o.customer+o.status).toLowerCase().includes(filter.toLowerCase())
    ).map(o=>`
      <tr>
        <td>#${o.id}</td>
        <td>${o.customer}</td>
        <td>${o.date}</td>
        <td><span class="badge
${o.status=== 'Entregado'? 'ok': o.status=== 'Pendiente'? 'warn': 'danger'}">${o.st
atus}</span></td>
        <td>${money(o.total)}</td>
        <td class="row right">
          <button class="btn small"
onclick="markDelivered(${o.id})">Entregar</button>
          <button class="btn small" onclick="openOrder(${o.id})">Ver</button>
        </td>
      </tr>`.join('');
    qs('#tb-orders').innerHTML = rows || `<tr><td colspan="6">No hay
pedidos</td></tr>`;
  }

```

```

function renderSales(){
  const rows = store.sales.map(s=>{
    const total = s.qty * s.price;
    return
`<tr><td>#${s.id}</td><td>${s.product}</td><td>${s.qty}</td><td>${money(s.pri
ce)}</td><td>${money(total)}</td></tr>`
  }).join('');
  qs('#tb-sales').innerHTML = rows || '<tr><td colspan="5">Sin
ventas</td></tr>';
}

function renderInventory(filter=''){
  const rows =
store.inventory.filter(p=>p.name.toLowerCase().includes(filter.toLowerCase()))
).map(p=>{
  const status = p.stock<=2? 'Bajo': 'OK';
  const badge = p.stock<=2? 'warn': 'ok';
  return `<tr>
    <td>#${p.id}</td><td>${p.name}</td><td>${money(p.price)}</td><td>${p.
stock}</td>
    <td><span class="badge ${badge}">${status}</span></td>
    <td class="row right">
      <button class="btn small"
onclick="editProduct(${p.id})">Editar</button>
      <button class="btn small"
onclick="adjustStock(${p.id},1)">+1</button>
      <button class="btn small" onclick="adjustStock(${p.id},-1)">-
1</button>
    </td>
  </tr>`
  }).join('');
  qs('#tb-inventory').innerHTML = rows || '<tr><td colspan="6">Sin
productos</td></tr>';
}

function renderHotel(filter=''){
  const rows = store.hotel.filter(h=>
    (h.customer+h.plant).toLowerCase().includes(filter.toLowerCase()))
).map(h=>`

```

```

    <tr>
      <td>#{h.id}</td><td>#{h.customer}</td><td>#{h.plant}</td><td>#{h.in}
</td><td>#{h.out}</td>
      <td><span class="badge #{h.status=== 'Hospedada'? 'ok': h.status=== 'Por
salir'? 'warn': 'danger'}">#{h.status}</span></td>
      <td class="row right">
        <button class="btn small"
onclick="finishStay(#{h.id})">Finalizar</button>
        <button class="btn small" onclick="openStay(#{h.id})">Ver</button>
      </td>
    </tr>`.join('');
    qs('#tb-hotel').innerHTML = rows || '<tr><td colspan="7">Sin
reservas</td></tr>';
  }

function renderUsers(filter='') {
  const rows = store.users.filter(u=>
    (u.name+u.email+u.role).toLowerCase().includes(filter.toLowerCase())
  ).map(u=>`
    <tr>
      <td>#{u.id}</td><td>#{u.name}</td><td>#{u.email}</td><td>#{u.role}</
td>
      <td class="row right">
        <button class="btn small"
onclick="editUser(#{u.id})">Editar</button>
        <button class="btn small"
onclick="removeUser(#{u.id})">Eliminar</button>
      </td>
    </tr>`.join('');
  qs('#tb-users').innerHTML = rows || '<tr><td colspan="5">Sin
usuarios</td></tr>';
}

// --- Actions ---
function markDelivered(id) {
  const o = store.orders.find(x=>x.id===id); if(!o) return;
  o.status = 'Entregado';
  renderOrders(qs('#ordersFilter').value);
  renderKPIs();
}

```

```

    toast('Pedido #' + id + ' marcado como entregado');
  }
  function openOrder(id) {
    const o = store.orders.find(x => x.id === id);
    openModal('Pedido #' + id, `
      <div class='grid-2'>
        <div class='card'><h3>Cliente</h3><p>${o.customer}</p></div>
        <div class='card'><h3>Estado</h3><p>${o.status}</p></div>
      </div>
    `);
  }

  function adjustStock(id, delta) {
    const p = store.inventory.find(x => x.id === id); if (!p) return;
    p.stock = Math.max(0, p.stock + delta);
    renderInventory(qs('#invSearch').value); renderKPIs();
  }

  function editProduct(id) {
    const p = store.inventory.find(x => x.id === id);
    openModal('Editar producto', `
      <form id='form-edit-product' class='grid-2'>
        <label>Nombre<br/><input id='ep-name' value='${p.name}' required
      /></label>
        <label>Precio<br/><input id='ep-price' type='number' step='0.01'
      min='0' value='${p.price}' required /></label>
        <label>Stock<br/><input id='ep-stock' type='number' min='0'
      value='${p.stock}' required /></label>
        <div class='row right' style='grid-column:1/-1'>
          <button class='btn ghost' type='button'
      onclick='closeModal()'>Cancelar</button>
          <button class='btn brand'>Guardar</button>
        </div>
      </form>
    `);
    qs('#form-edit-product').addEventListener('submit', e => {
      e.preventDefault();
      p.name = qs('#ep-name').value.trim(); p.price
      = parseFloat(qs('#ep-price').value); p.stock
      = parseInt(qs('#ep-stock').value, 10);
    });
  }

```

```

        closeModal(); renderInventory(qs('#invSearch').value); renderKPIs();
toast('Producto actualizado');
    });
}

function openStay(id){
    const it = store.hotel.find(x=>x.id===id);
    openModal('Reserva #' + id, `<div class='grid-2'>
        <div class='card'><h3>Cliente</h3><p>${it.customer}</p></div>
        <div class='card'><h3>Planta</h3><p>${it.plant}</p></div>
        <div class='card'><h3>Ingreso</h3><p>${it.in}</p></div>
        <div class='card'><h3>Salida</h3><p>${it.out}</p></div>
    </div>`);
}

function finishStay(id){
    const it = store.hotel.find(x=>x.id===id); if(!it) return;
    it.status = 'Finalizada';
    renderHotel(qs('#hotelSearch').value); renderKPIs(); toast('Estancia
finalizada');
}

function editUser(id){
    const u = store.users.find(x=>x.id===id);
    openModal('Editar usuario', `
        <form id='form-edit-user' class='grid-2'>
            <label>Nombre<br/><input id='eu-name' value='${u.name}' required
/></label>
            <label>Email<br/><input id='eu-email' type='email' value='${u.email}'
required /></label>
            <label>Rol<br/>
                <select id='eu-role'>
                    <option
${u.role==='Administrador'? 'selected':''}>Administrador</option>
                    <option ${u.role==='Empleado'? 'selected':''}>Empleado</option>
                </select>
            </label>
            <div class='row right' style='grid-column:1/-1'>
                <button class='btn ghost' type='button'
onclick='closeModal()'>Cancelar</button>

```

```

        <button class='btn brand'>Guardar</button>
    </div>
</form>
`);
qs('#form-edit-user').addEventListener('submit', e=>{
    e.preventDefault();
    u.name = qs('#eu-name').value.trim();
    u.email = qs('#eu-email').value.trim();
    u.role = qs('#eu-role').value;
    closeModal(); renderUsers(qs('#userSearch').value); toast('Usuario
actualizado');
});
}
function removeUser(id){
    const idx = store.users.findIndex(u=>u.id===id);
    if(idx>-1){ store.users.splice(idx,1);
renderUsers(qs('#userSearch').value); toast('Usuario eliminado'); }
}

// --- Modal ---
function openModal(title, bodyHTML){
    qs('#modalTitle').textContent = title;
    qs('#modalBody').innerHTML = bodyHTML;
    const b = qs('#modalBackdrop'); b.style.display='grid';
b.setAttribute('aria-hidden','false');
}
function closeModal(){
    const b = qs('#modalBackdrop'); b.style.display='none';
b.setAttribute('aria-hidden','true');
    qs('#modalBody').innerHTML = '';
}
qs('#btnCloseModal').addEventListener('click', closeModal);

// --- Forms & buttons
---qs('#ordersFilter').addEventListener('input',
t',
e=>renderOrders(e.target.value));
    qs('#invSearch').addEventListener('input',
e=>renderInventory(e.target.value));

```

```

    qs('#hotelSearch').addEventListener('input',
e=>renderHotel(e.target.value));
    qs('#userSearch').addEventListener('input',
e=>renderUsers(e.target.value));

    qs('#btnNewOrder').addEventListener('click',()=>{
        openModal('Nuevo Pedido', `
            <form id='form-order' class='grid-2'>
                <label>Cliente<br/><input id='o-customer' required /></label>
                <label>Total<br/><input id='o-total' type='number' step='0.01'
min='0' required /></label>
                <label>Estado<br/>
                    <select id='o-status'>
                        <option>Pendiente</option>
                        <option>En curso</option>
                        <option>Entregado</option>
                    </select>
                </label>
                <div class='row right' style='grid-column:1/-1'>
                    <button class='btn ghost' type='button'
onclick='closeModal()'>Cancelar</button>
                    <button class='btn brand'>Crear</button>
                </div>
            </form>
        `);
        qs('#form-order').addEventListener('submit', e=>{
            e.preventDefault();
            const id = Math.max(100, ...store.orders.map(o=>o.id)) + 1;
            store.orders.push({id, customer:qs('#o-customer').value.trim(),
date:new Date().toISOString().slice(0,10), status:qs('#o-status').value,
total:parseFloat(qs('#o-total').value)});
            closeModal(); renderOrders(qs('#ordersFilter').value); renderKPIs();
            toast('Pedido creado');
        })
    });

    qs('#btnAddProduct').addEventListener('click',()=>{
        openModal('Nuevo Producto', `
            <form id='form-product' class='grid-2'>

```



```

        <label>Nombre<br/><input id='p-name' required /></label>
        <label>Precio<br/><input id='p-price' type='number' step='0.01'
min='0' required /></label>
        <label>Stock<br/><input id='p-stock' type='number' min='0' required
/></label>
        <div class='row right' style='grid-column:1/-1'>
            <button class='btn ghost' type='button'
onclick='closeModal()'>Cancelar</button>
            <button class='btn brand'>Crear</button>
        </div>
    </form>
`);
qs('#form-product').addEventListener('submit', e=>{
    e.preventDefault();
    const id = (store.inventory.at(-1)?.id || 0) + 1;
    store.inventory.push({id,name:qs('#p-name').value.trim(),
price:parseFloat(qs('#p-price').value),
stock:parseInt(qs('#p-stock').value,10)});
    closeModal(); renderInventory(qs('#invSearch').value); renderKPIs();
    toast('Producto creado');
    })
});

qs('#btnNewStay').addEventListener('click',()=>{
    openModal('Nueva Reserva (Hotel de Plantas)', `
        <form id='form-stay' class='grid-2'>
            <label>Cliente<br/><input id='h-customer' required /></label>
            <label>Planta<br/><input id='h-plant' required /></label>
            <label>Ingreso<br/><input id='h-in' type='date' required /></label>
            <label>Salida<br/><input id='h-out' type='date' required /></label>
            <div class='row right' style='grid-column:1/-1'>
                <button class='btn ghost' type='button'
onclick='closeModal()'>Cancelar</button>
                <button class='btn brand'>Crear</button>
            </div>
        </form>
    `);
    qs('#form-stay').addEventListener('submit', e=>{
        e.preventDefault();

```

```

    const id = (store.hotel.at(-1)?.id || 0) + 1;
    store.hotel.push({id, customer:qs('#h-customer').value.trim(),
plant:qs('#h-plant').value.trim(), in:qs('#h-in').value,
out:qs('#h-out').value, status:'Hospedada'});
    closeModal(); renderHotel(qs('#hotelSearch').value); renderKPIs();
    toast('Reserva creada');
  })
});

qs('#btnNewUser').addEventListener('click',()=>{
  openModal('Nuevo Usuario', `
    <form id='form-user' class='grid-2'>
      <label>Nombre<br/><input id='u-name' required /></label>
      <label>Email<br/><input id='u-email' type='email' required /></label>
      <label>Rol<br/>
        <select id='u-role'>
          <option>Administrador</option>
          <option>Empleado</option>
        </select>
      </label>
      <div class='row right' style='grid-column:1/-1'>
        <button class='btn ghost' type='button'
onclick='closeModal()'>Cancelar</button>
        <button class='btn brand'>Crear</button>
      </div>
    </form>
  `);
  qs('#form-user').addEventListener('submit', e=>{
    e.preventDefault();
    const id = (store.users.at(-1)?.id || 0) + 1;
    store.users.push({id, name:qs('#u-name').value.trim(), email:qs('#u-
email').value.trim(), role:qs('#u-role').value});
    closeModal(); renderUsers(qs('#userSearch').value); toast('Usuario
creado');
  })
});

// Global quick search (Ctrl/Cmd+K)
document.addEventListener('keydown', e=>{

```

```

    if((e.ctrlKey||e.metaKey) && e.key.toLowerCase()=== 'k'){
      e.preventDefault(); qs('#globalSearch').focus();
    }
  });
  qs('#globalSearch').addEventListener('input', e=>{
    const v = e.target.value.toLowerCase();
    renderOrders(v); renderInventory(v); renderHotel(v); renderUsers(v);
  })

  // --- Sales form ---
  qs('#form-sale').addEventListener('submit', e=>{
    e.preventDefault();
    const id = (store.sales.at(-1)?.id || 0) + 1;
    const product = qs('#saleProduct').value.trim();
    const qty = parseInt(qs('#saleQty').value,10);
    const price = parseFloat(qs('#salePrice').value);
    store.sales.push({id, product, qty, price});
    // reduce inventory if exists
    const p =
store.inventory.find(x=>x.name.toLowerCase()===product.toLowerCase());
    if(p){ p.stock = Math.max(0, p.stock - qty); }
    renderSales(); renderInventory(qs('#invSearch').value); renderKPIs();
    e.target.reset();
    toast('Venta registrada');
  });

  // --- Initial render
  ---renderKPIs();
  renderOrders();
  renderSales();
  renderInventory();
  renderHotel();
  renderUsers();
</script>
</body>
</html>

```